

Badge of Shame

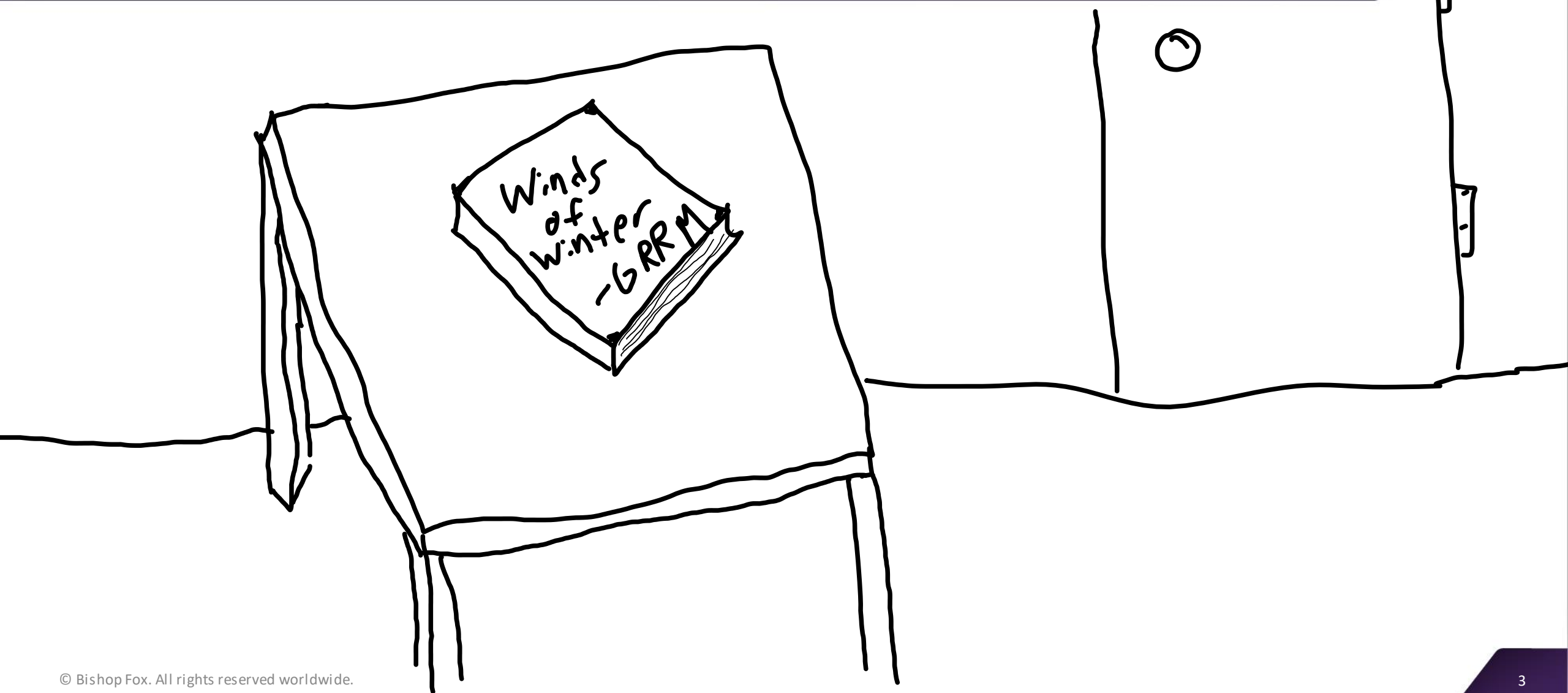
Breaking into Secure Facilities with
OSDP



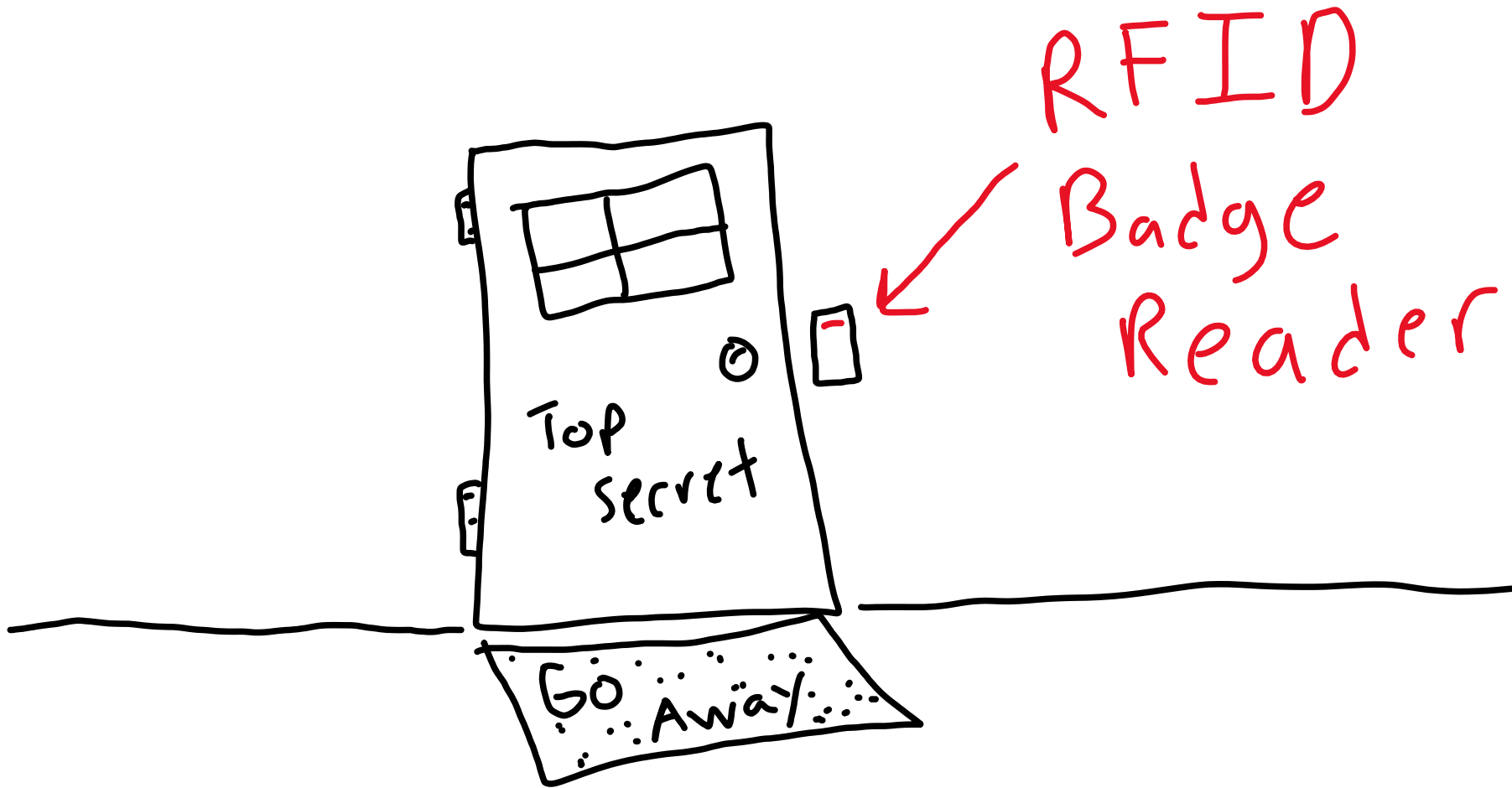
Secure Facility



Top Secret Materials







Other Ways to Hack RFID

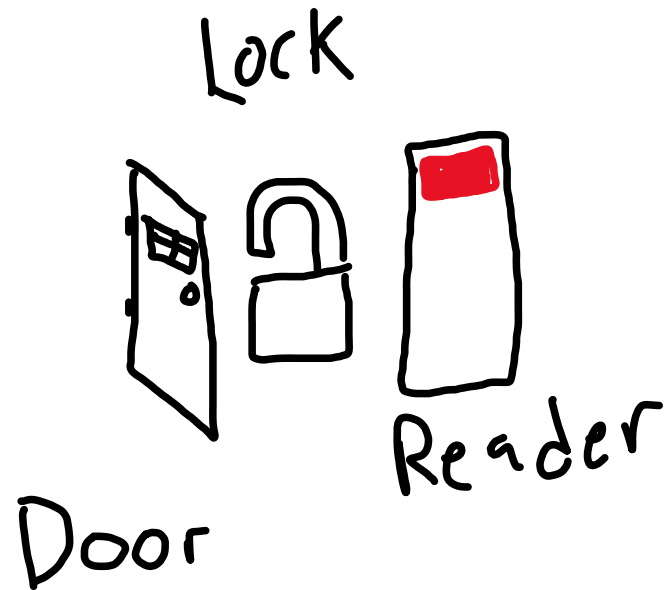


Tastic RFID
Thief

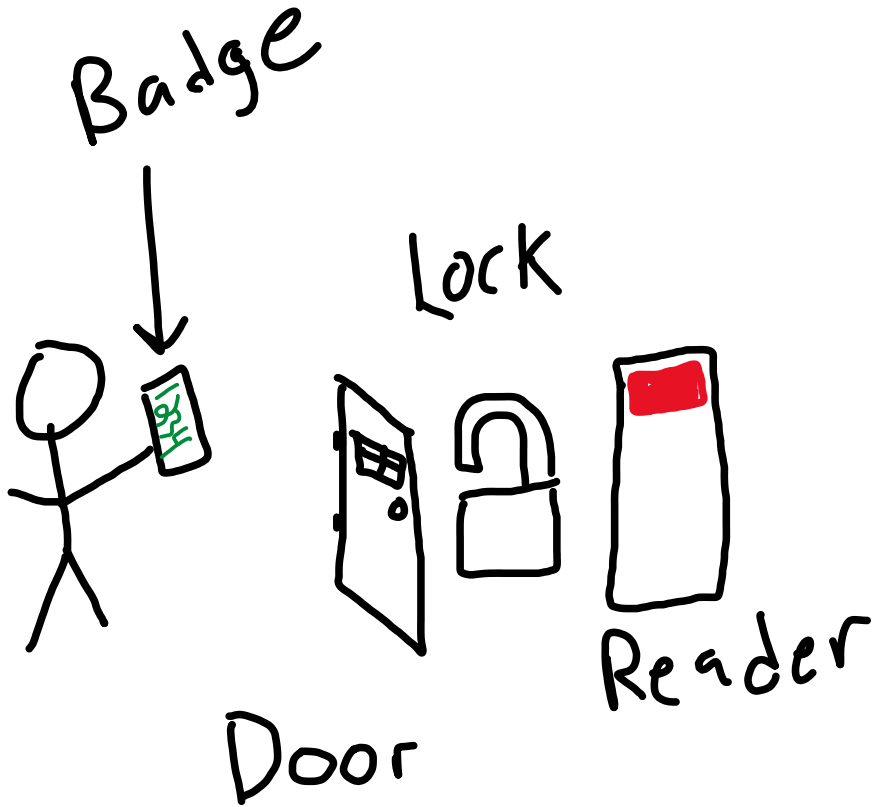


I made this! ↗

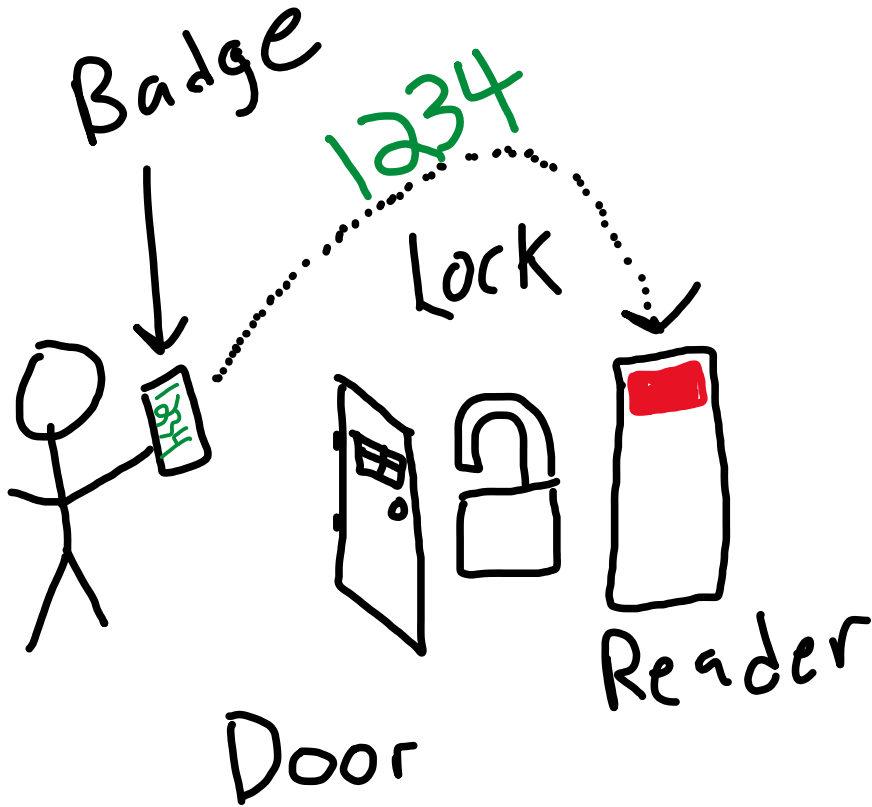
RFID Badge Setup



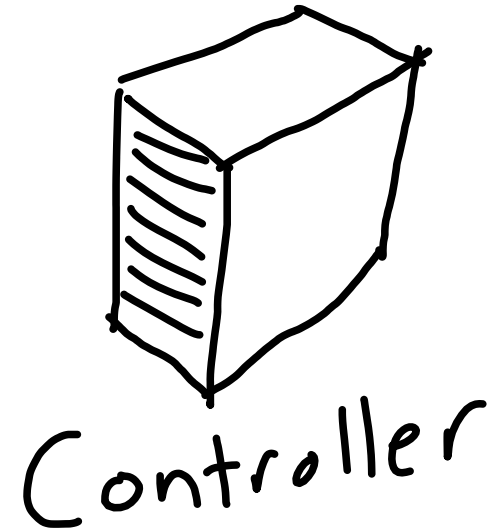
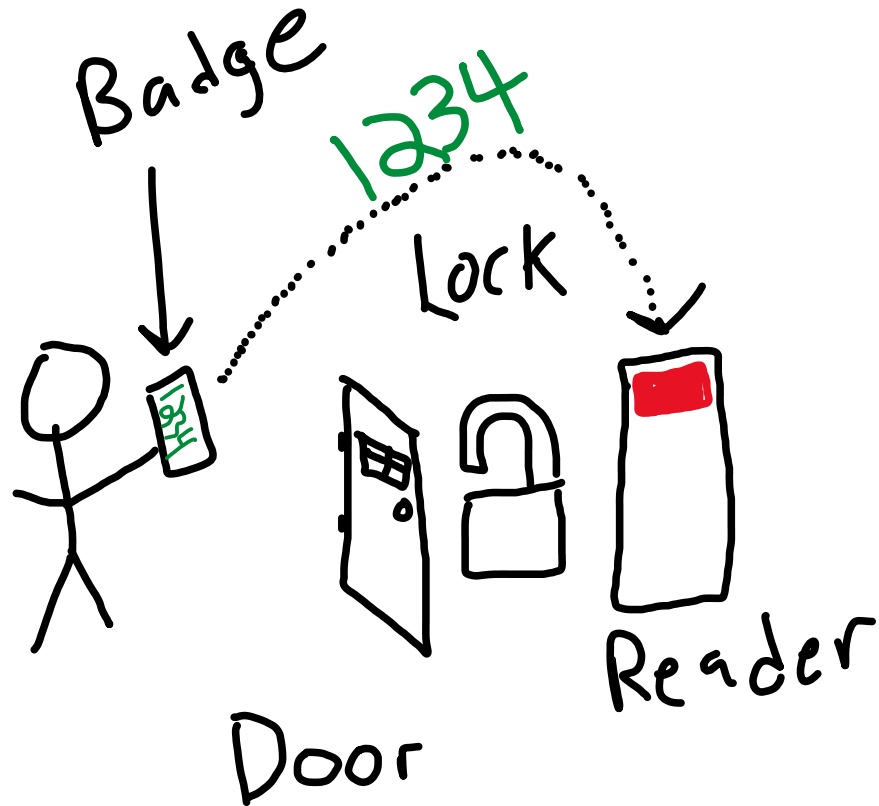
RFID Badge Setup



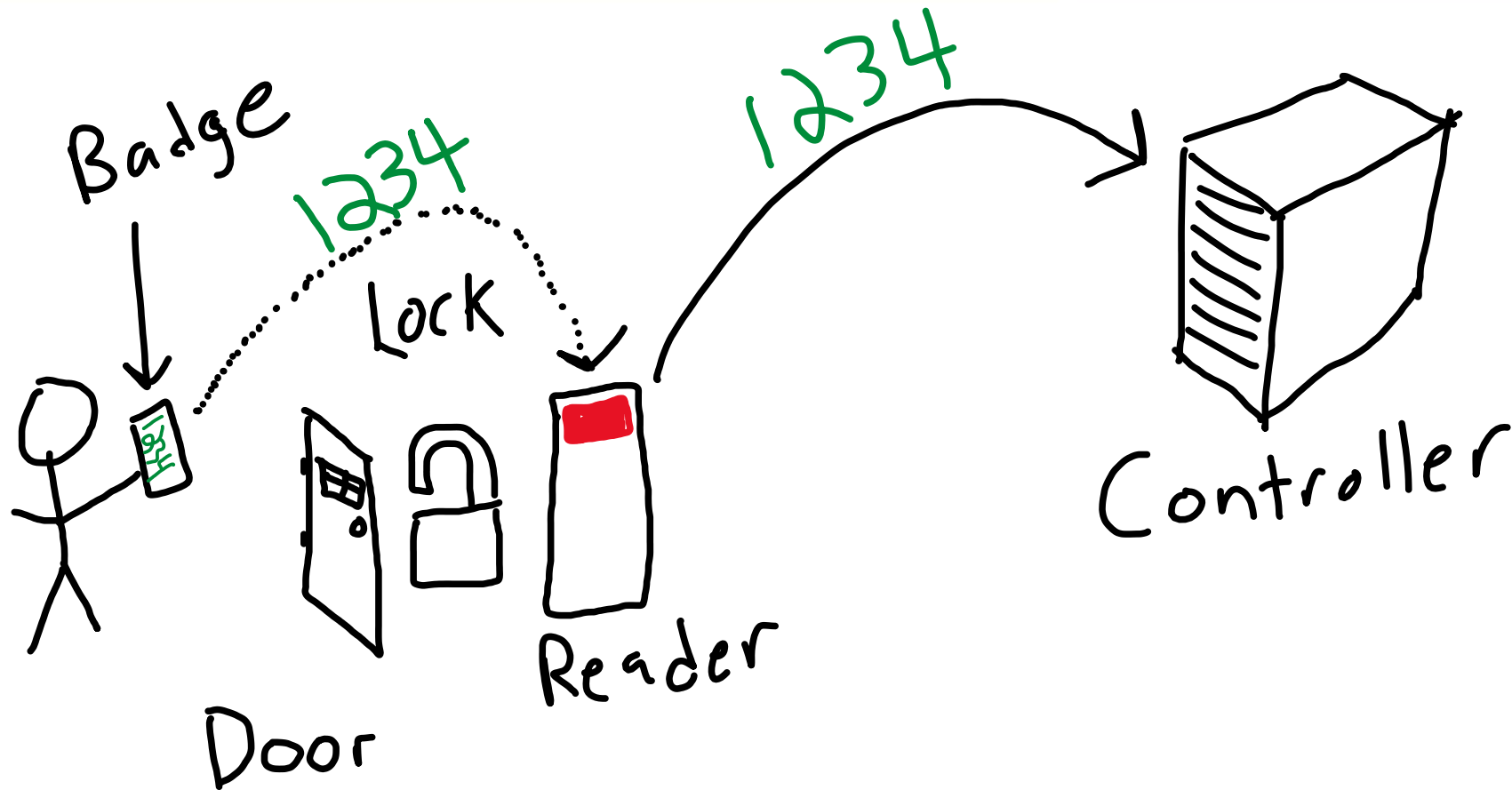
RFID Badge Setup



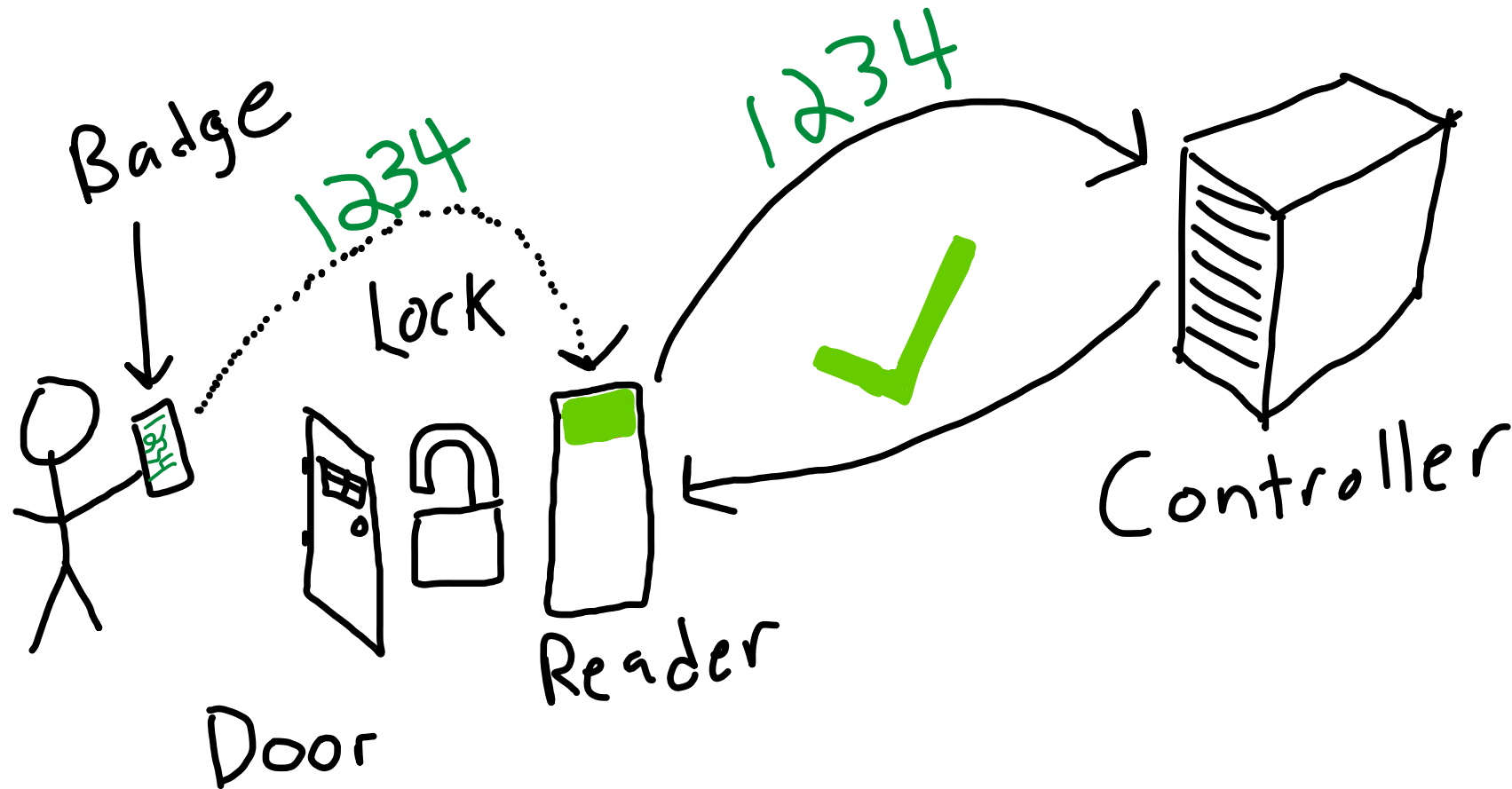
RFID Badge Setup



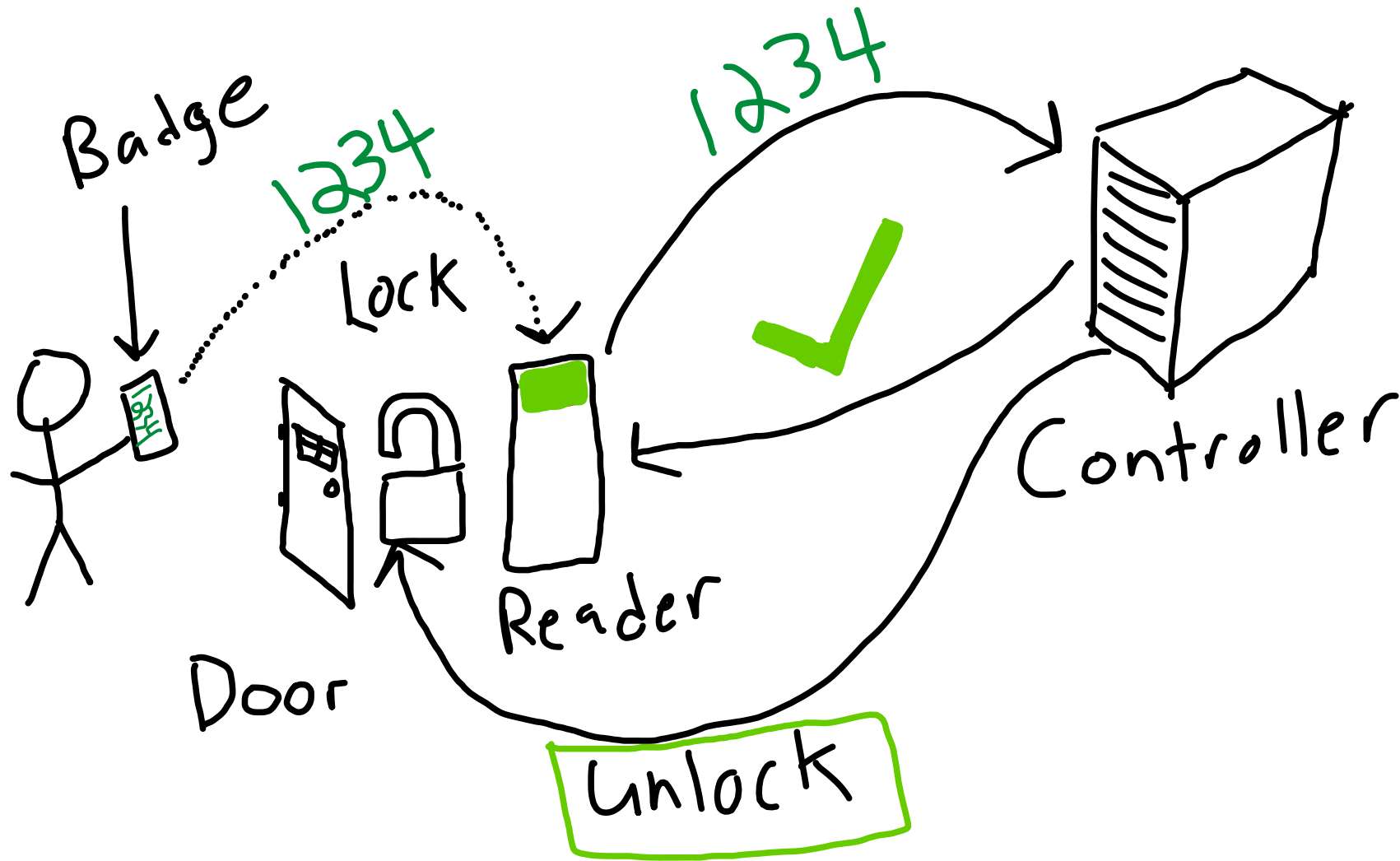
RFID Badge Setup



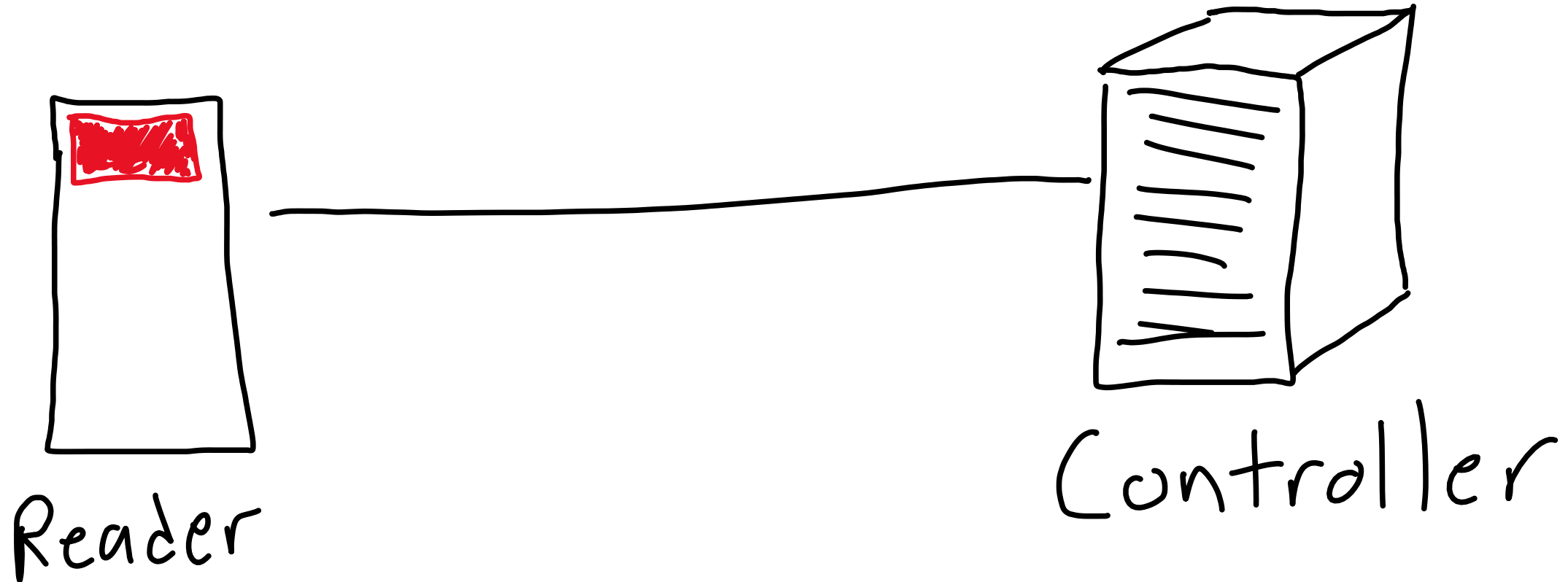
RFID Badge Setup



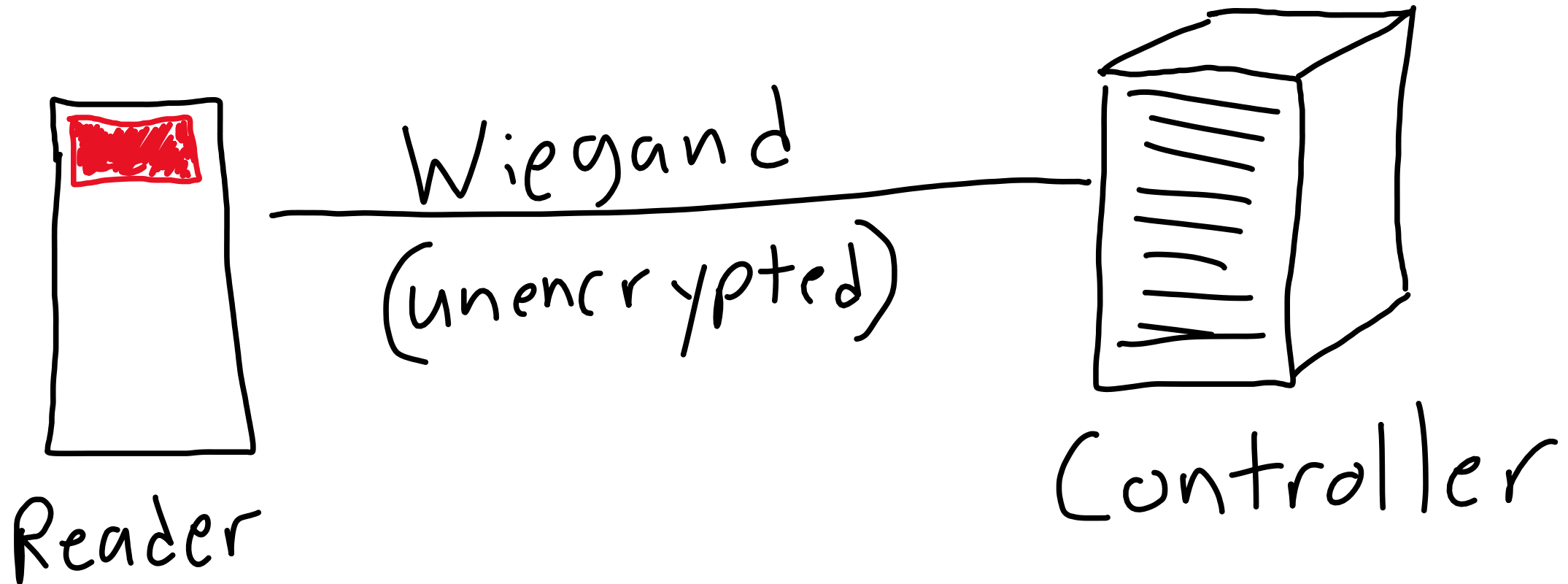
RFID Badge Setup



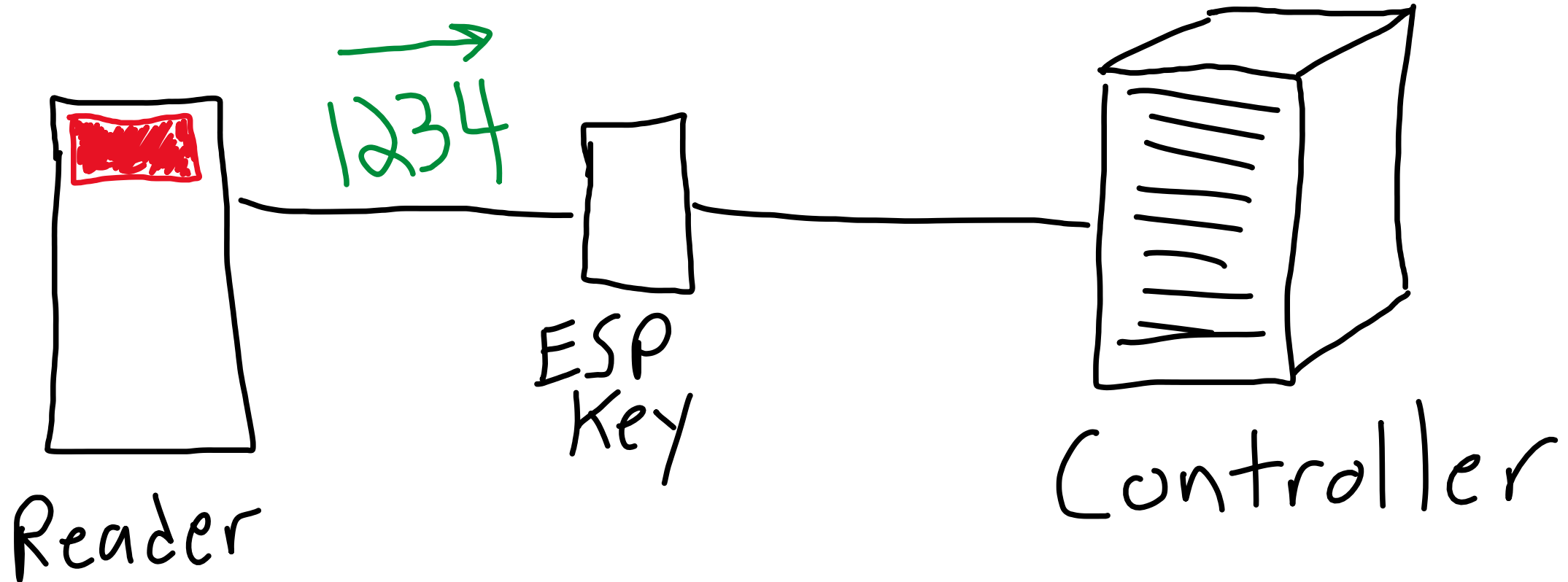
RFID Badge Setup



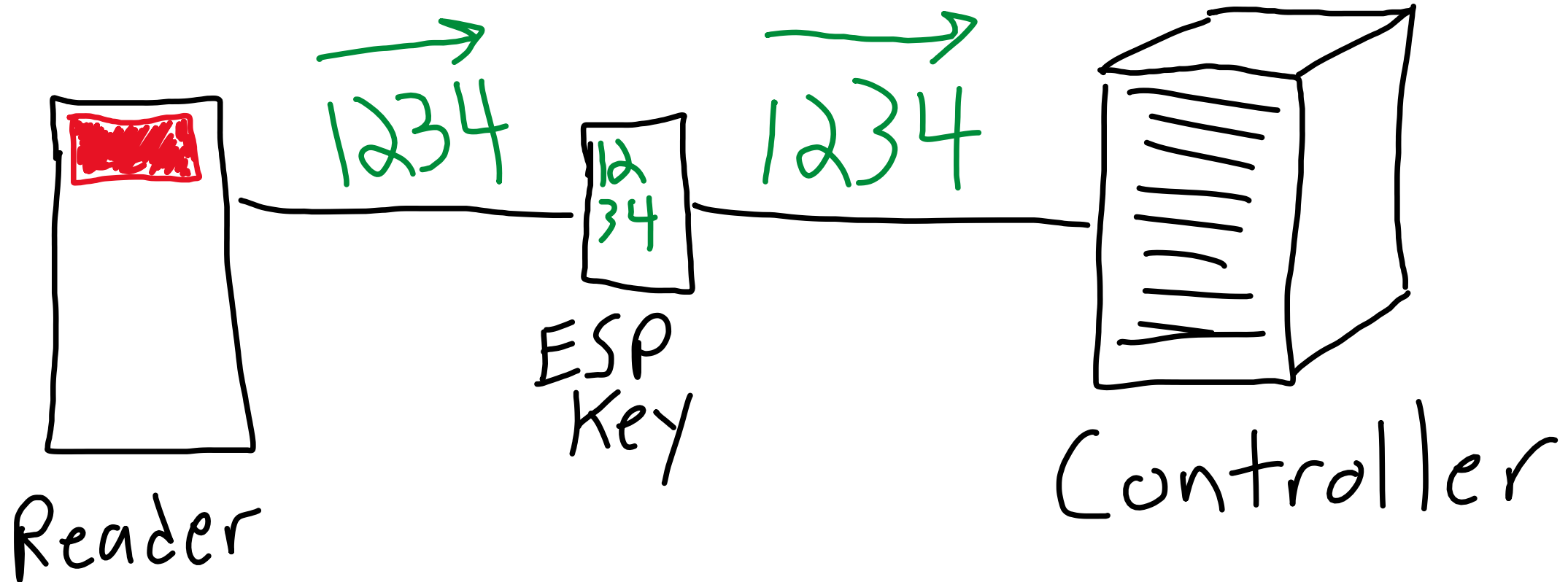
RFID Badge Setup



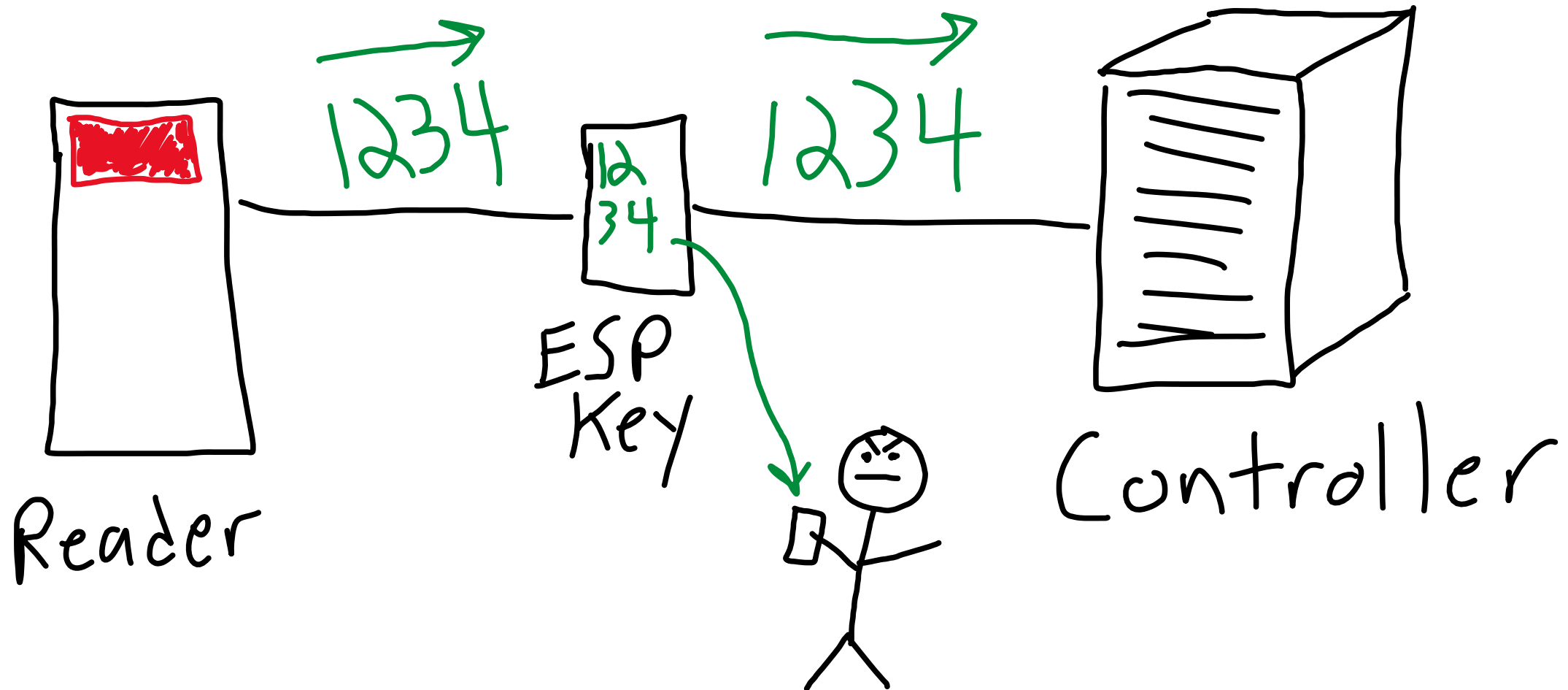
RFID Badge Setup



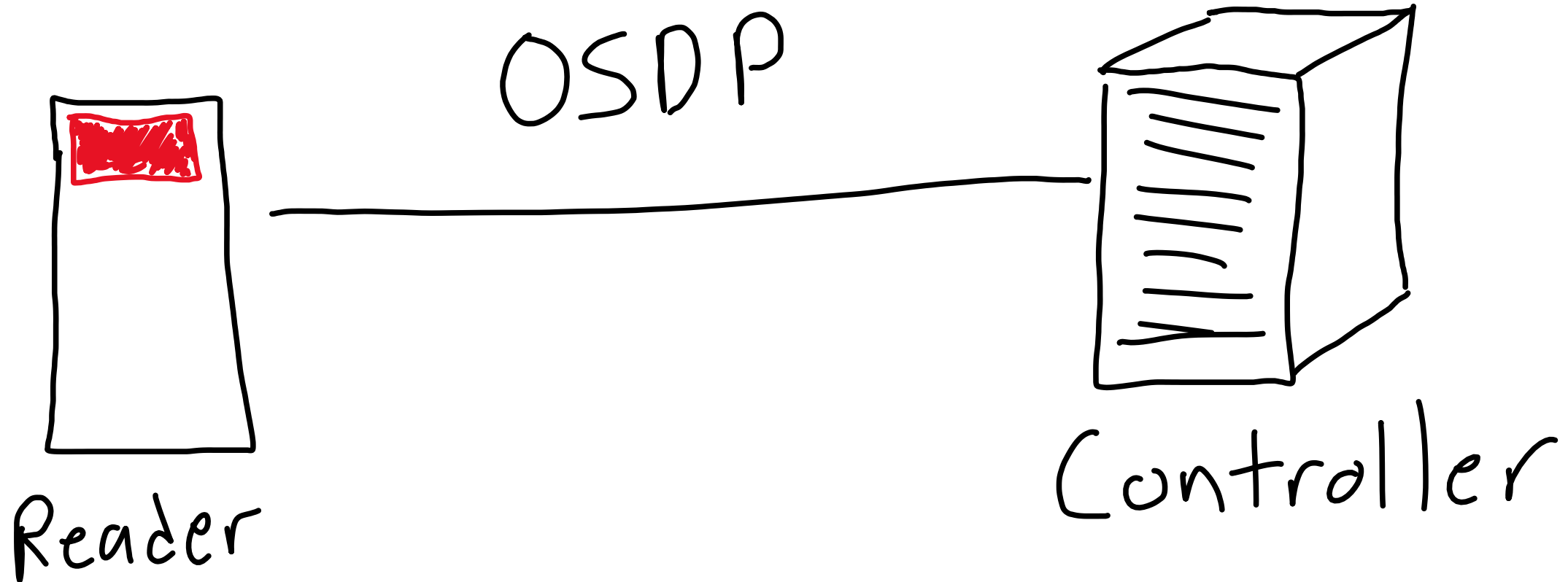
RFID Badge Setup

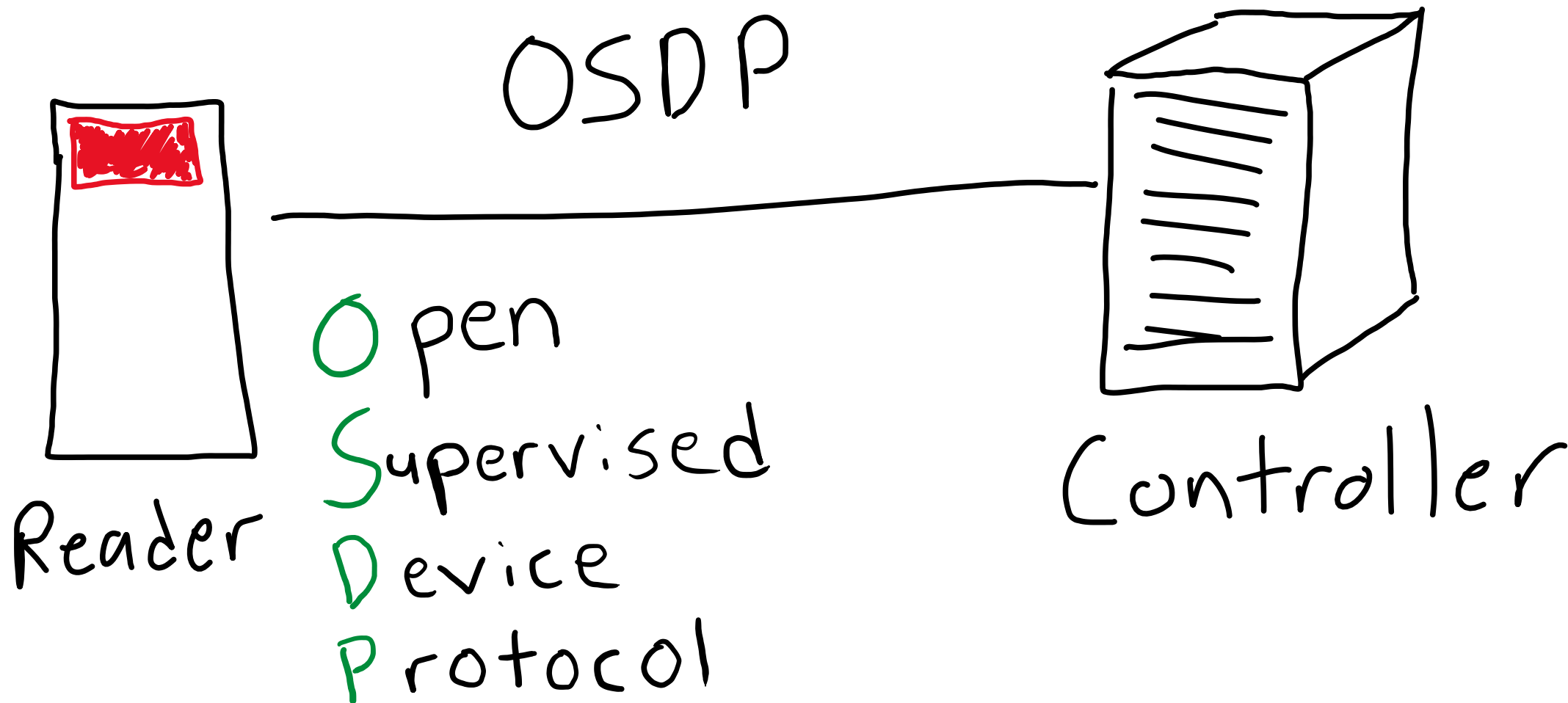


RFID Badge Setup

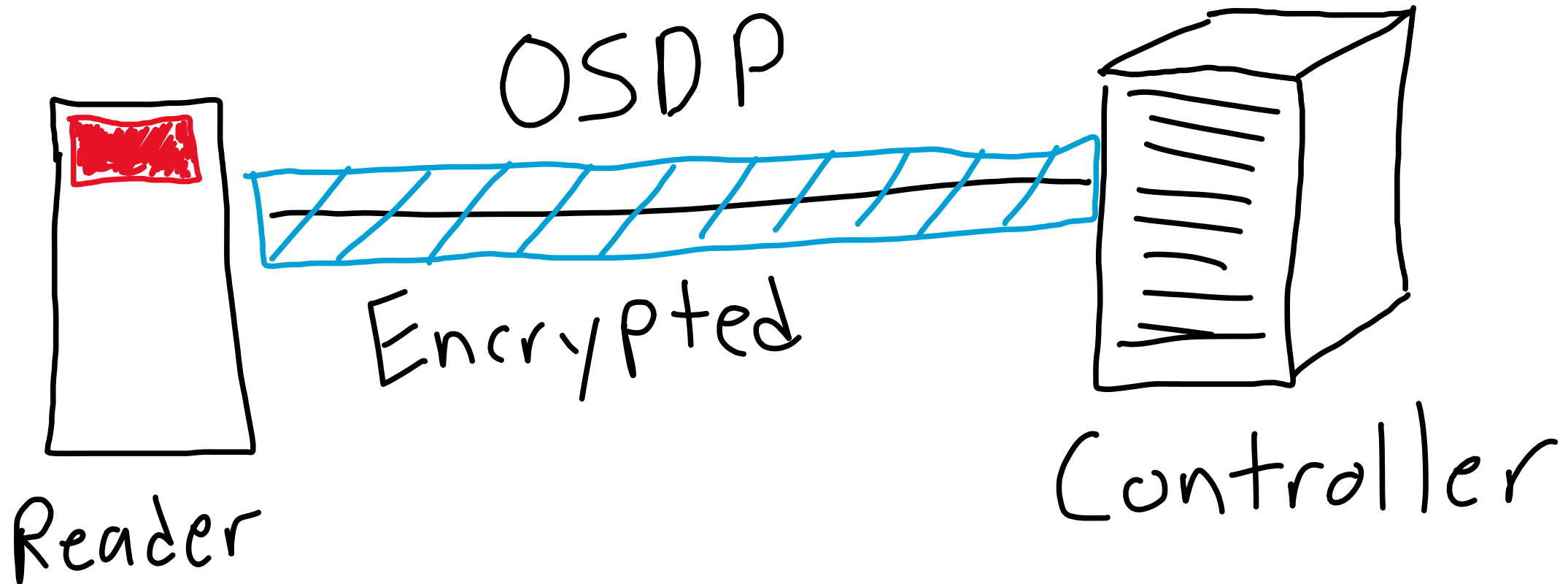


RFID Badge Setup

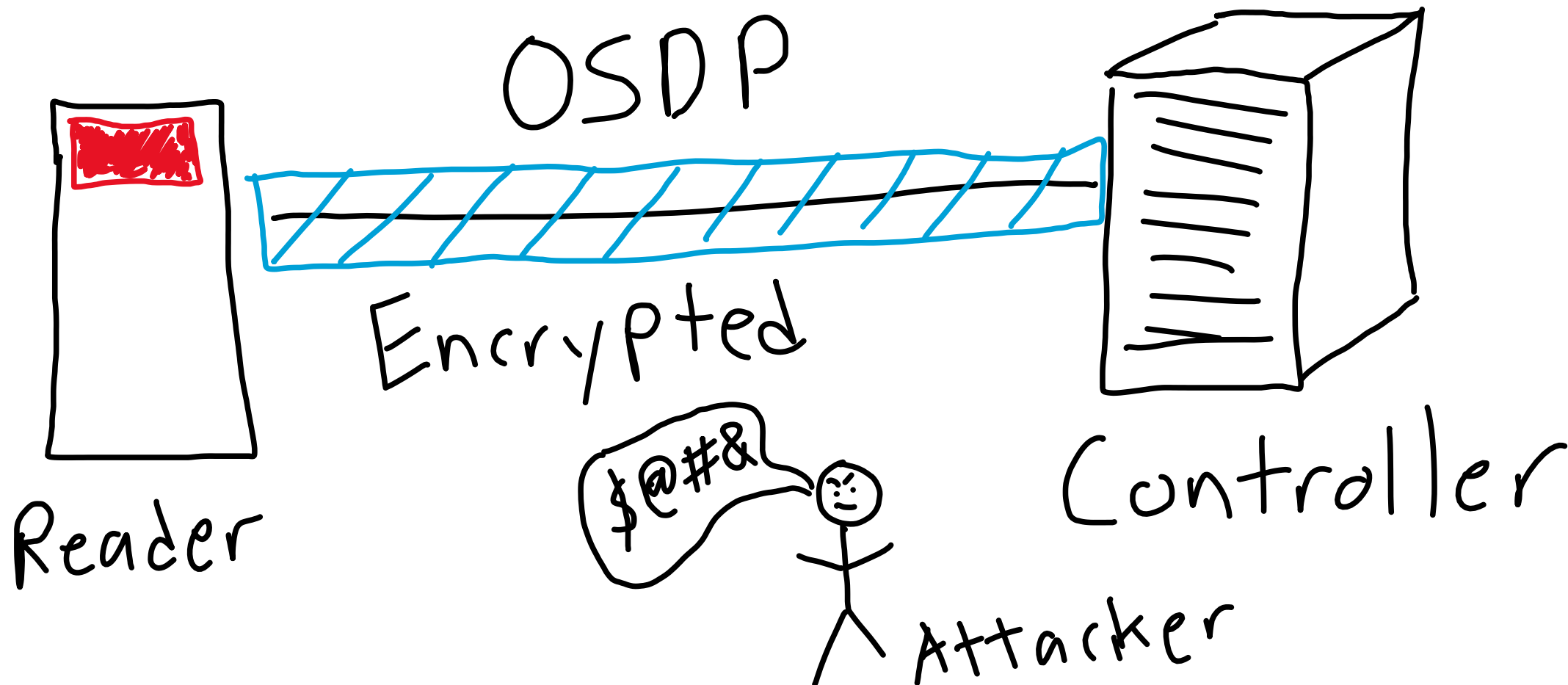




RFID Badge Setup



RFID Badge Setup



OSDP Reader Benefits:

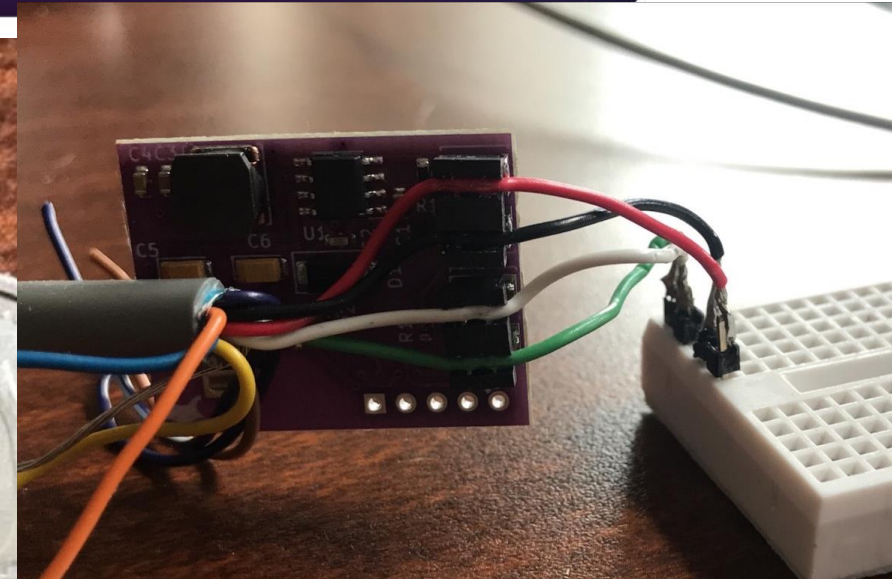
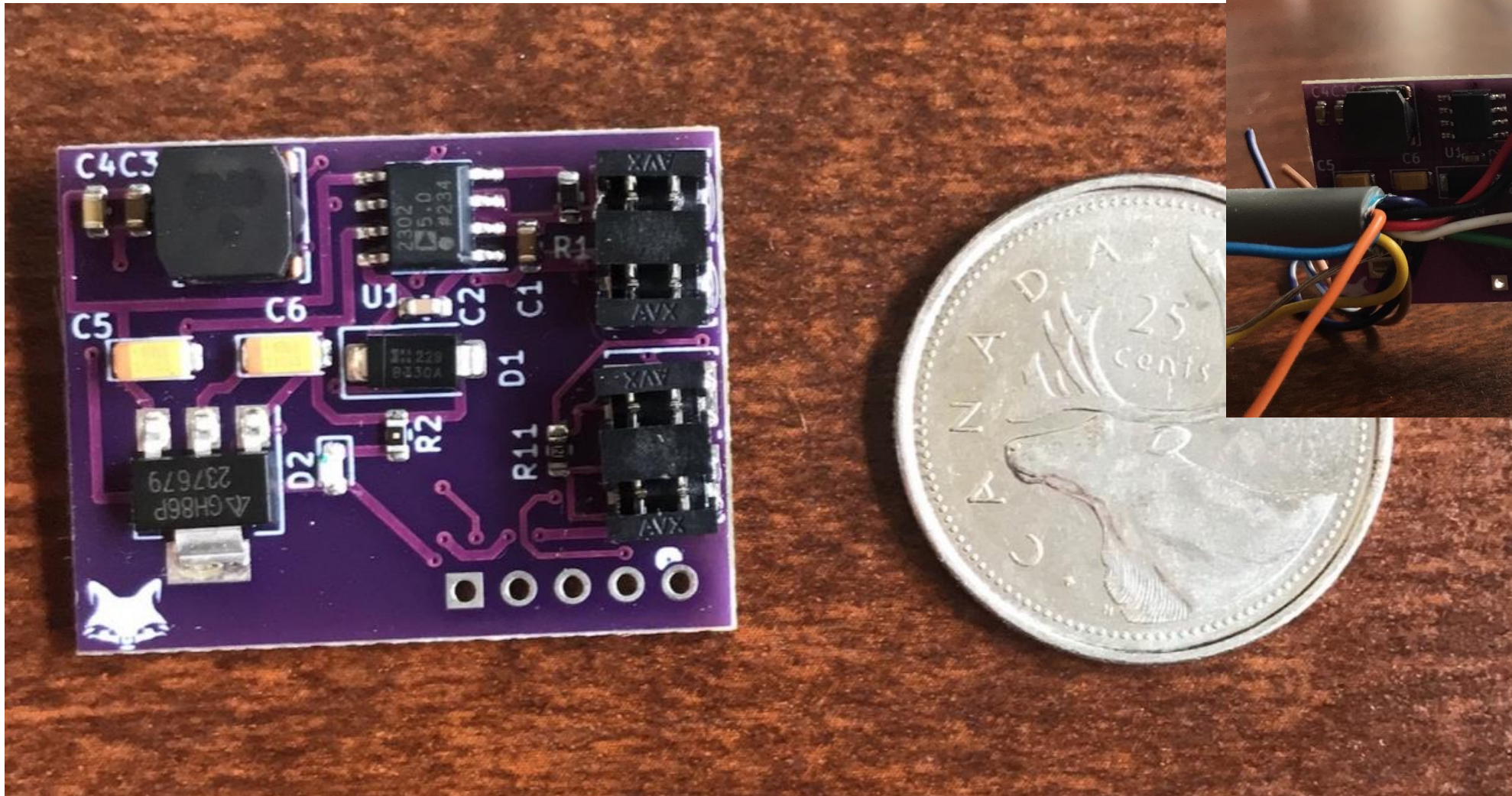
Secure Channel v2

Use existing wiring

128 Bit AES Encryption

Unhackable in 2020

Mellon



Demo #1



Axis

A1001

OSDP Supports

OSDP Supports
but doesn't *require*

OSDP Supports
but doesn't *require*
encryption

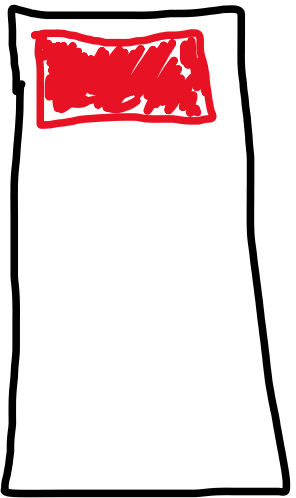
OSDP-SC ("secure" channel)

is an OSDP extension

The Protocol

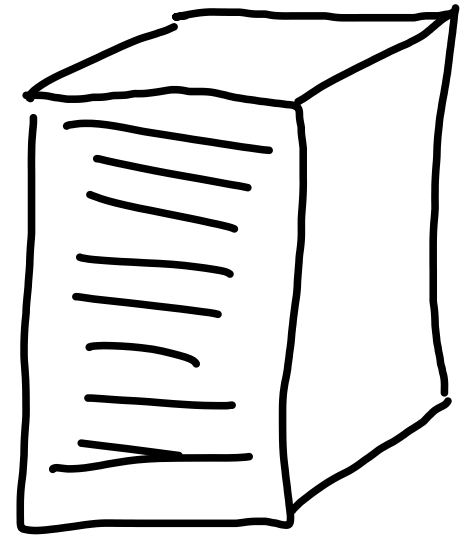


...to encrypt...



Reader

Controller



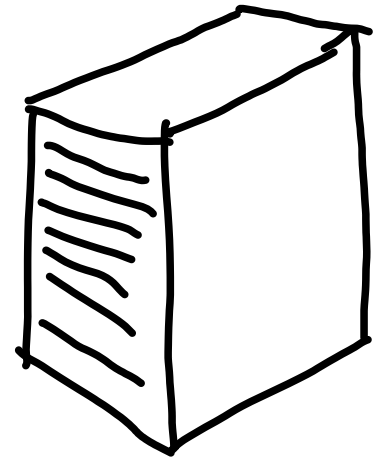
...or not
to encrypt

Protocol Basics

RS-485 (the other serial)

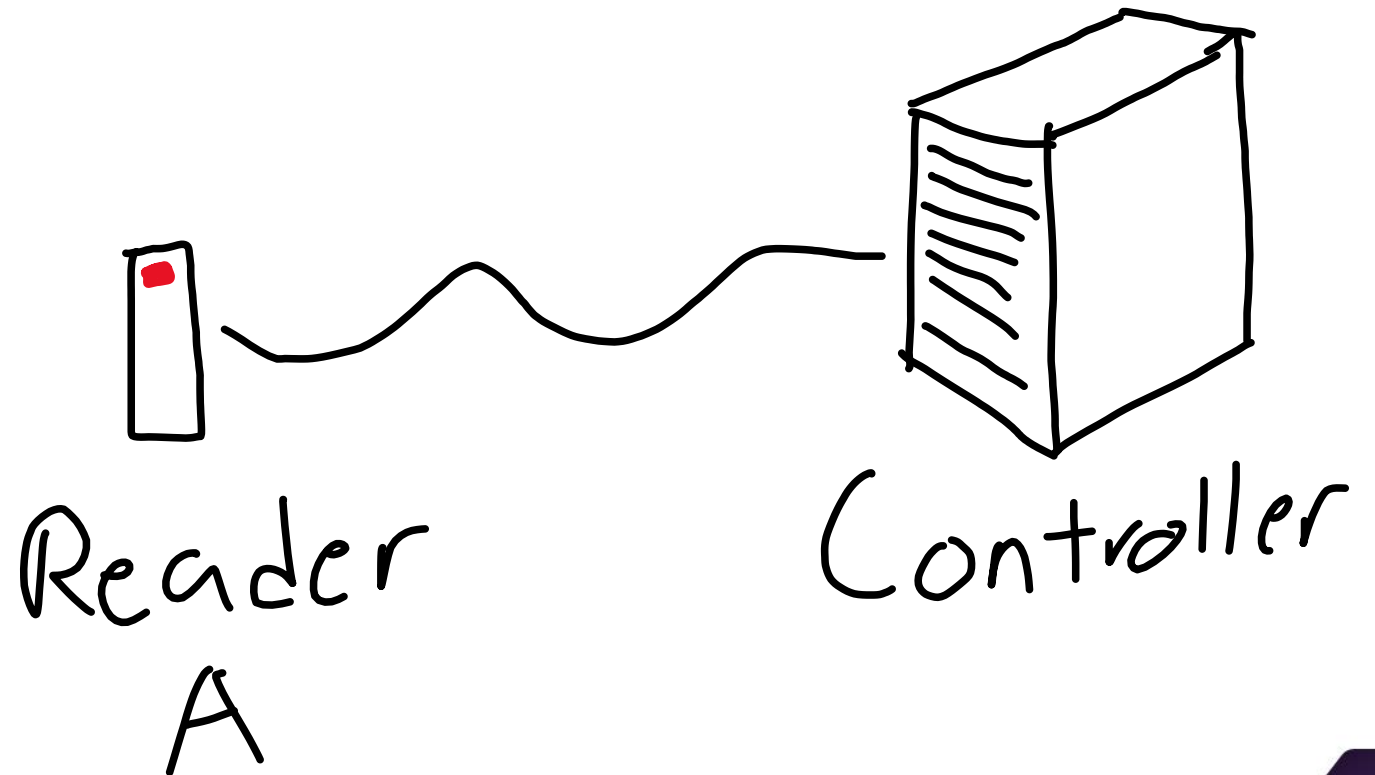
- multi-drop

RS-485 (the other serial)

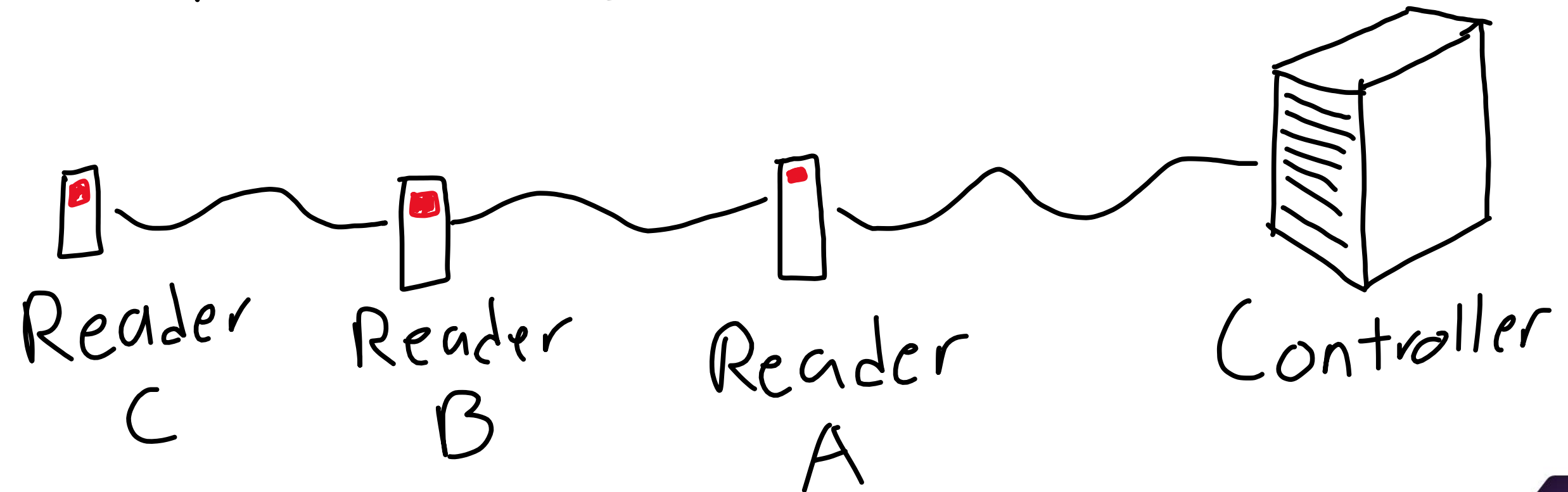


Controller

RS-485 (the other serial)



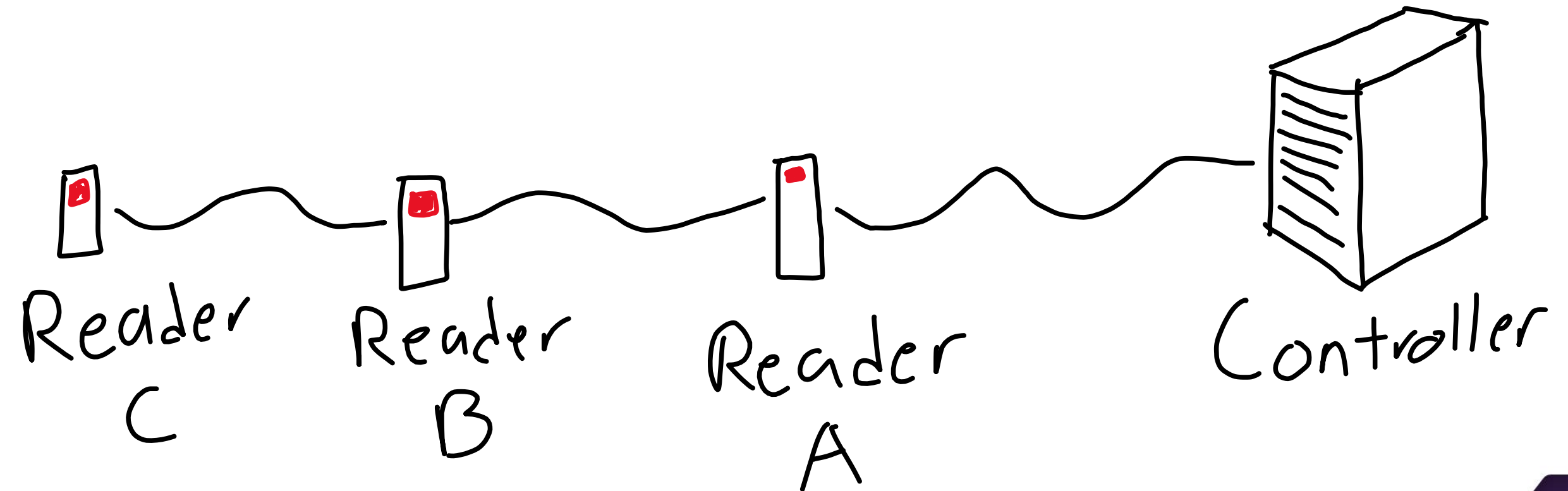
RS-485 (the other serial)



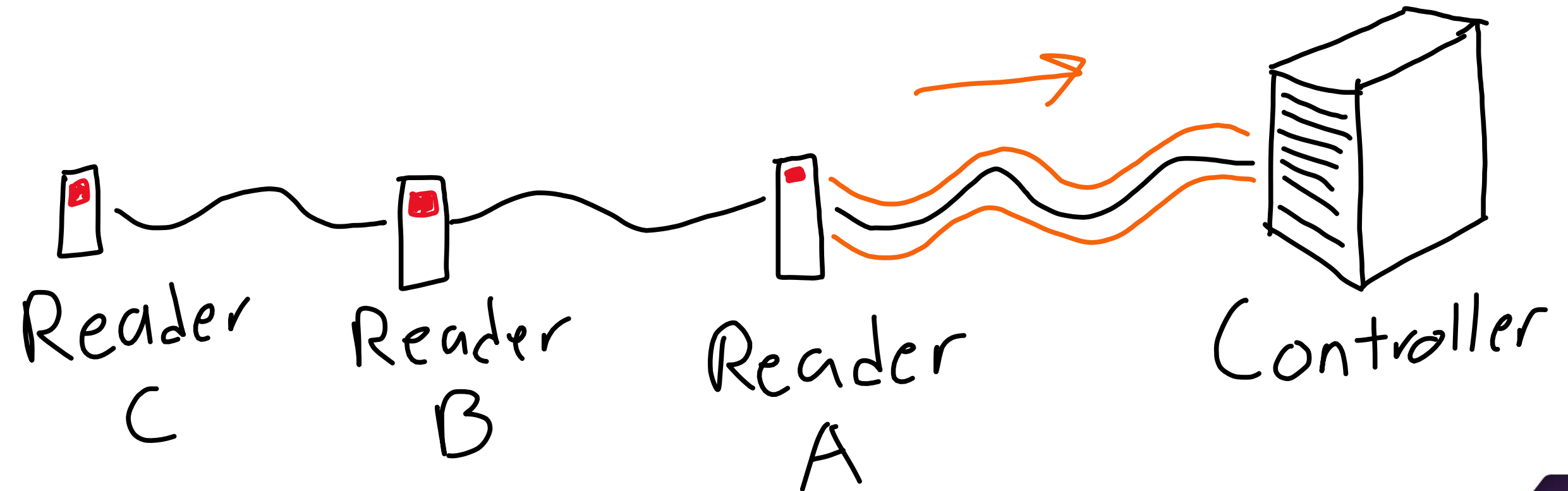
OSDP:

All messages are
broadcast

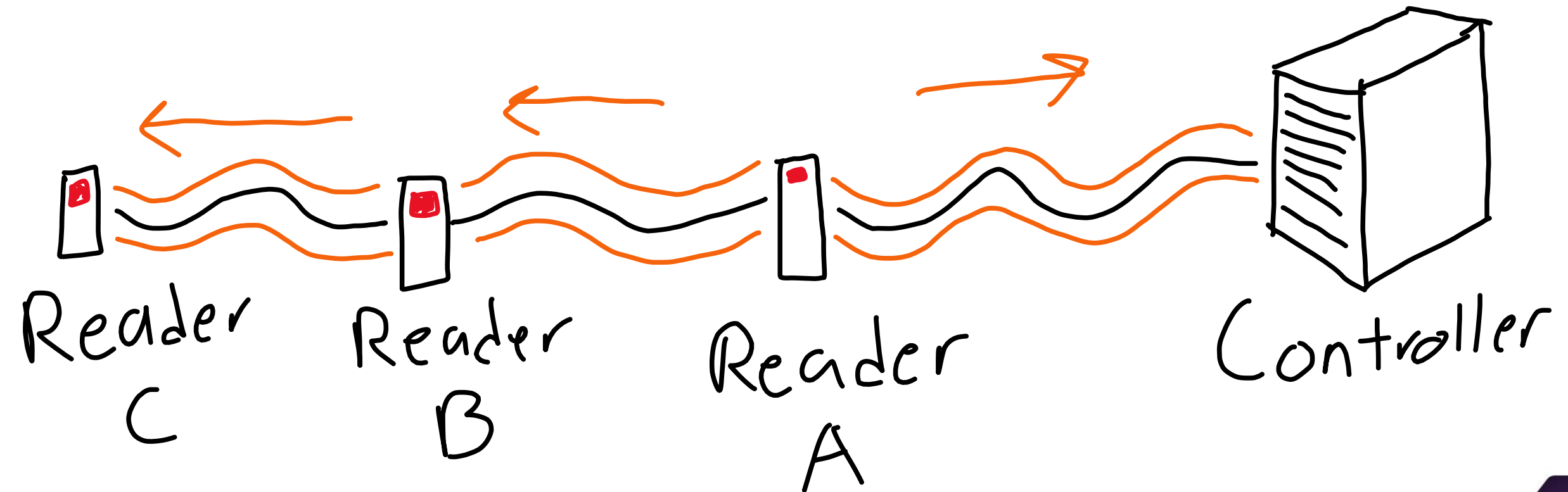
OSDP



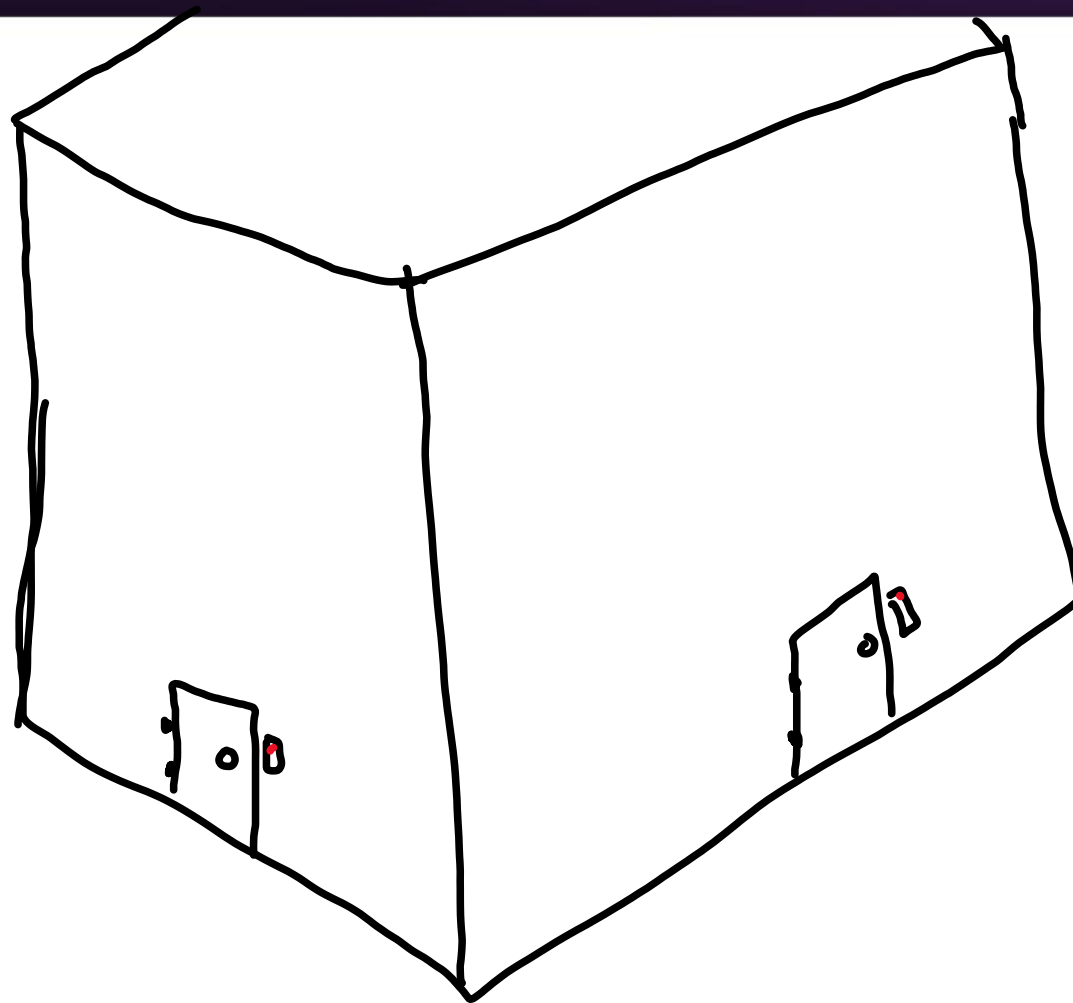
OSDP



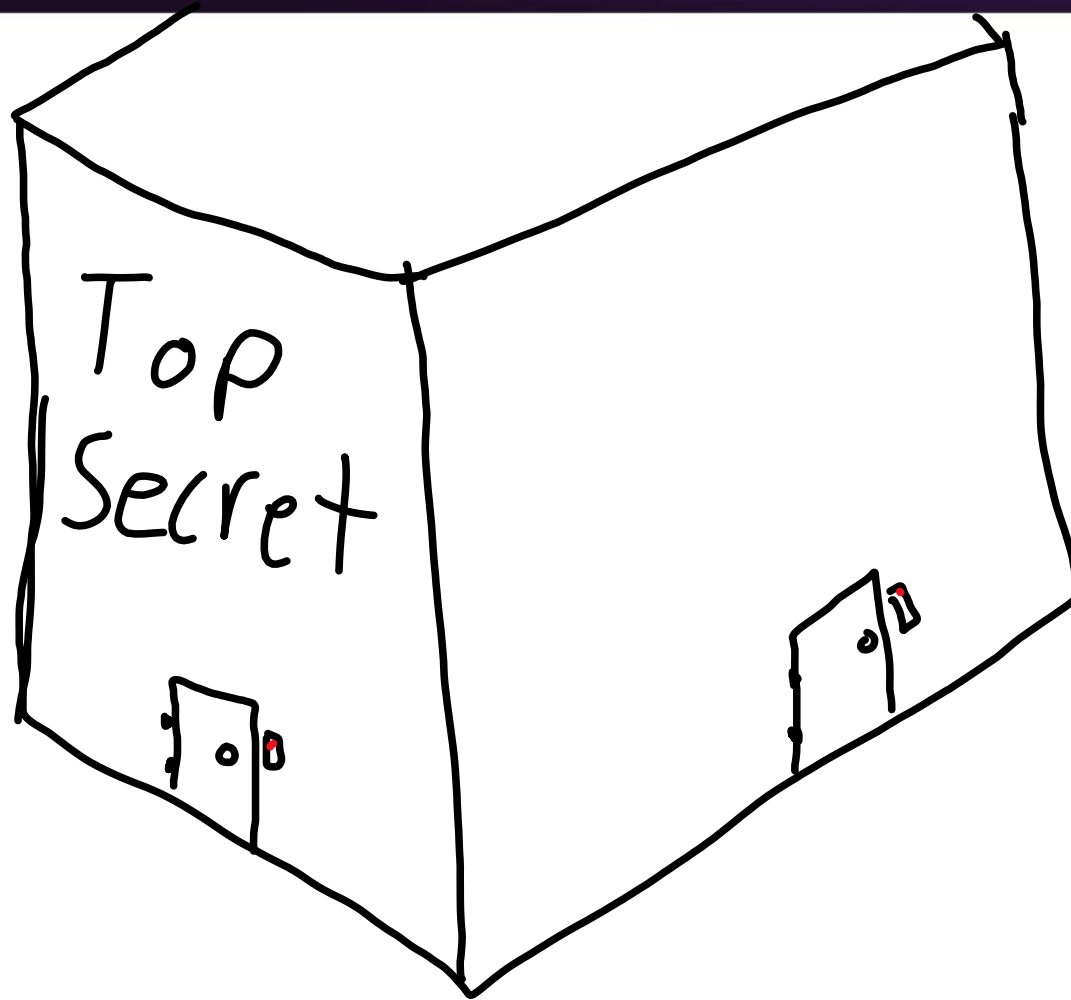
OSDP



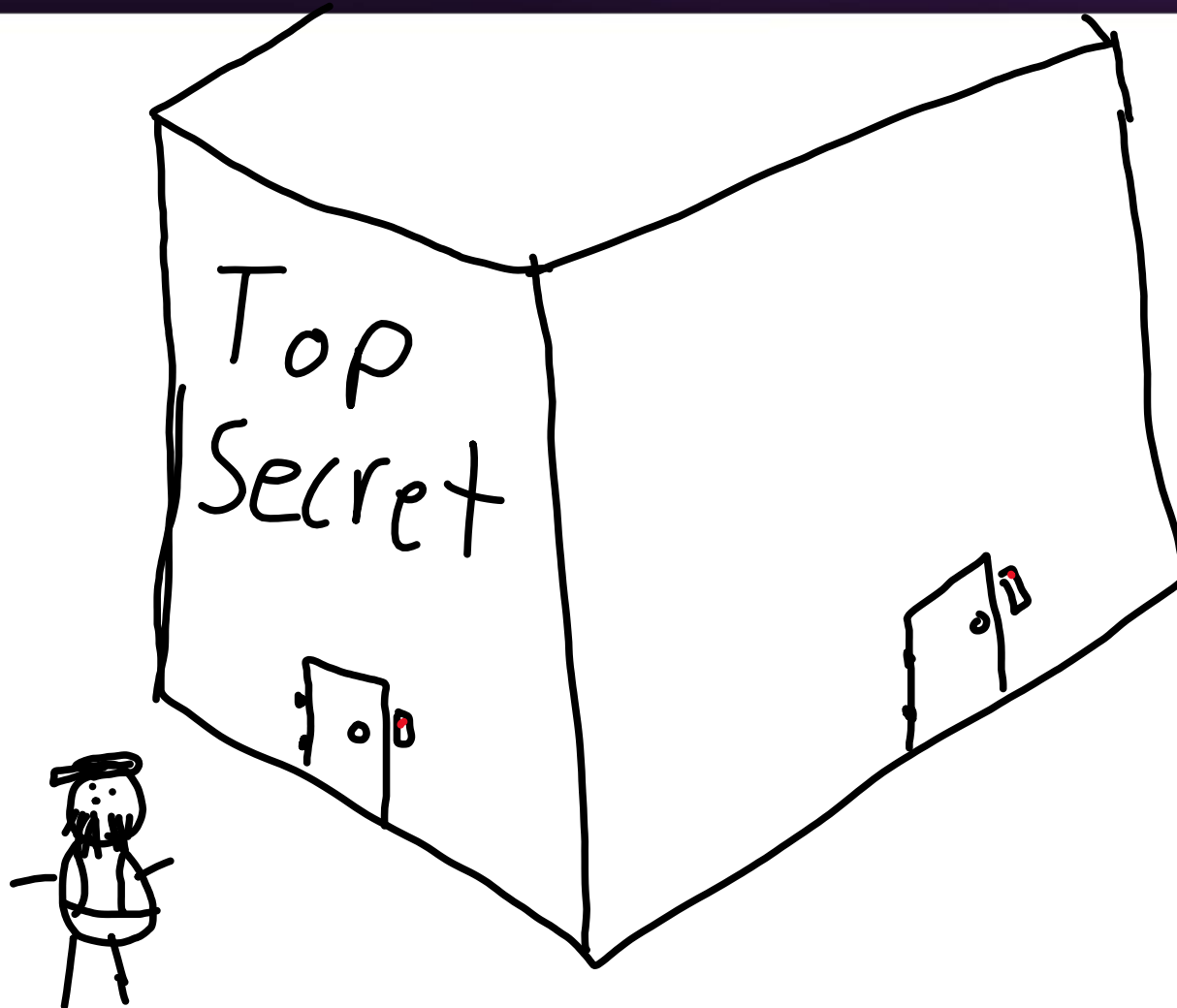
Undisclosed Location in Santa Fe, NM



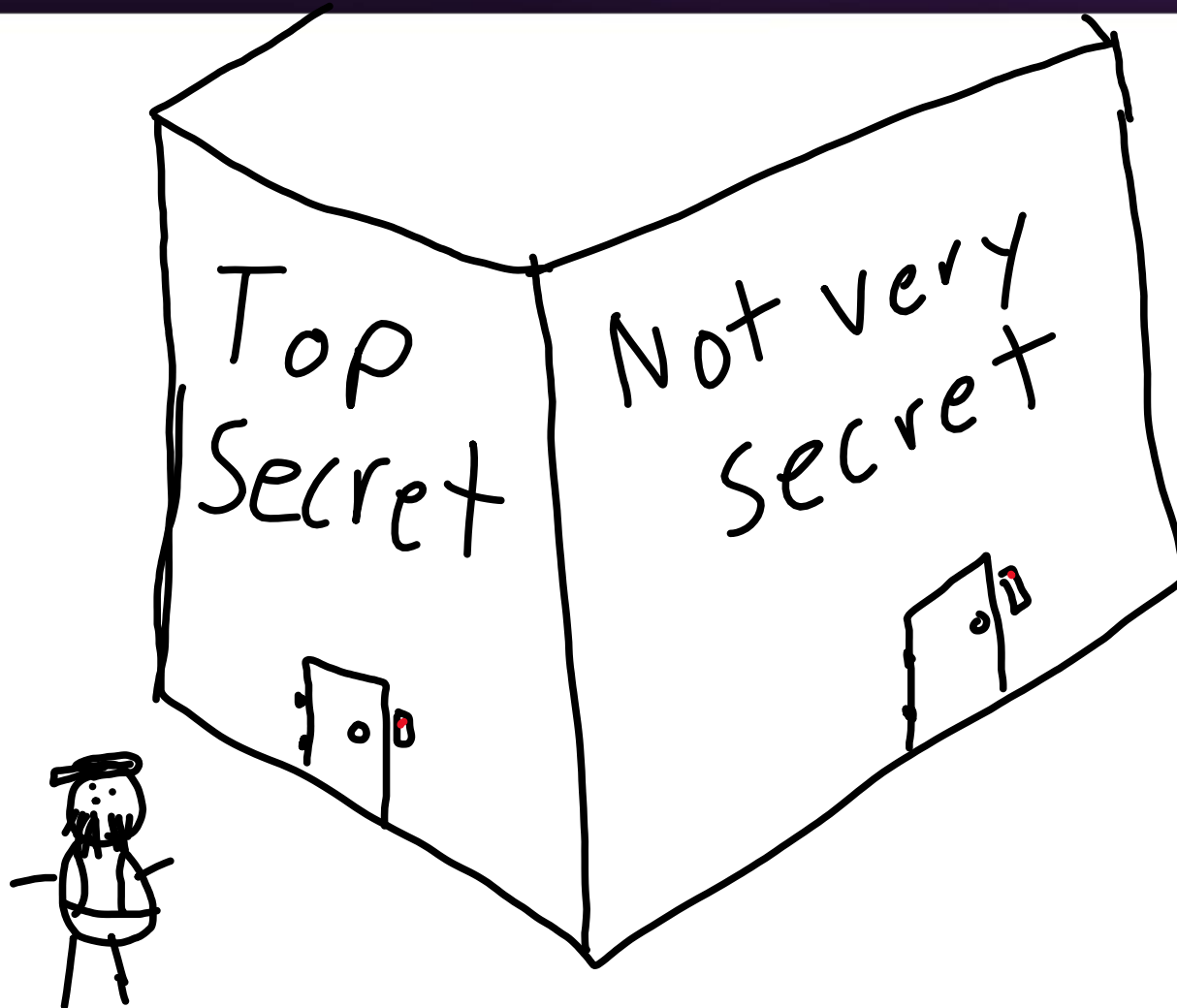
Undisclosed Location in Santa Fe, NM



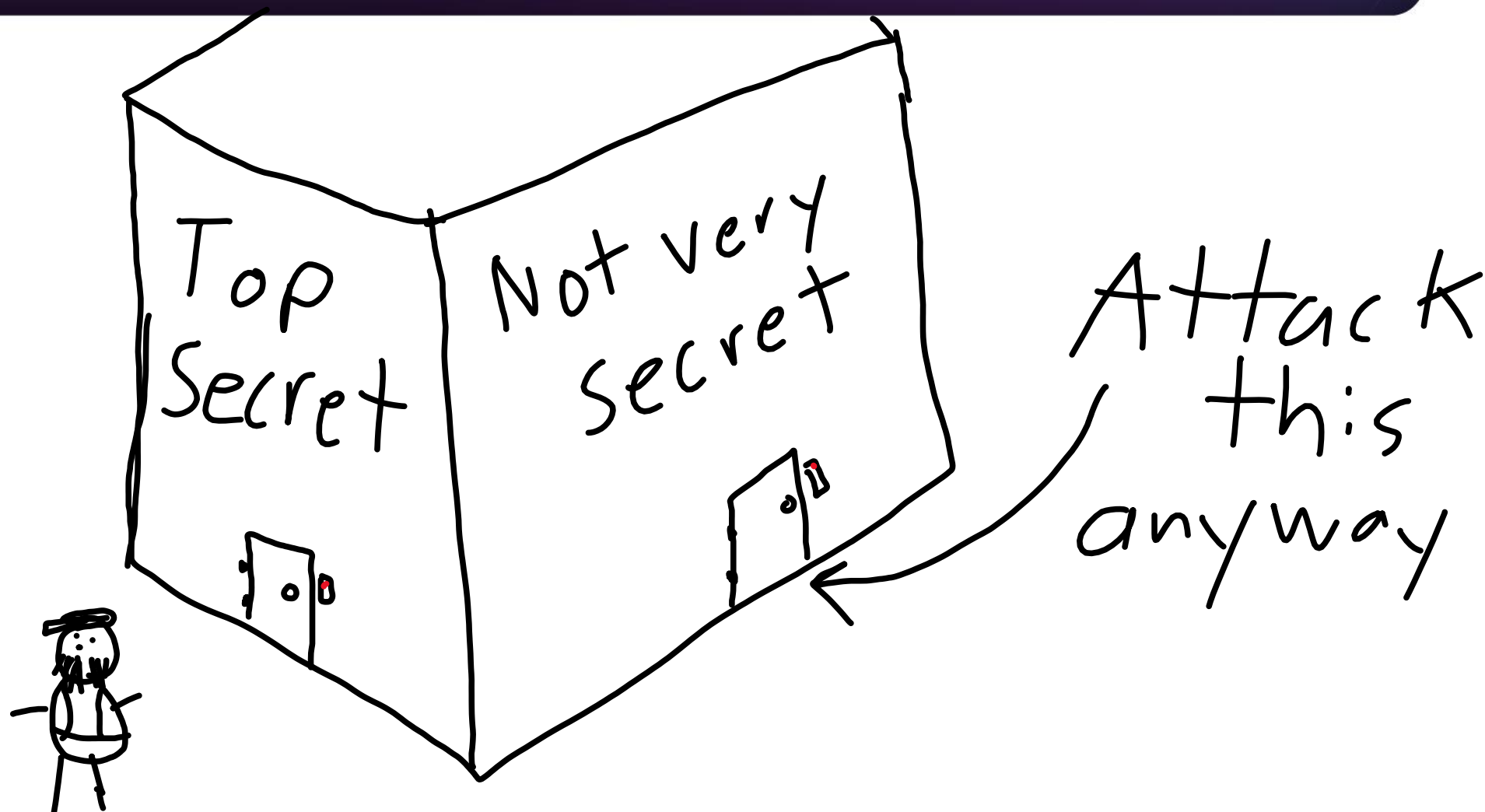
Undisclosed Location in Santa Fe, NM



Undisclosed Location in Santa Fe, NM



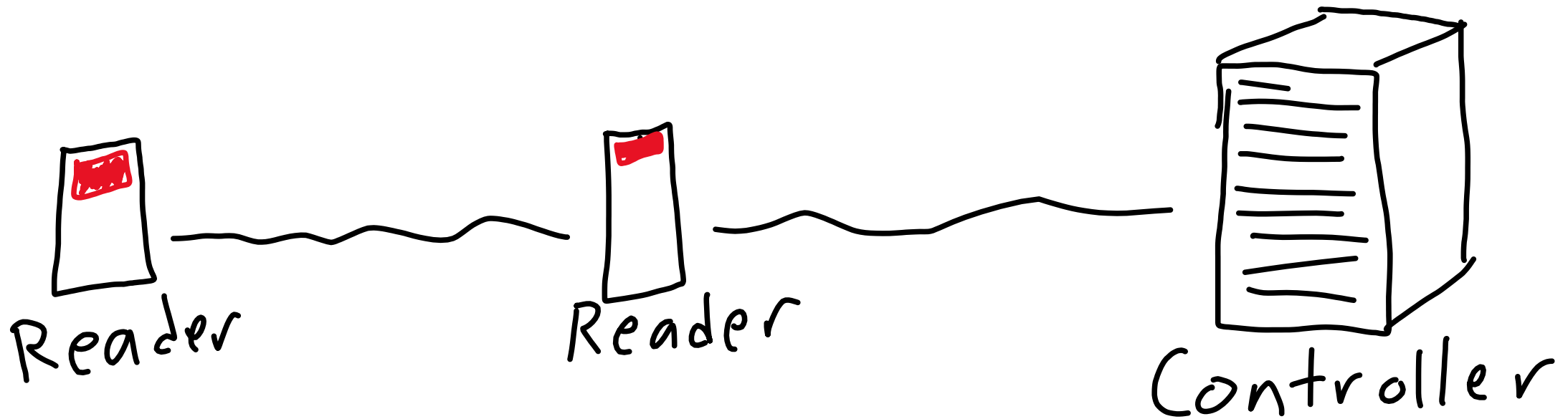
Undisclosed Location in Santa Fe, NM



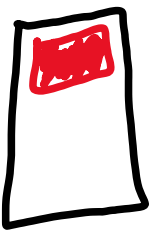
OSDP:

Client-Server
Model

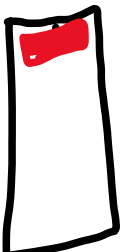
OSDP



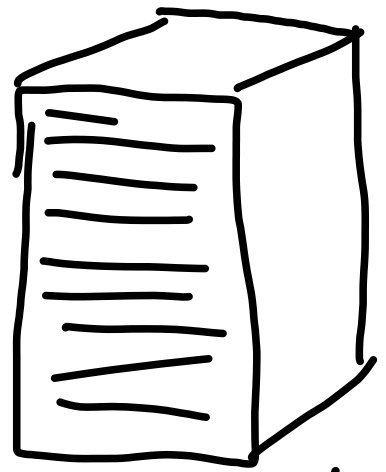
OSDP



Reader
Server

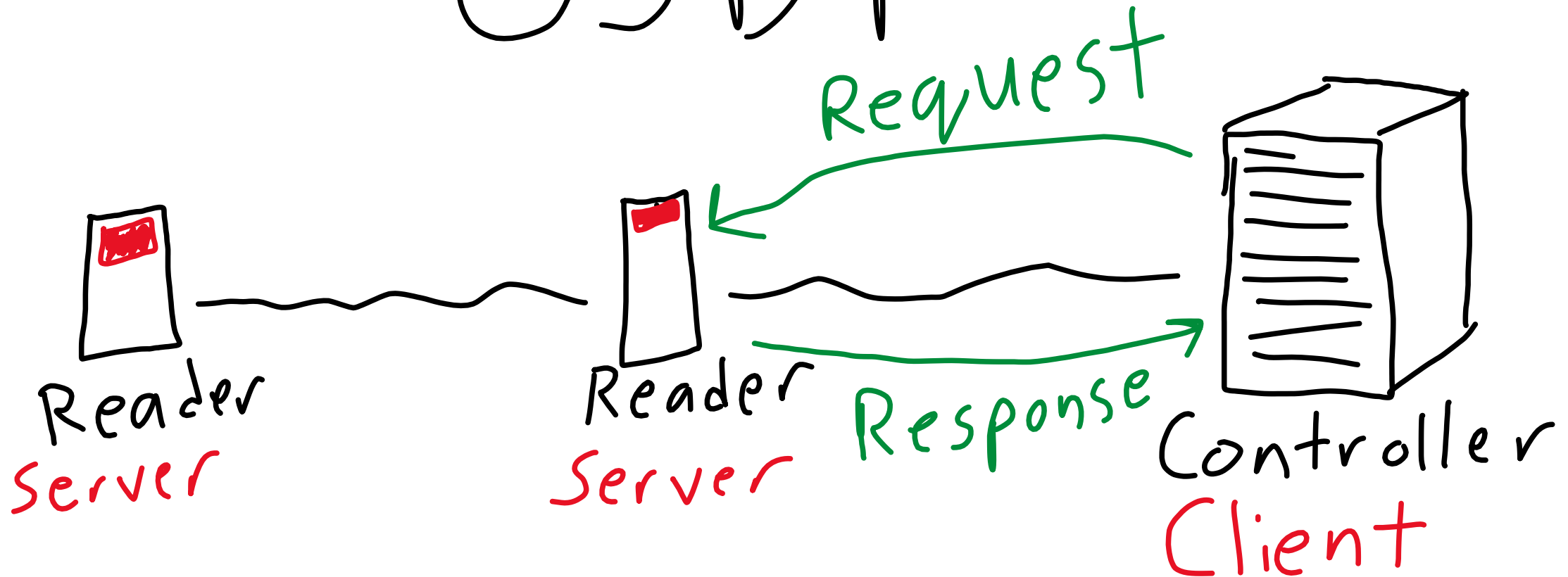


Reader
Server



Controller
Client

OSDP

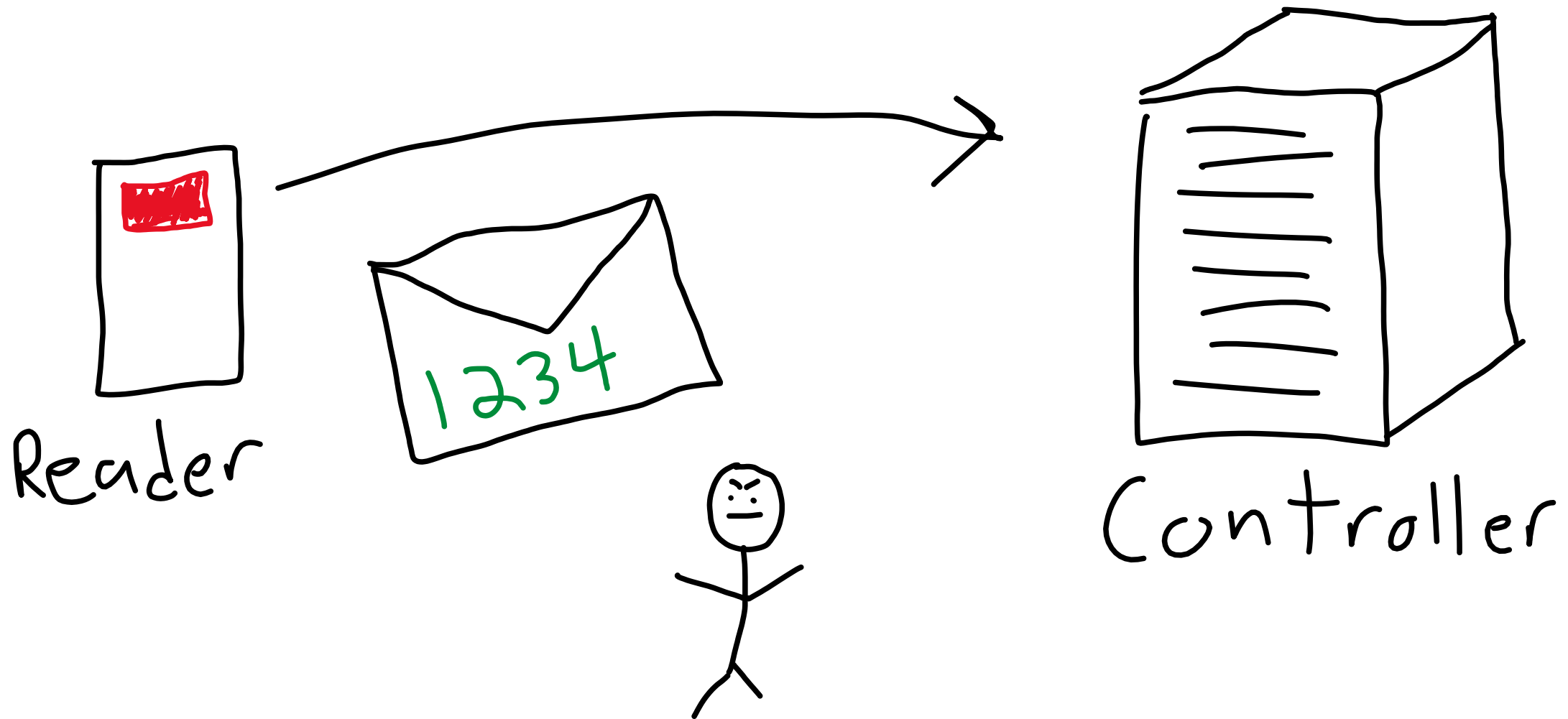


Protocol WTF #1

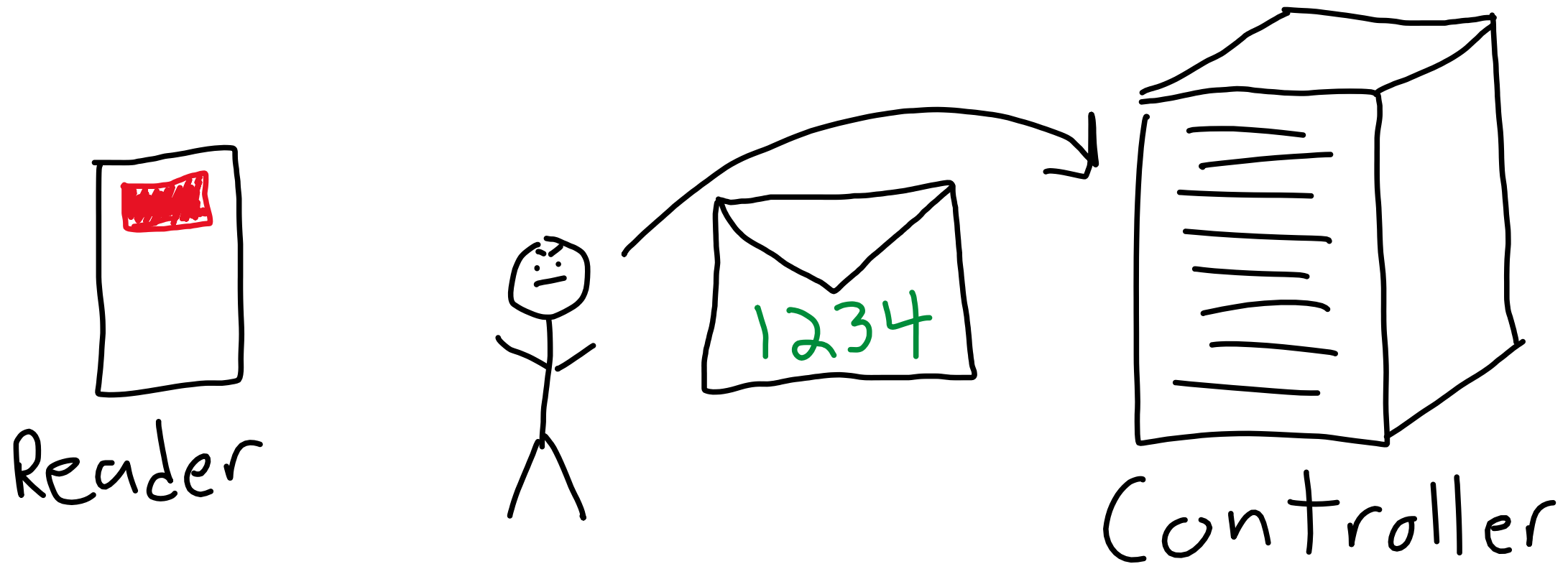


What, are we paying
by the bit now?

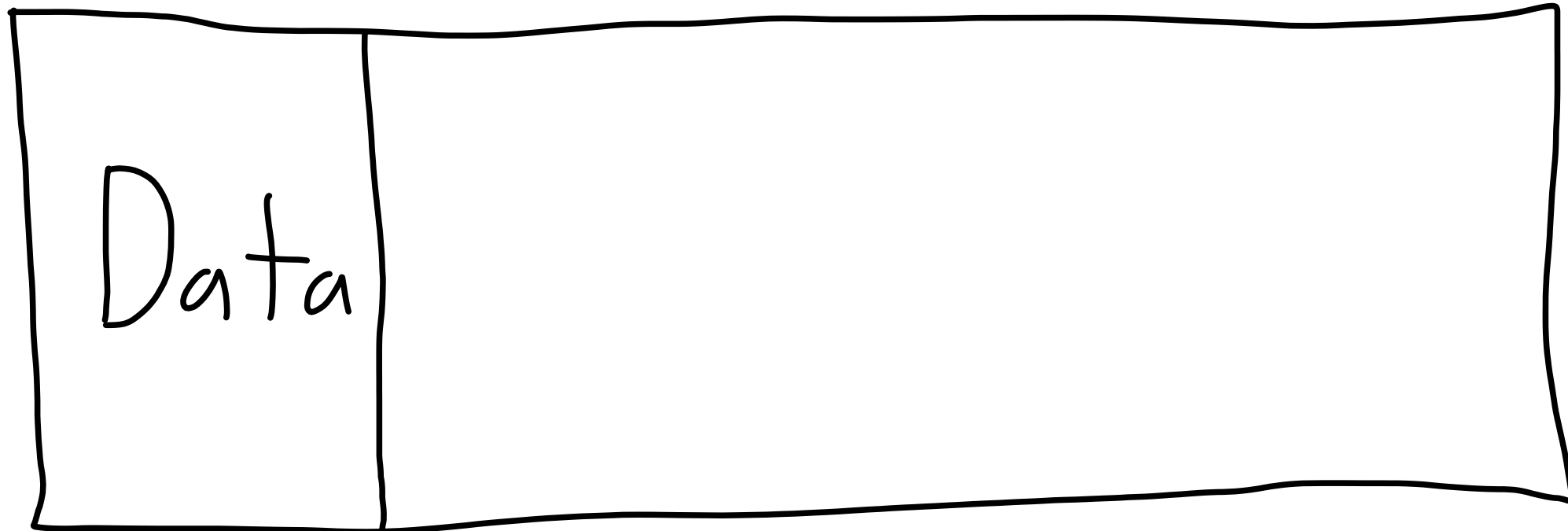
Replay Attacks



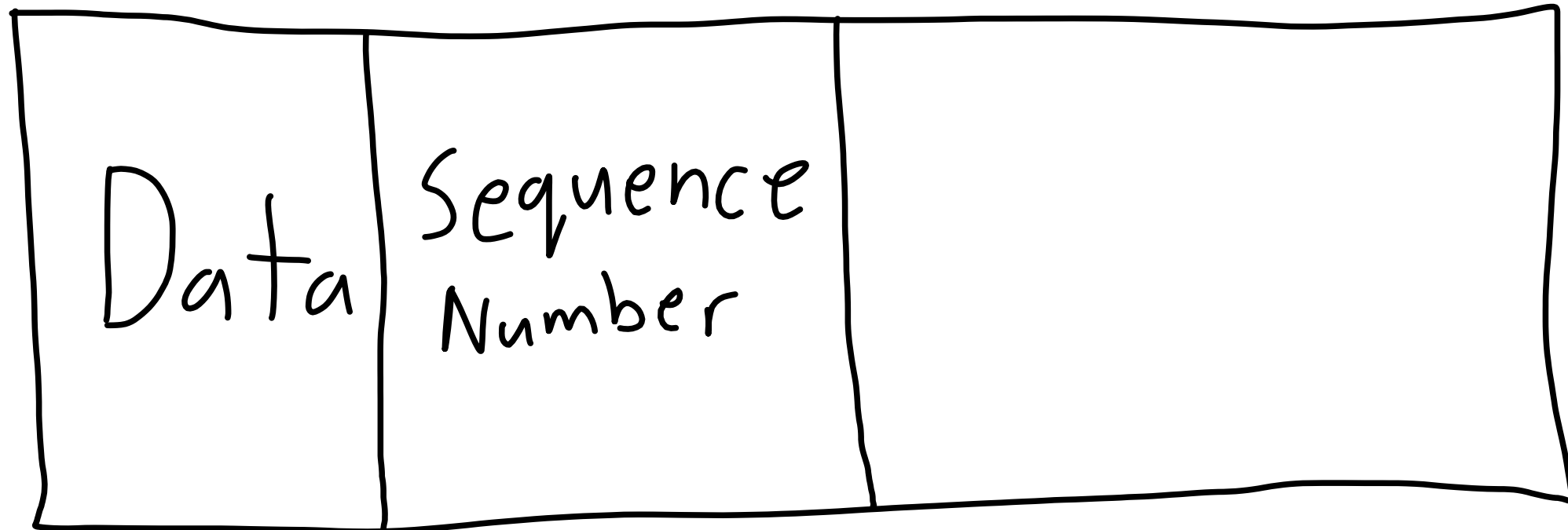
Replay Attacks



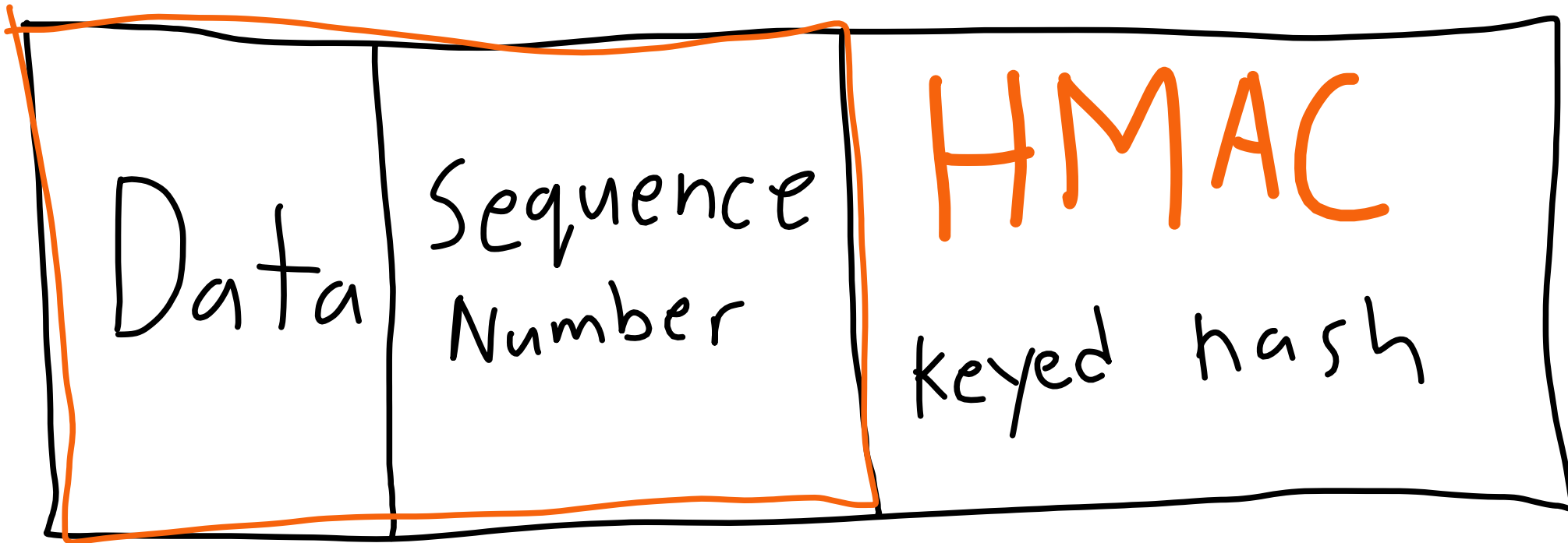
Sequence Number + HMAC



Sequence Number + HMAC



Sequence Number + HMAC



How Many Bits is Enough?

128-bits? - Cryptographic
Strength

How Many Bits is Enough?



~~128-bits?~~ - Cryptographic
Strength
64-bits? - Edge of Enumeration

How Many Bits is Enough?



~~128-bits?~~ - Cryptographic Strength
~~64-bits?~~ - Edge of Enumeration
32-bits? - Fine, maybe?

How Many Bits is Enough?



2-bits

How Many Bits is Enough?

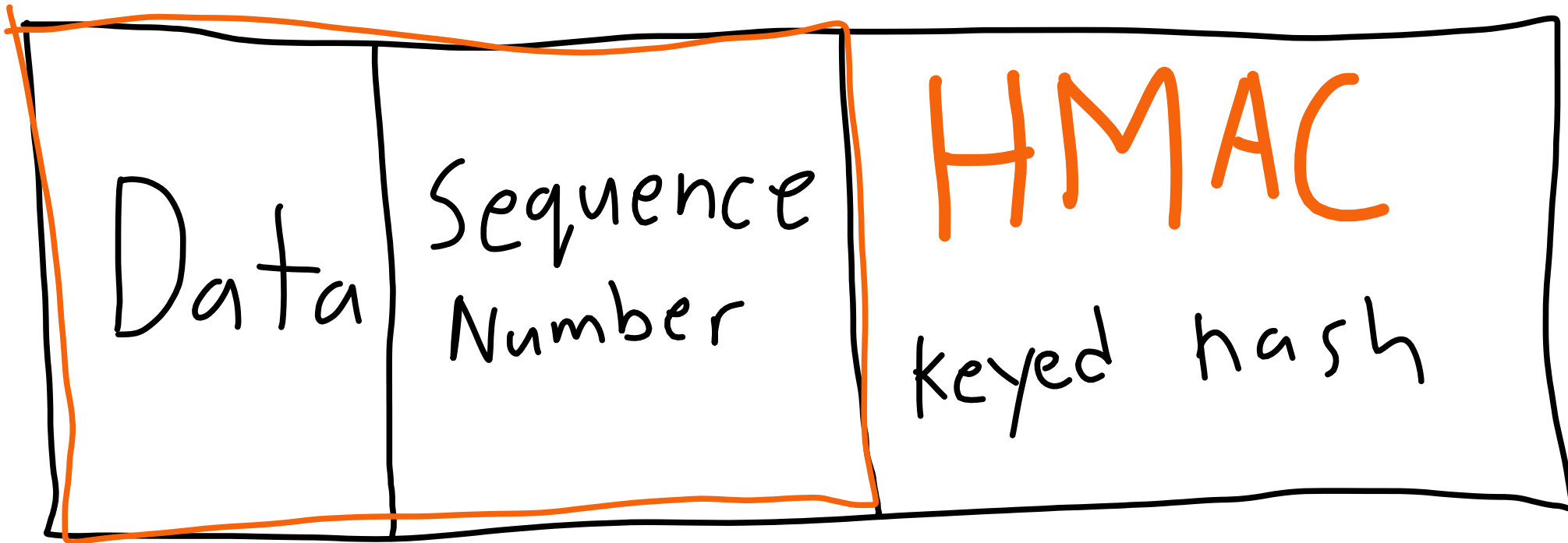


~~2~~^{1 1/2} - bits

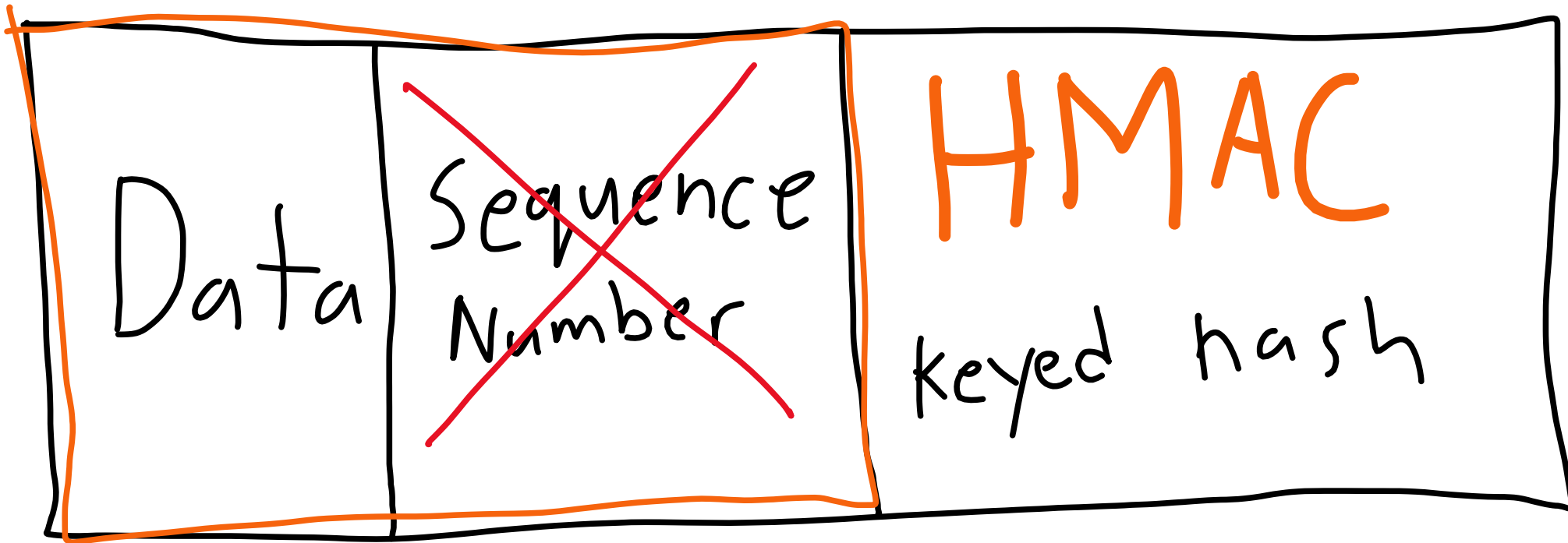
SQN Values

The sequence number is incremented by the CP from one command to the next, skipping zero: 0->1->2->3->1->... Non-zero sequence numbers support error recovery: the Control Panel (CP) acknowledges the last reply by sending the next command with the incremented sequence number,

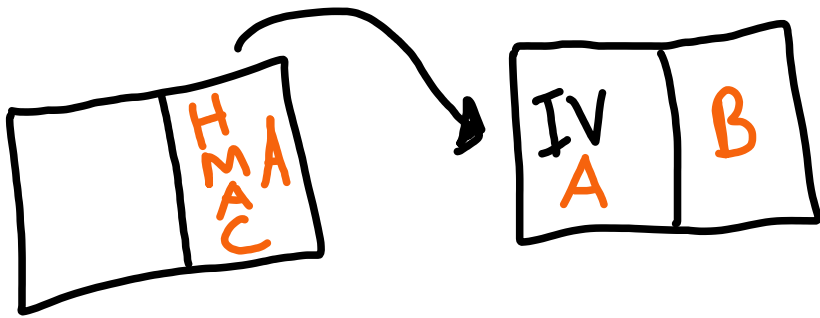
Sequence Number + HMAC



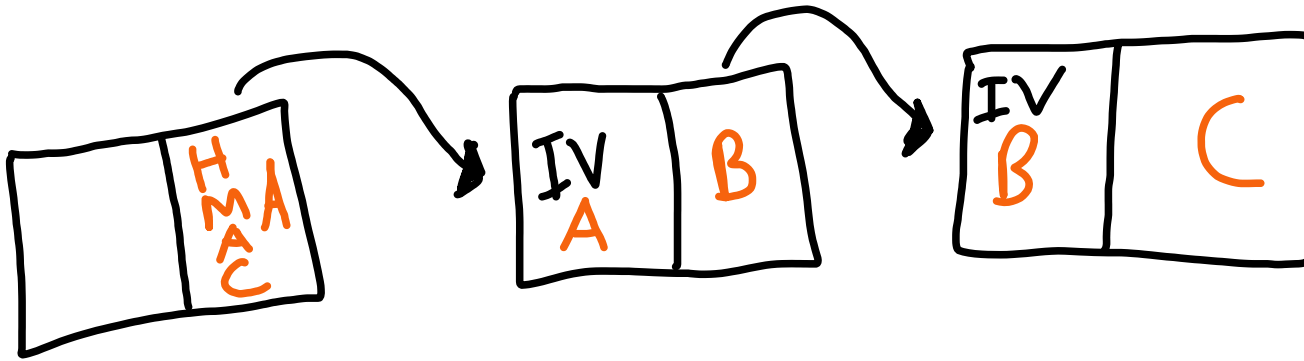
Sequence Number + HMAC



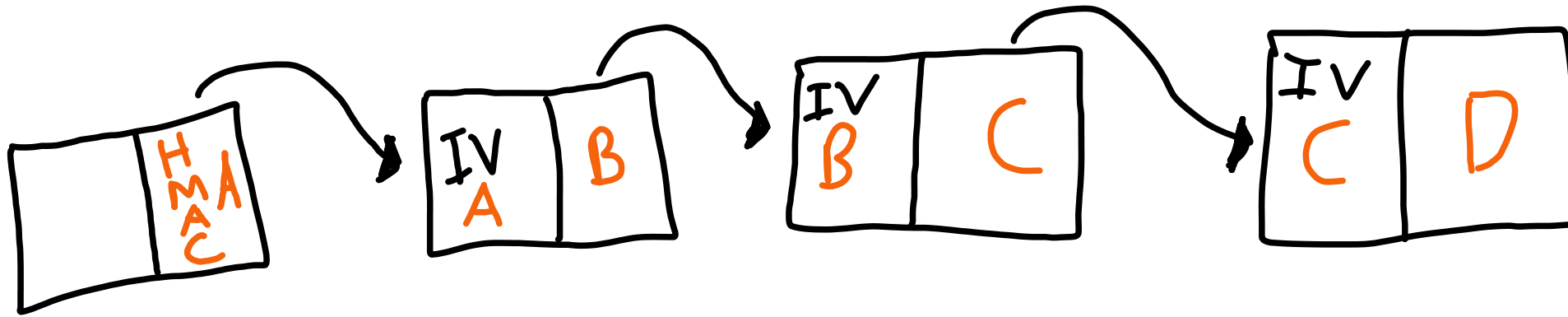
IV Chaining



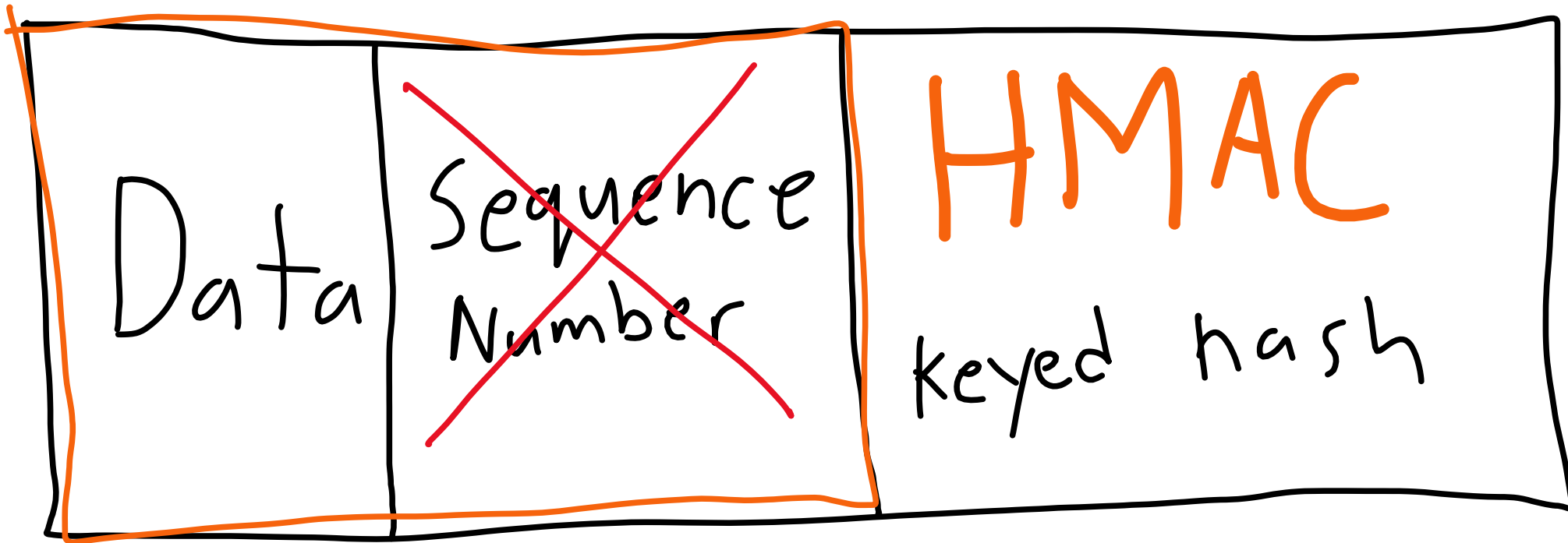
IV Chaining



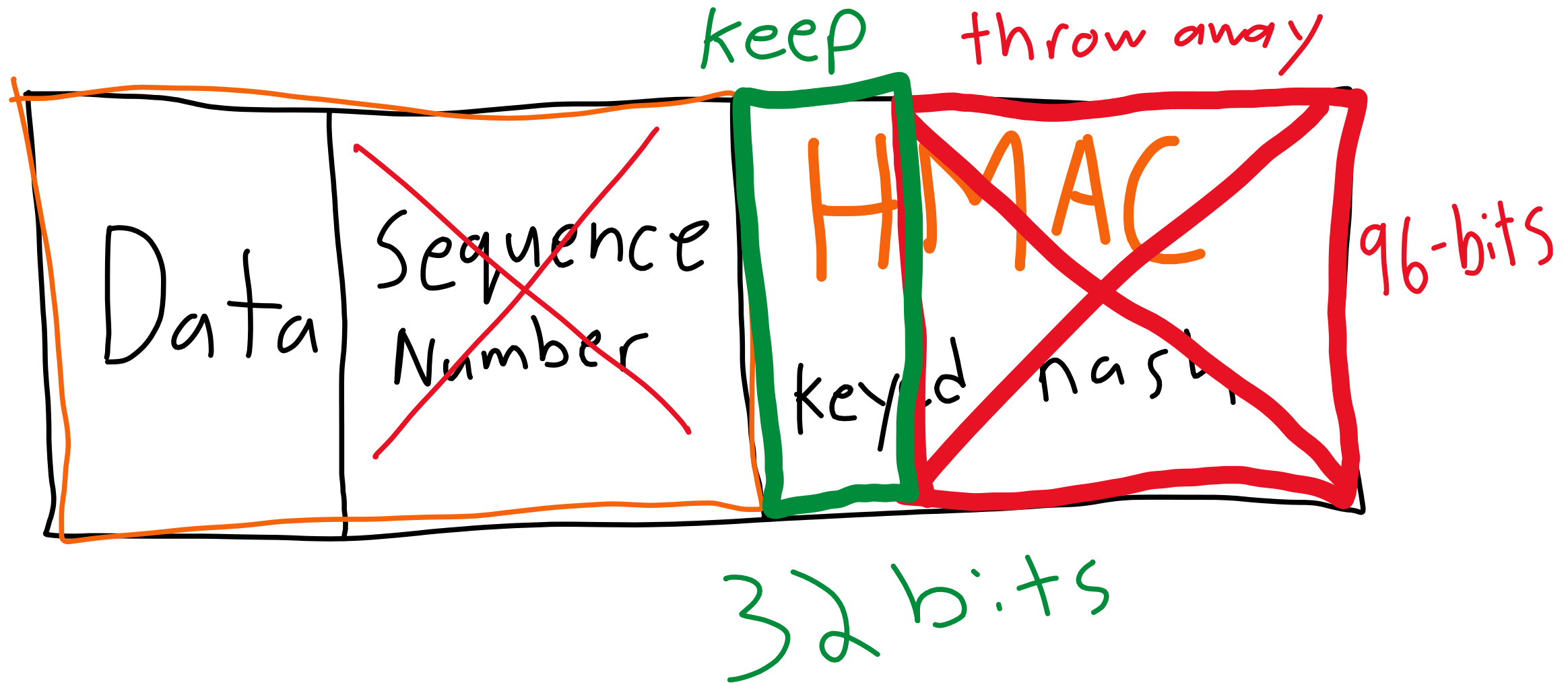
IV Chaining



Sequence Number + HMAC



Sequence Number + HMAC



D.4.6 Message Authentication Code (MAC) Generation

General: MAC is computed for and appended only to messages whose SEC_BLK_TYPE is SCS_15, SCS_16, SCS_17, and SCS_18. The AES algorithm is applied in CBC mode using S-MAC1 as the key for all blocks, except the last one, and using S-MAC2 as the key for the last block. If the message contains only one block, then only S-MAC2 is used.

ICV values: The ICV is initialized during the Secure Connection Sequence by the PD and is passed to the CP during SCS_14 in reply osdp_RMAC_I.

R-MAC – the ICV value for generating the R-MAC is the previously received C-MAC.

C-MAC – the ICV value for generating the C-MAC is the previously received R-MAC.

After the initial OSDP-SC setup, in order to reduce the message size and transmission time overhead, the messages will contain only a partial MAC. For messages whose SEC_BLK_TYPE is SCS_15, SCS_16, SCS_17, and SCS_18 only the first four bytes of the computed MAC are sent. The MAC verification will locally generate the full MAC[16] and compare the actual bytes that were received.



Wait

only the first four bytes of the computed MAC are sent.

only the first four bytes



Reduced



Overhead



Reduced



Overhead



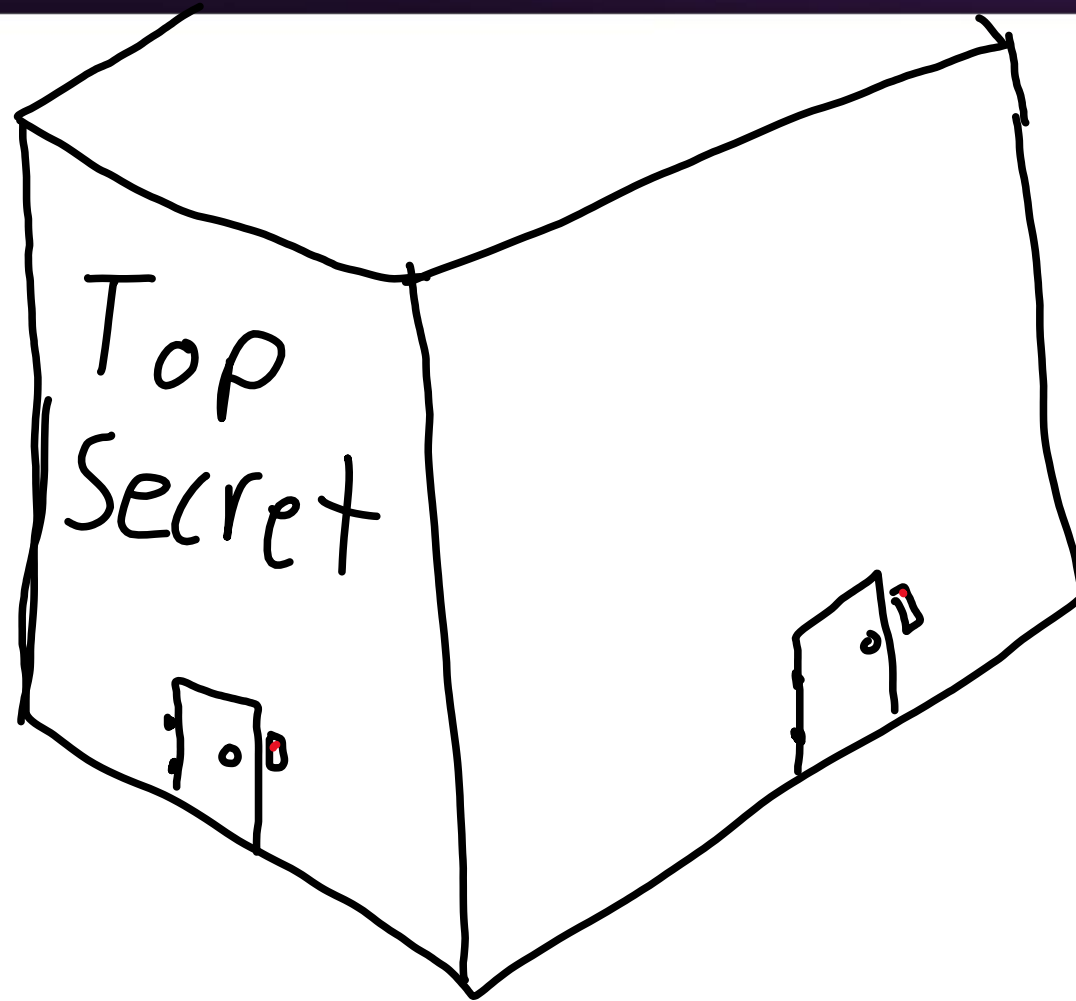
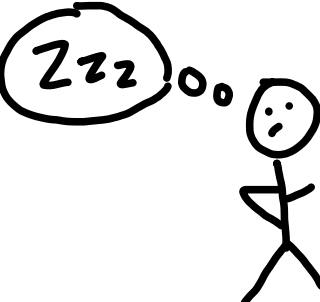


2^{31} attempts (on average)

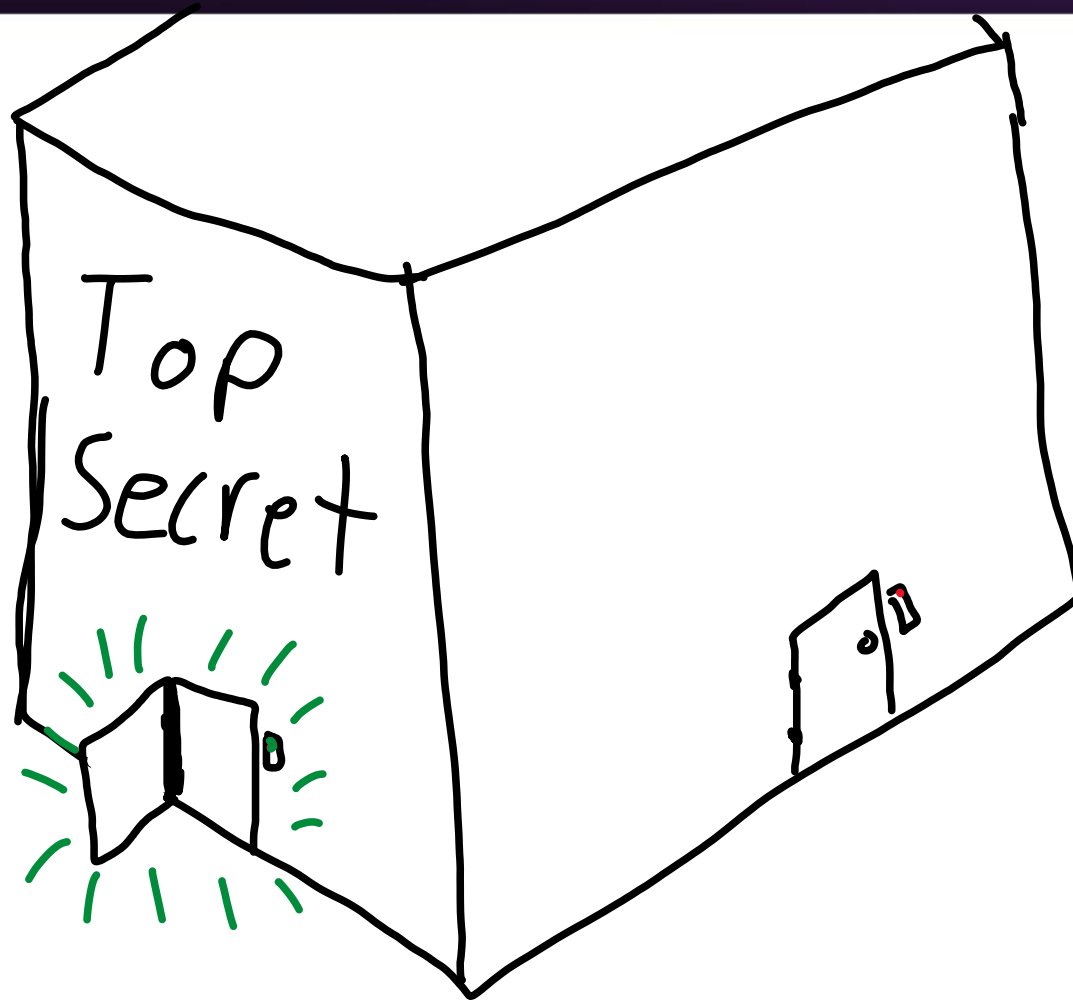
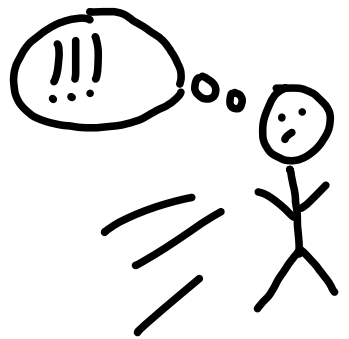
2^{31} attempts (on average)
115,200 baud (RS-485)

2^{31} attempts (on average)
115,200 band (RS-485)
~ success after
35 days

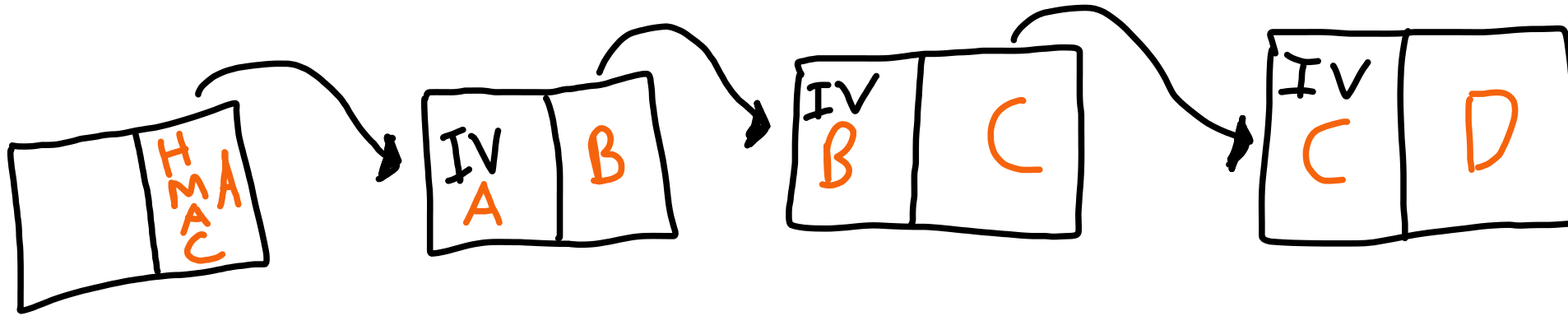
35
days

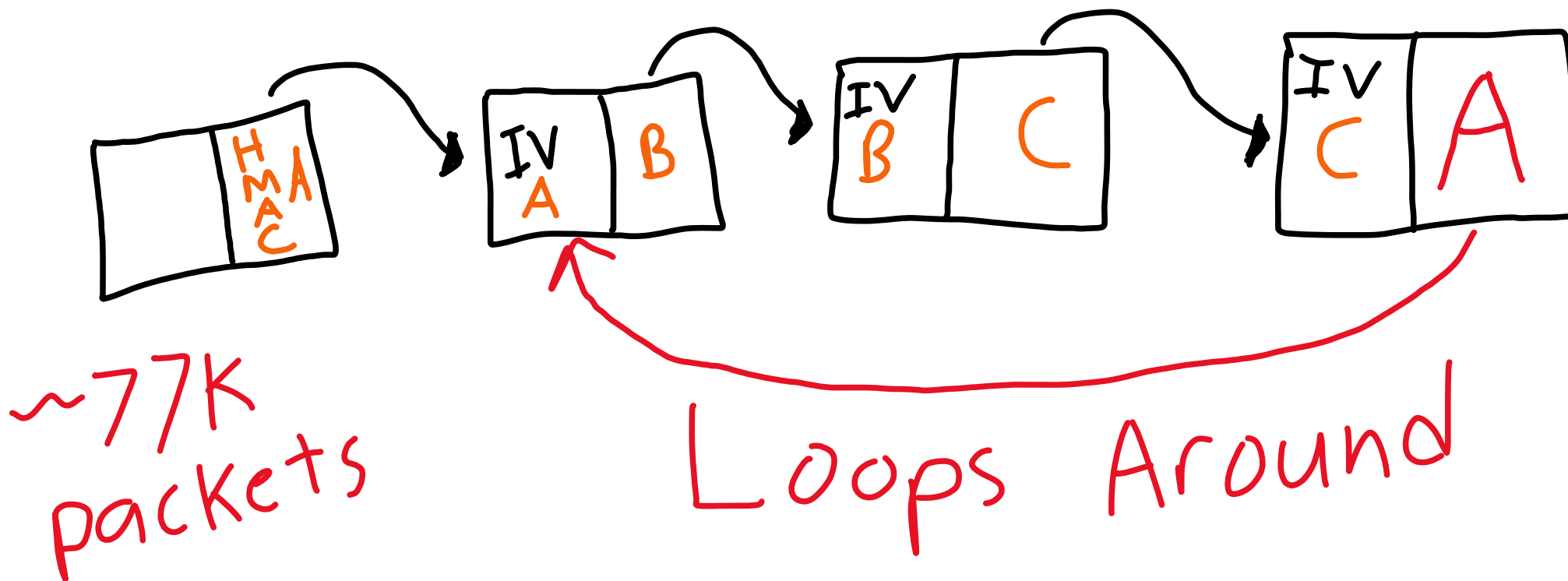


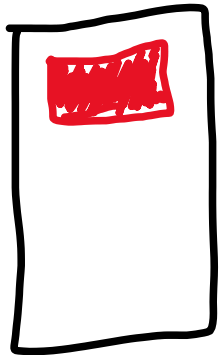
35
days



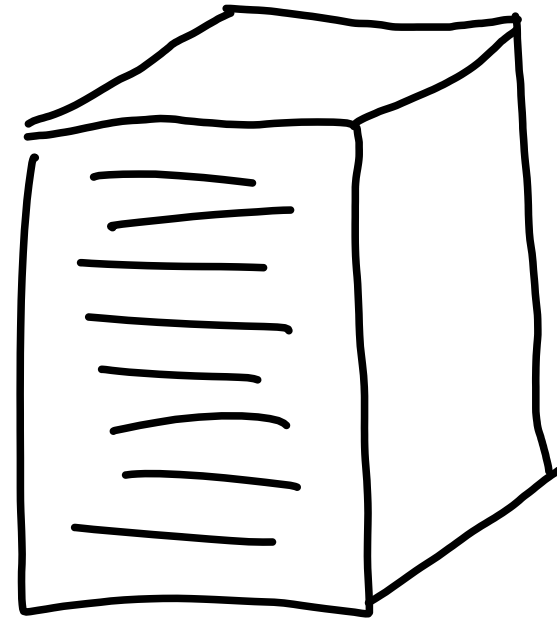
Happy Birthday





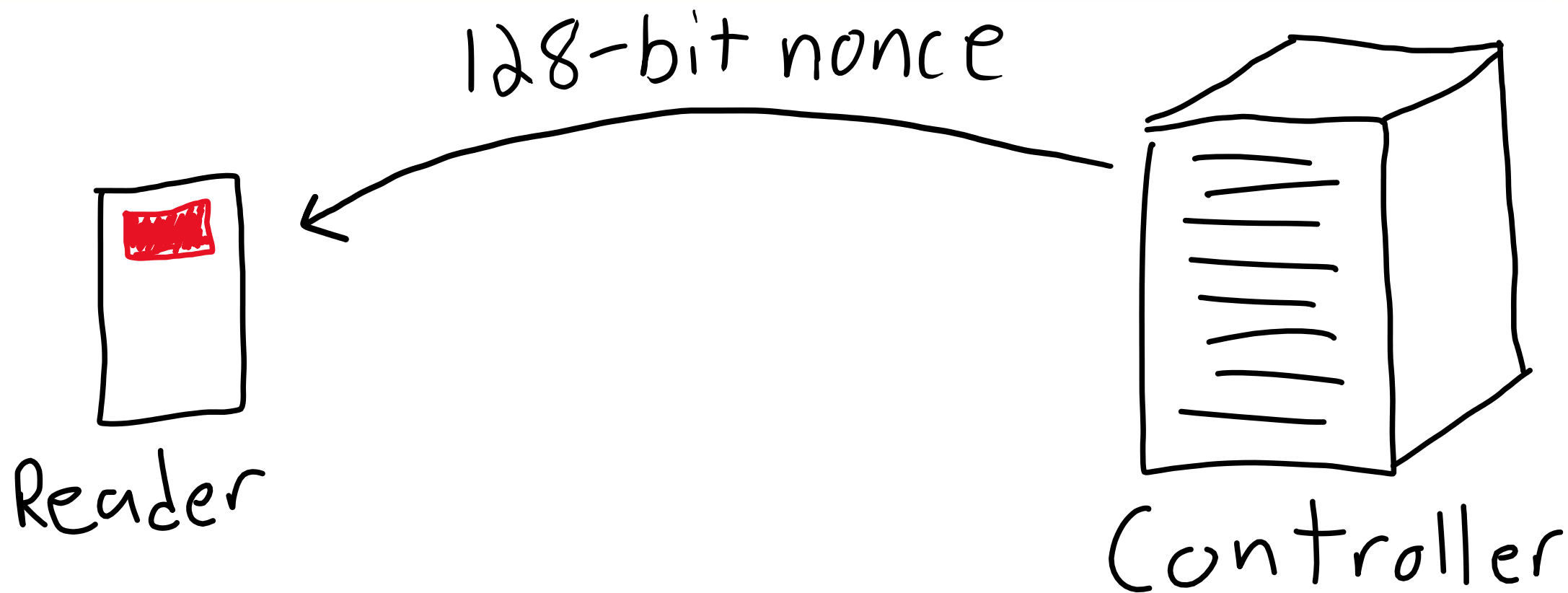


Reader

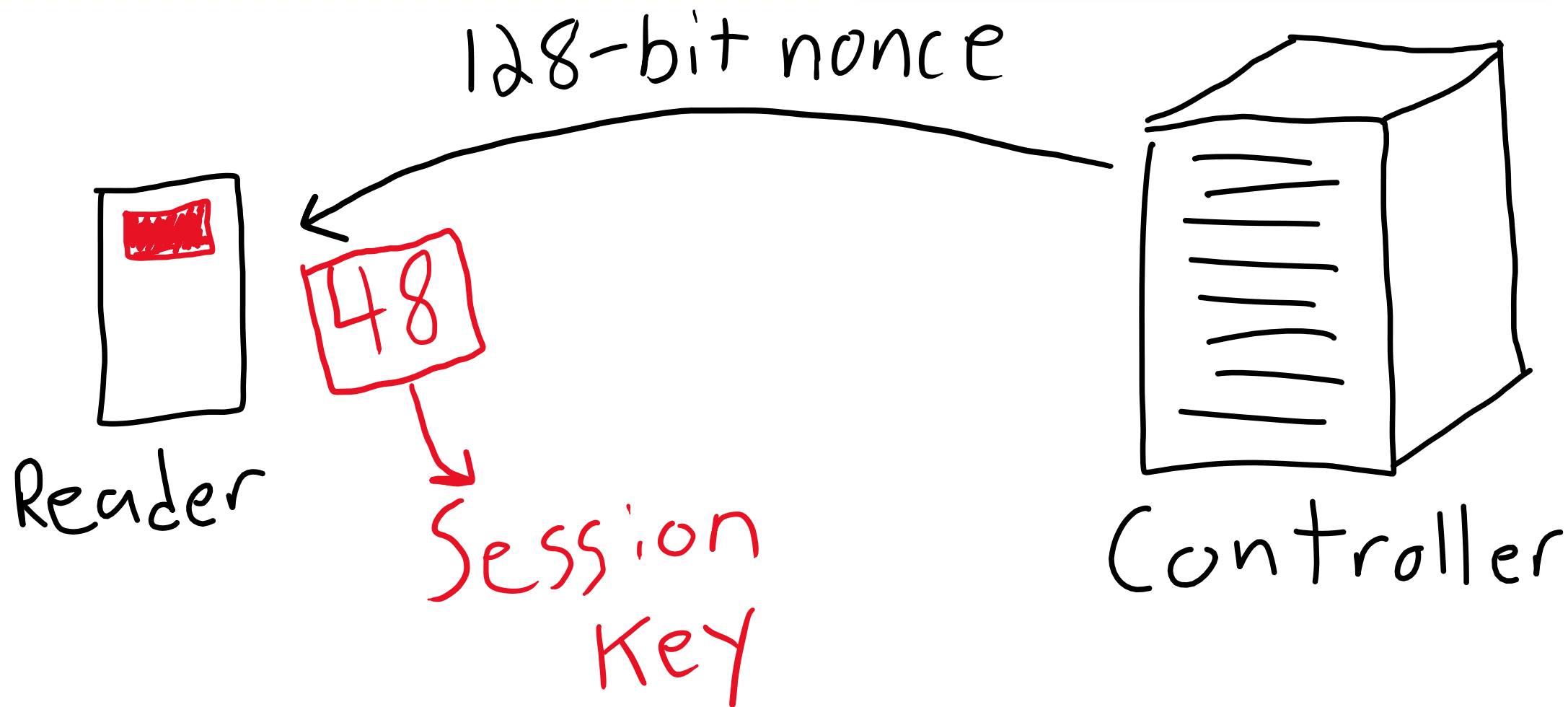


Controller

Session Keys



Session Keys

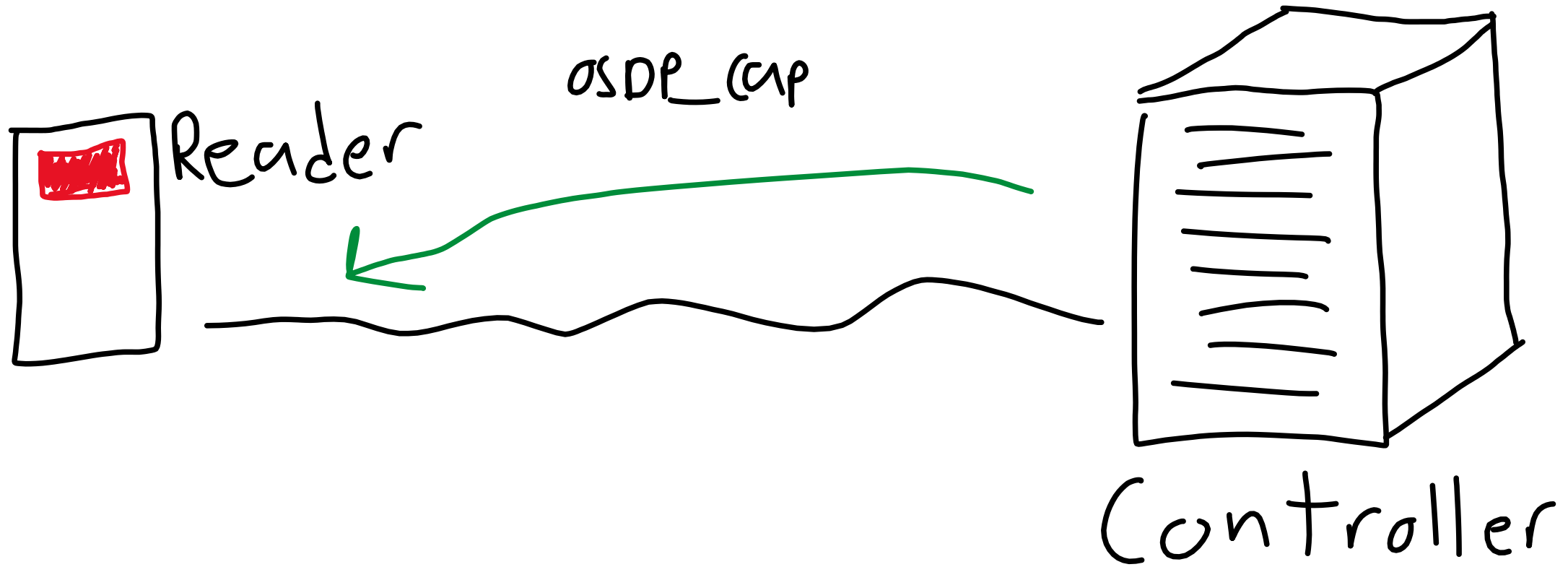




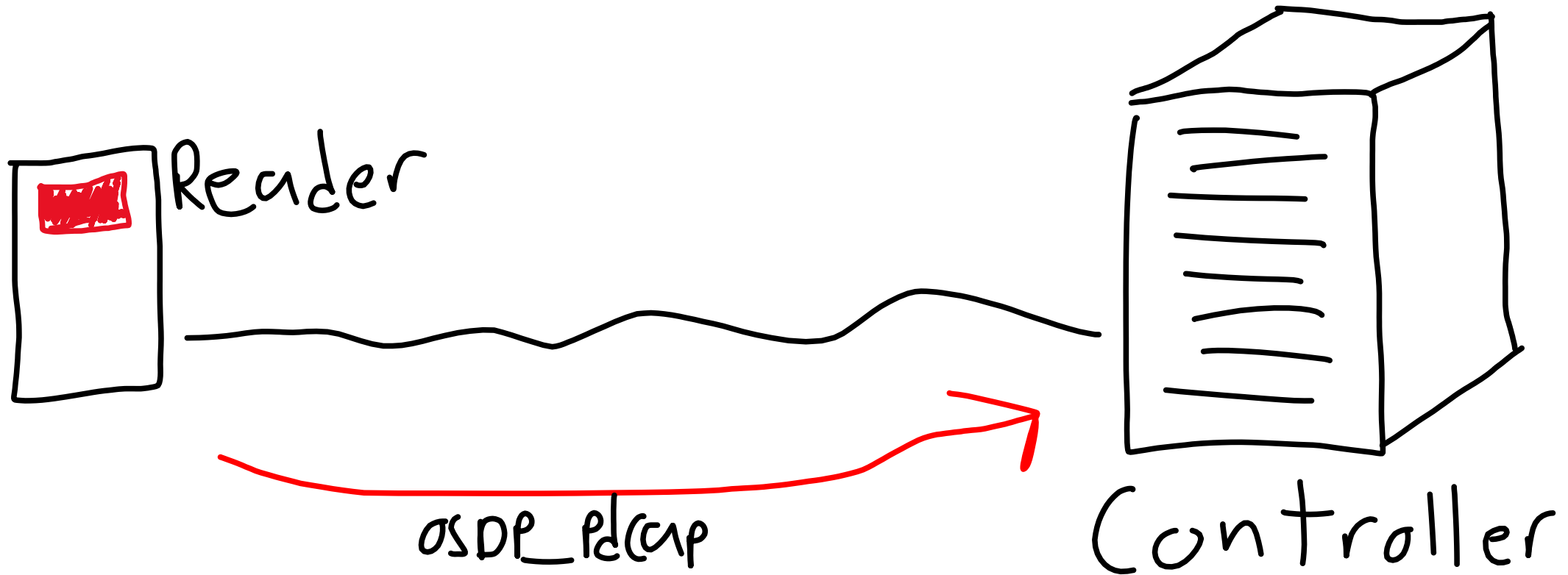
Downgrade

Attack

Me no speak AES



Me no speak AES



Me no speak AES



OSDP_PDCAP = biometrics

Me no speak AES



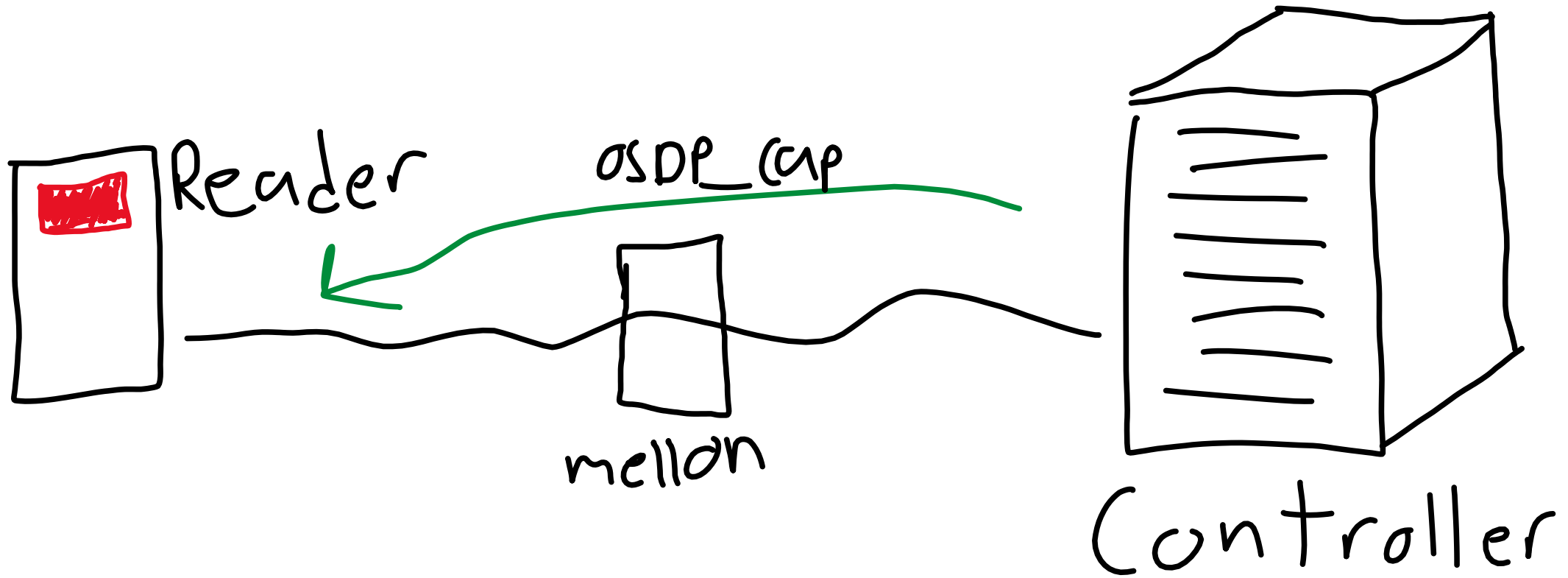
OSDP_PDCMP = KEY/PAD

Me no speak AES

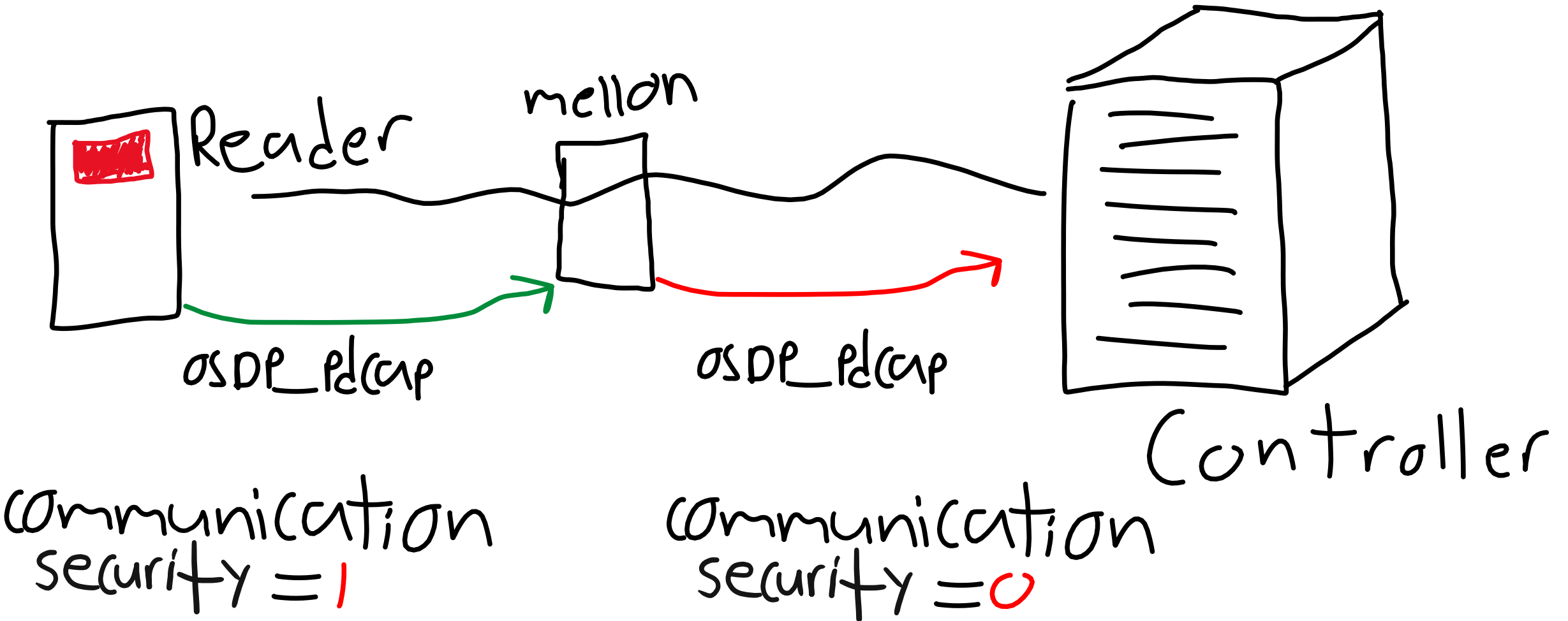


OSDP_PDCAP = Communication
Security

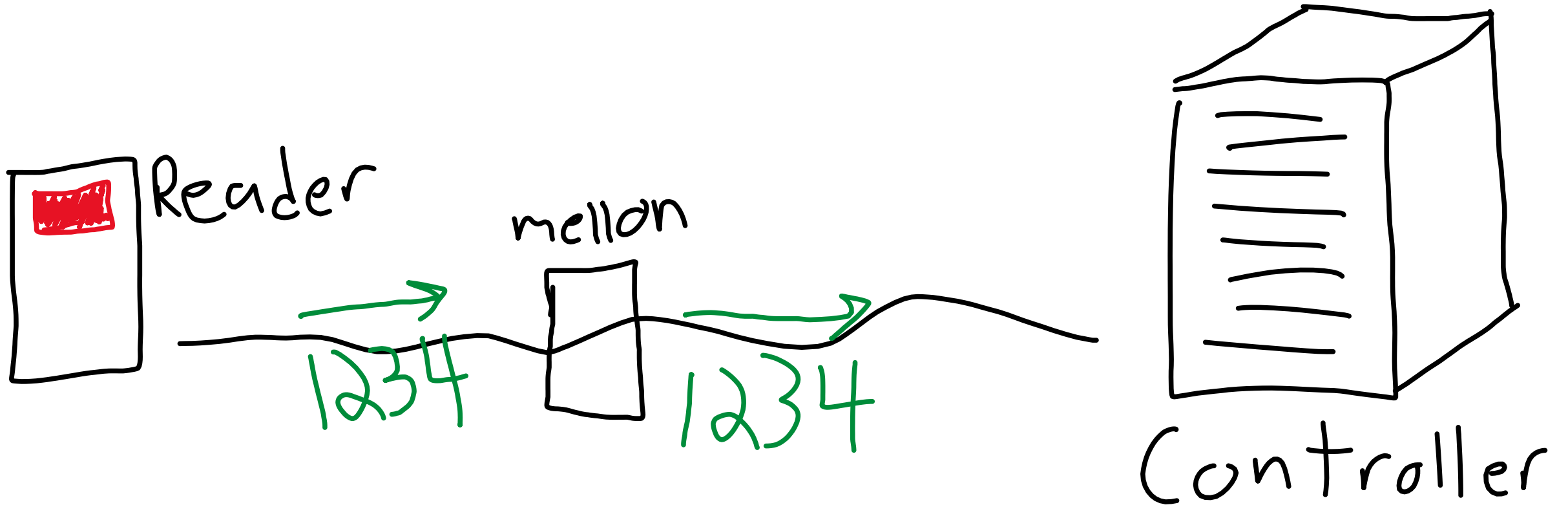
Me no speak AES



Me no speak AES



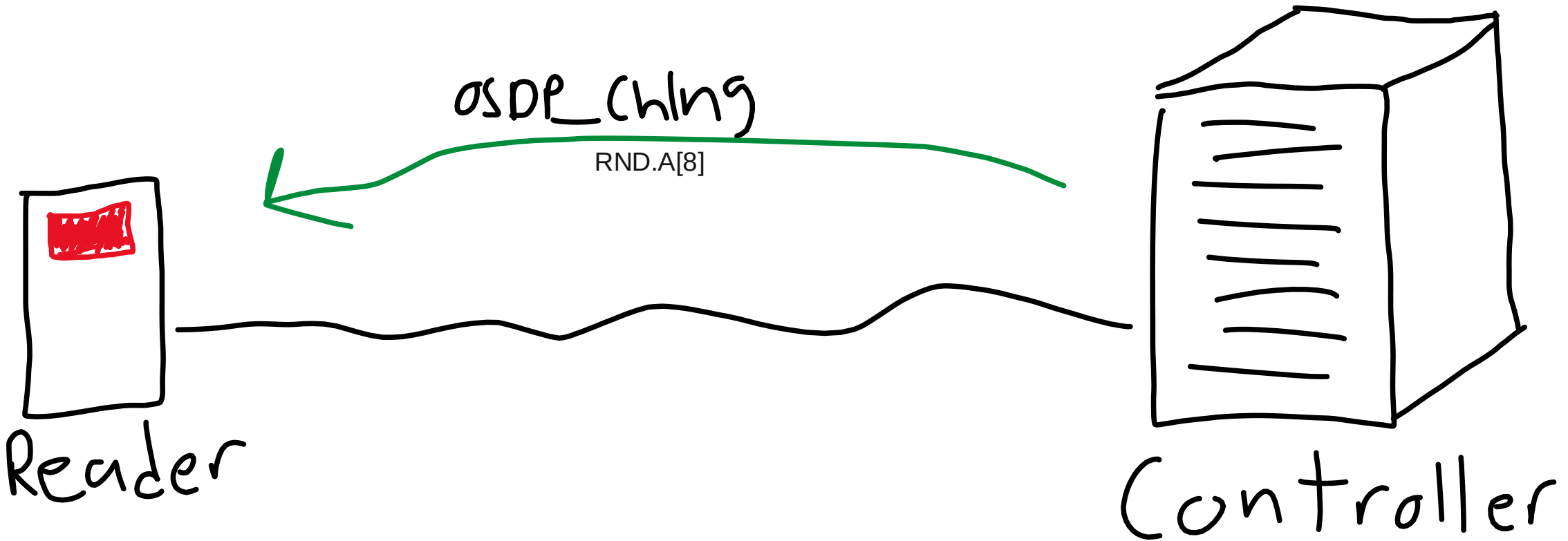
Me no speak AES



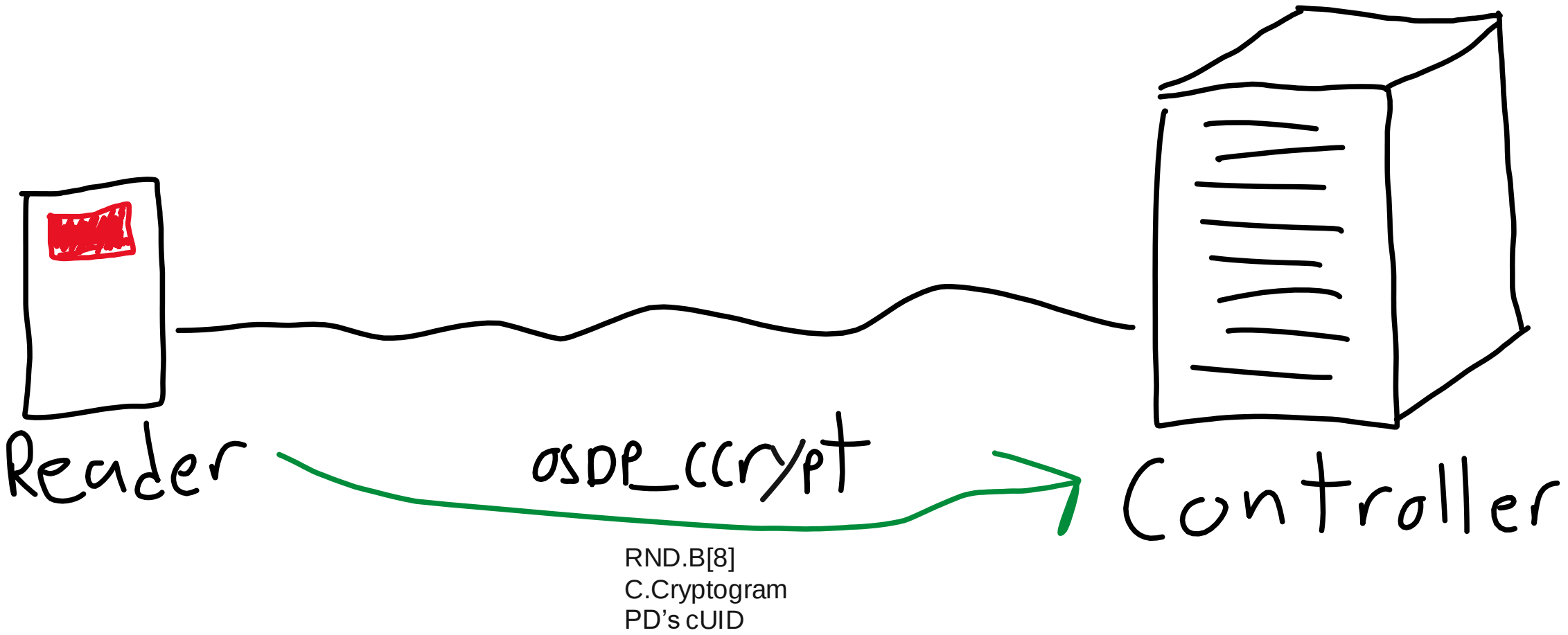


Stop Making
Null Ciphers

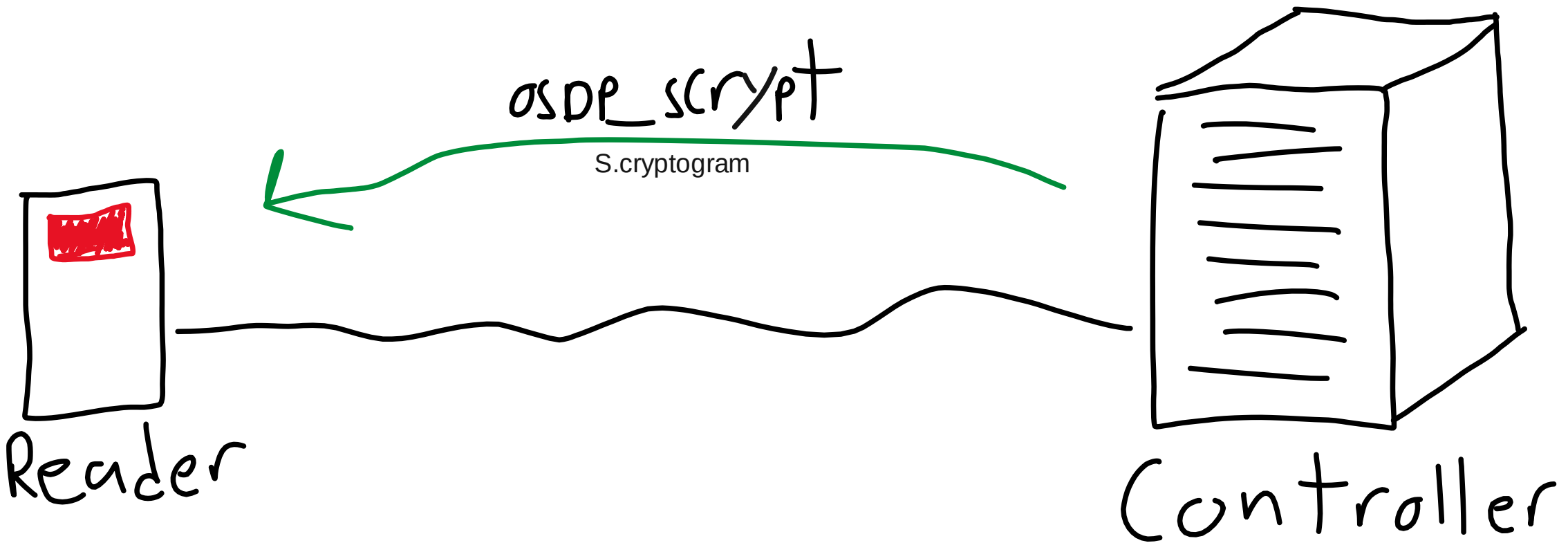
"Secure" Channel - Connection Sequence: SCS_11



"Secure" Channel - Connection Sequence: SCS_12



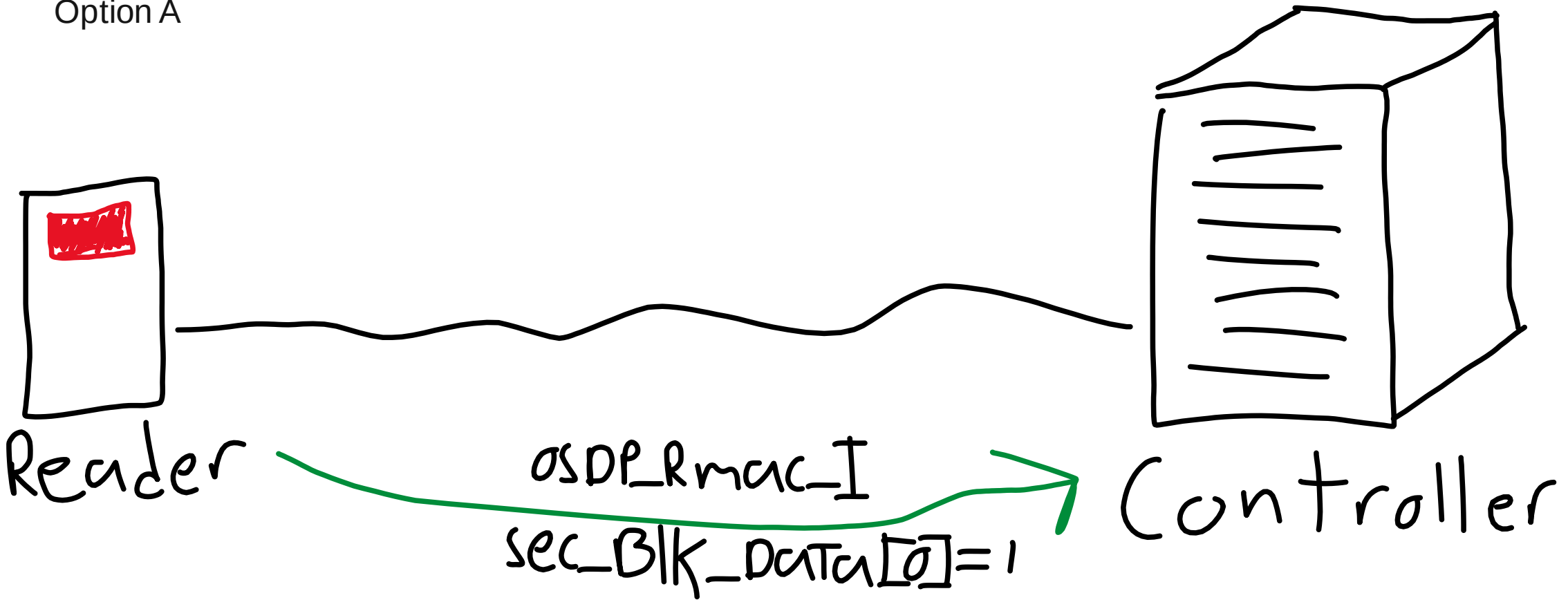
"Secure" Channel - Connection Sequence: SCS_13



"Secure" Channel - Connection Sequence: SCS_14



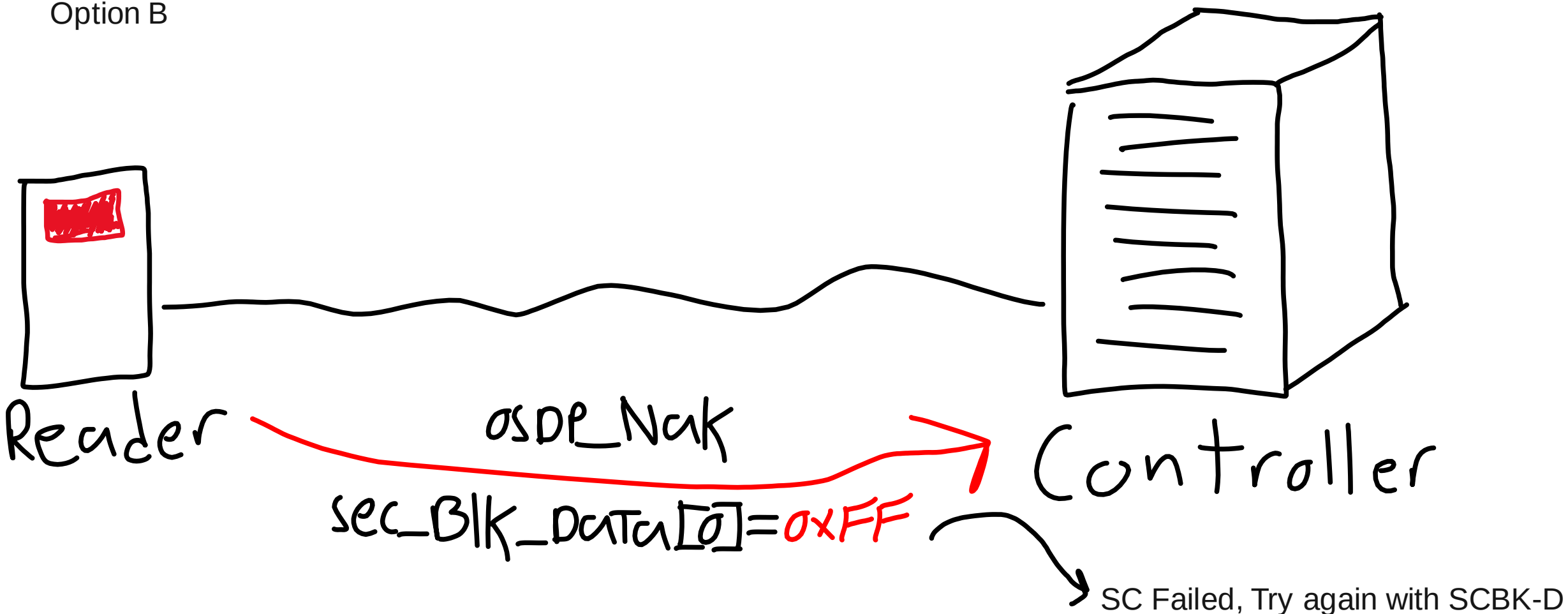
Option A



"Secure" Channel - Connection Sequence: SCS_14



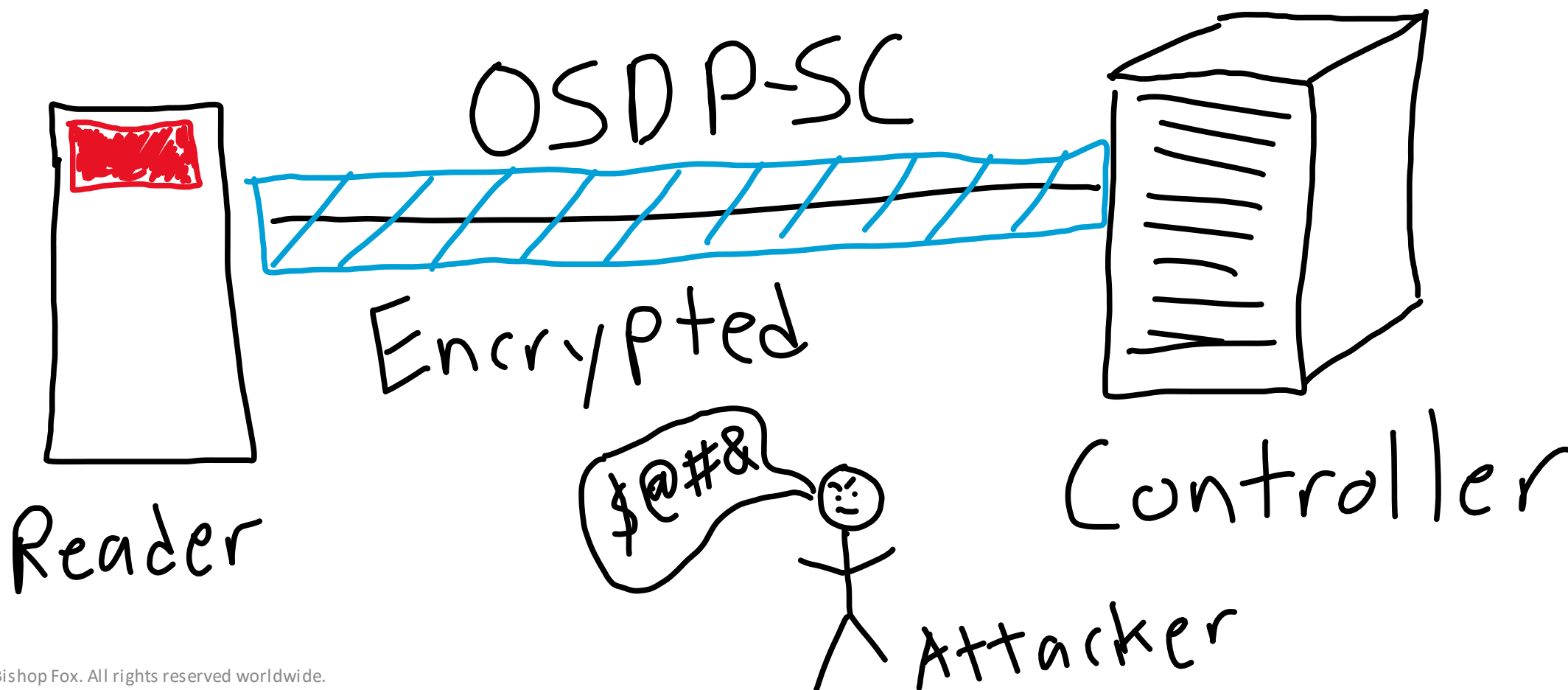
Option B



"Secure" Channel - Established

D.3.2 Communication during a Secure Channel Session

The successful completion of the synchronization sequence SCS_11 through SCS_14 confirms that the CP and PD established a valid Secure Channel Session. In order to maintain the SCS, the CP must send each message with SEC_BLK_TYPE set to SCS_15 or SCS_17, and the PD must send each if its replies with SEC_BLK_TYPE set to SCS_16 or SCS_18.



"Secure" Channel - Connection Sequence: SCS_15 & SCS_16



D.3.2.1 SCS_15 CP->PD

The DATA field is sent in plain text (unencrypted)

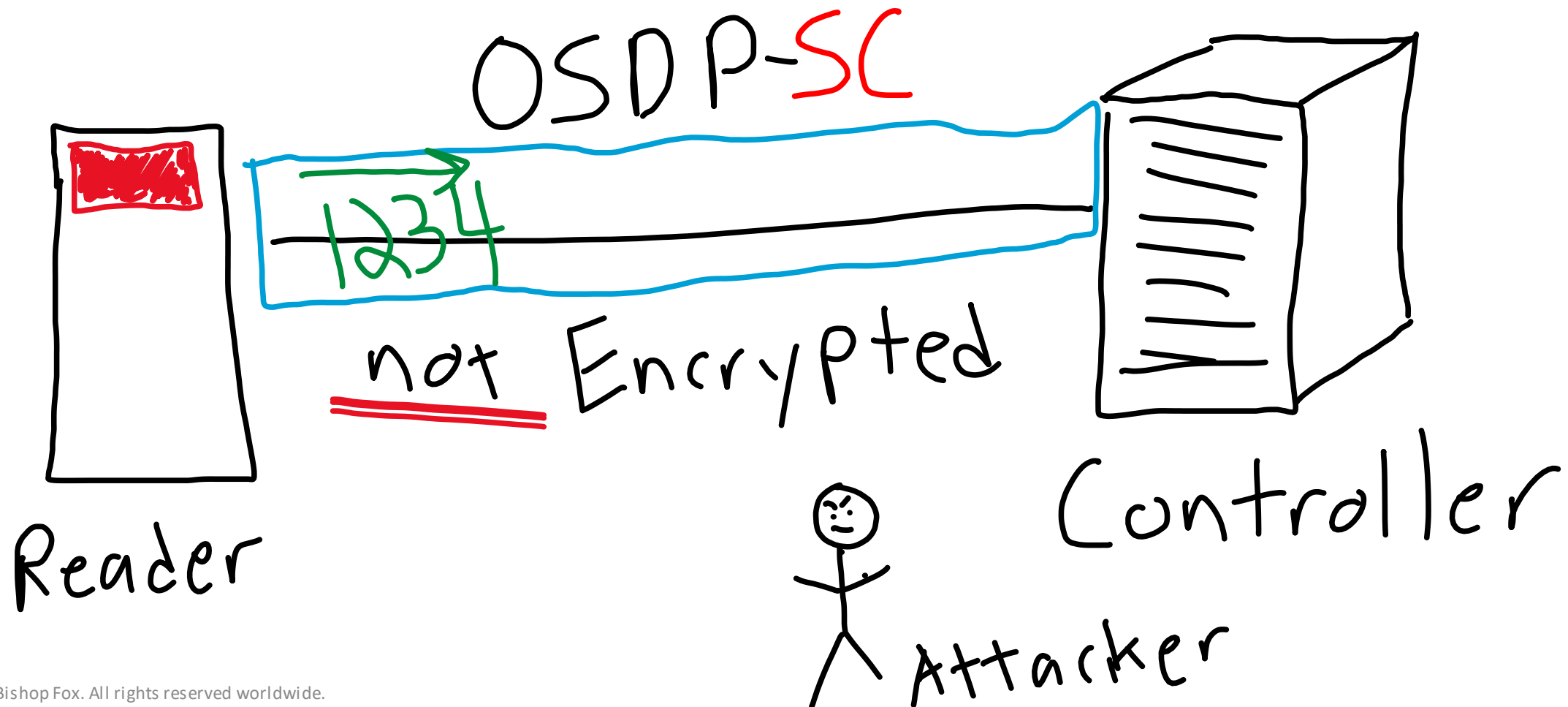
Note: this form provides Message Authentication, but does not contain encrypted DATA.

D.3.2.2 SCS_16 PD->CP

The data field is sent in plain text (unencrypted)

Note: this form provides Message Authentication, but does not contain encrypted DATA.

"Secure" Channel - Established



"Secure" Channel - SCS_15 & SCS_16



Install-mode

Attack

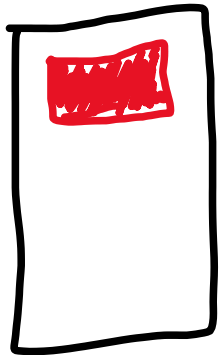
SSH Security Model

SSH Security Model

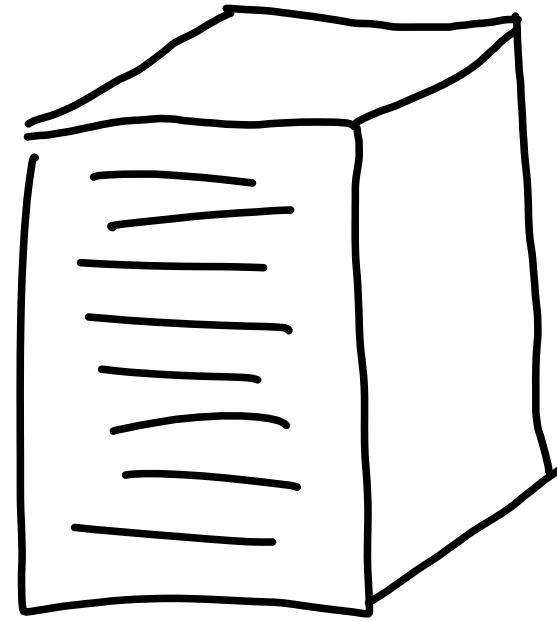
One-time Insecure Setup

SSH Security Model

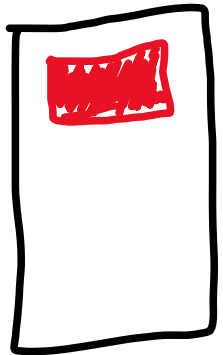
One-time Insecure Setup



Reader

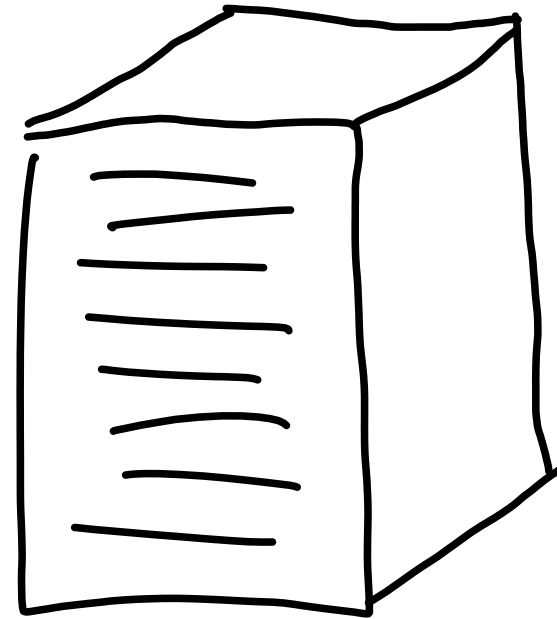


Controller



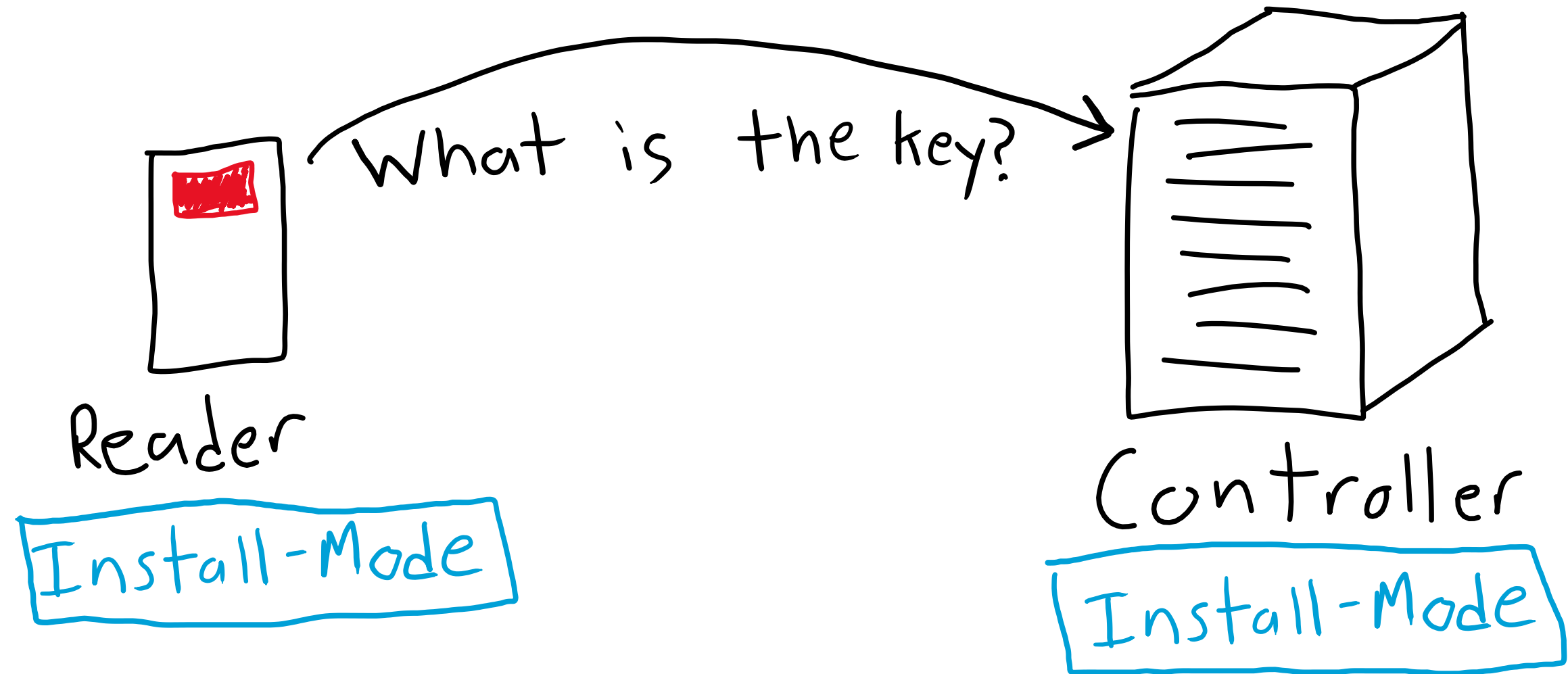
Reader

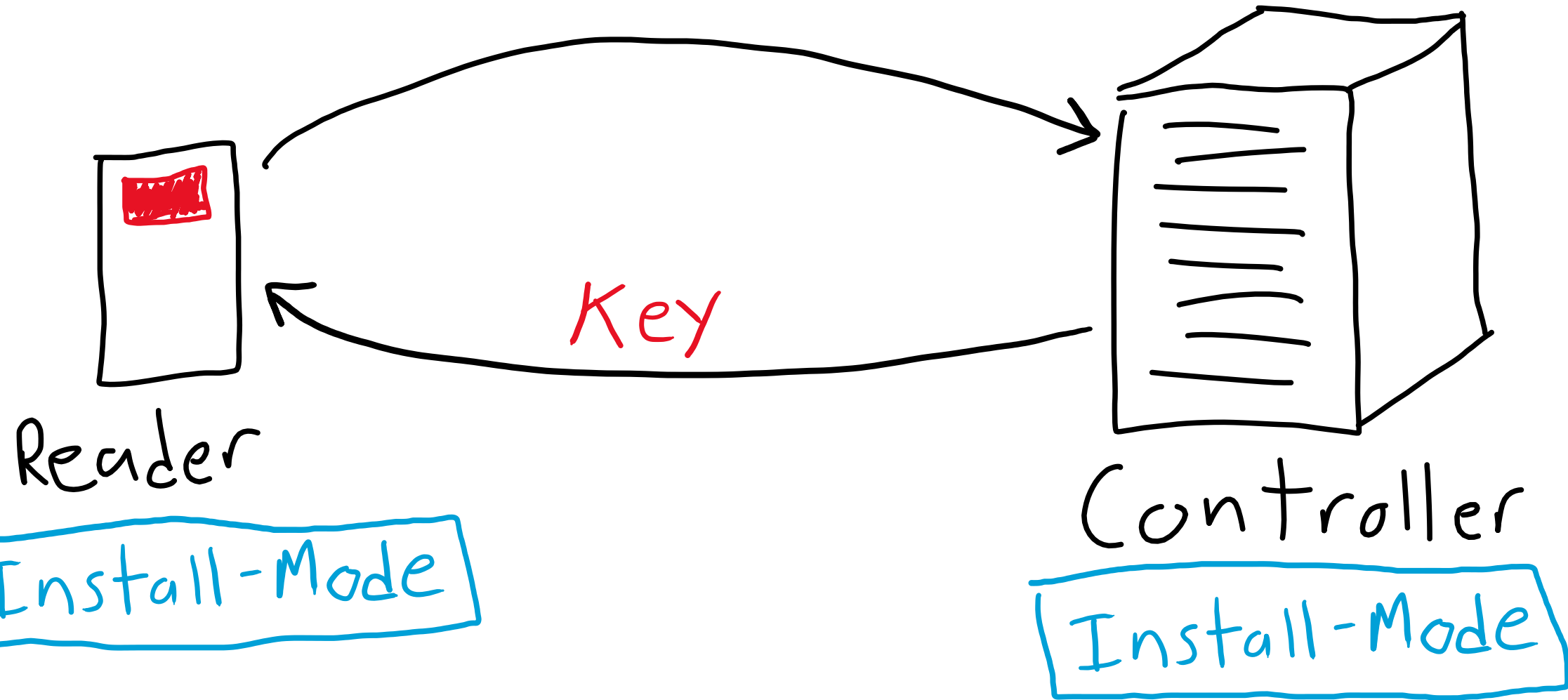
Install-Mode

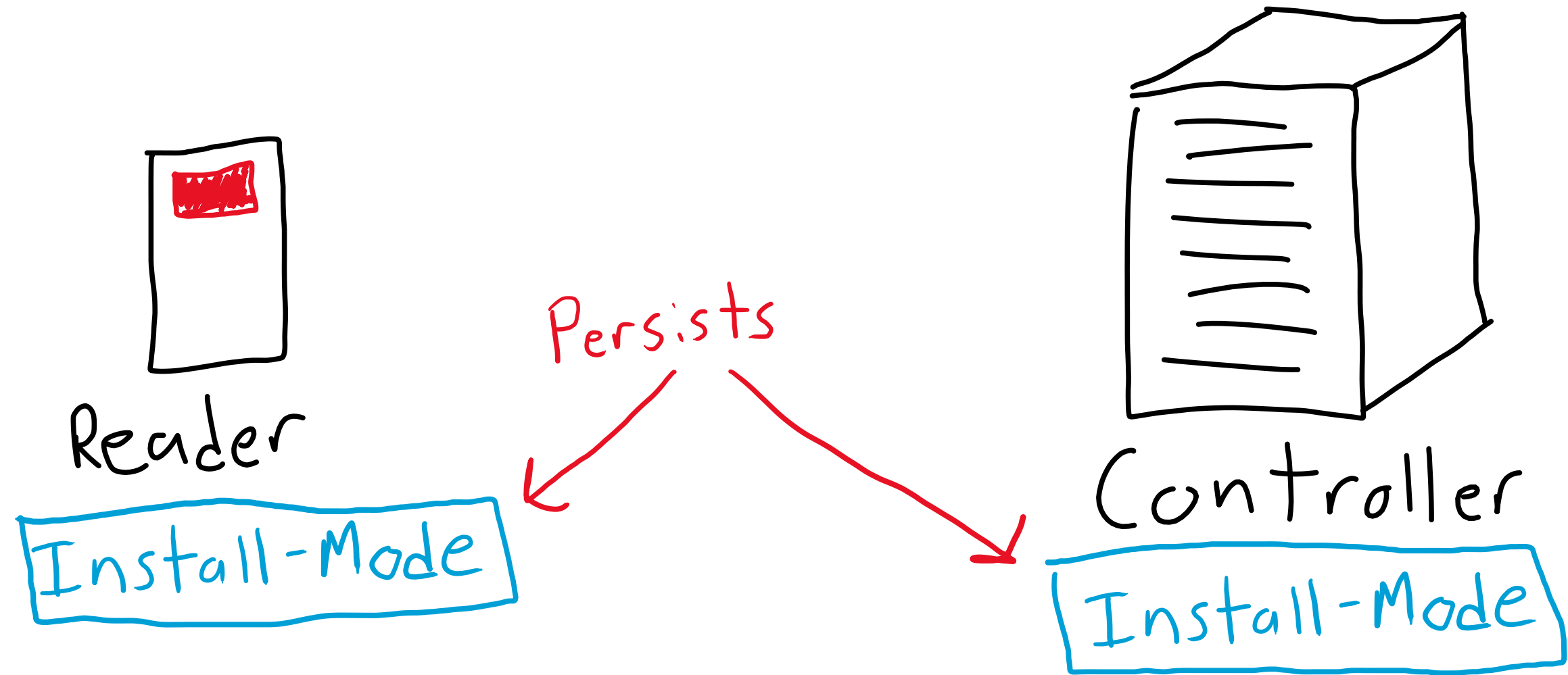


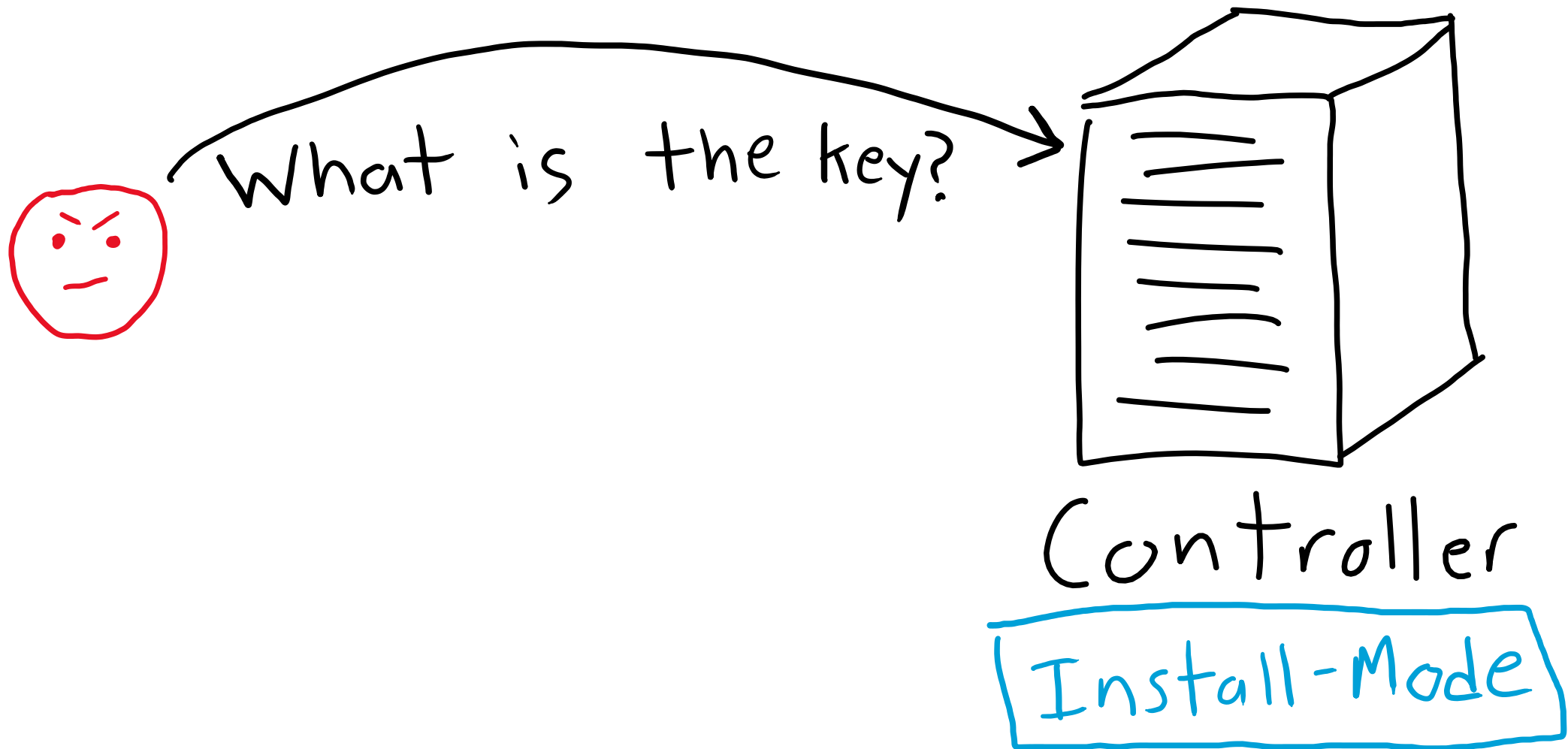
Controller

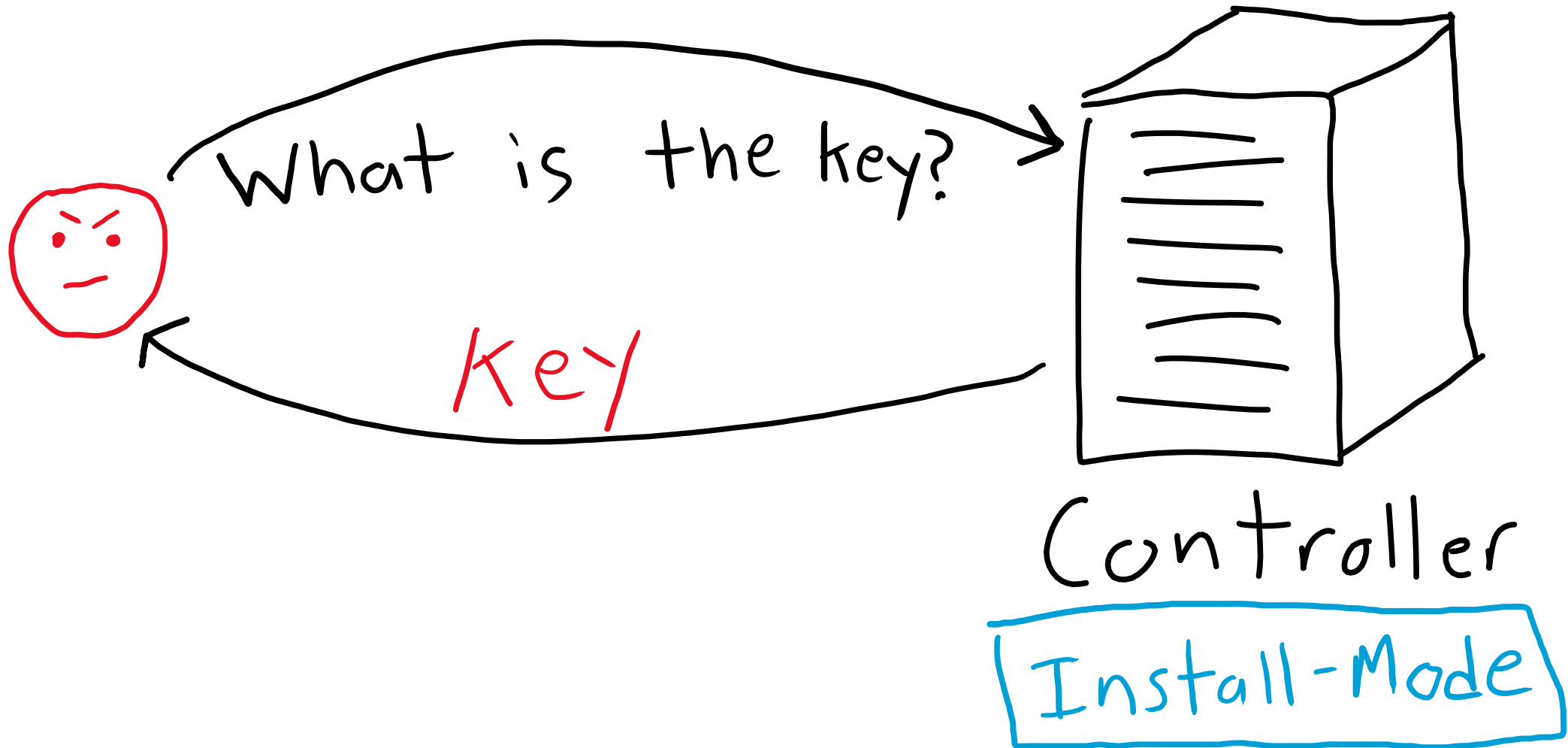
Install-Mode











Where does the vuln lie?



Protocol

Where does the vuln lie?



Protocol

Library

Where does the vuln lie?



Protocol

Library

Config

Where does the vuln lie?



Protocol

Library

Config

Documentation

Where does the vuln lie?



Protocol

Library

Config

Documentation

Implementation

Where does the vuln lie?



Protocol

Library

Config

Documentation

Implementation

Marketing



We never said we'd encrypt
ALL of the data...

SCS_17&SCS_18: The whole packet is encrypted...right?



OSD P-SC

expectation



SCS_17&SCS_18: The whole packet is encrypted...right?



OSD P-SC
reality

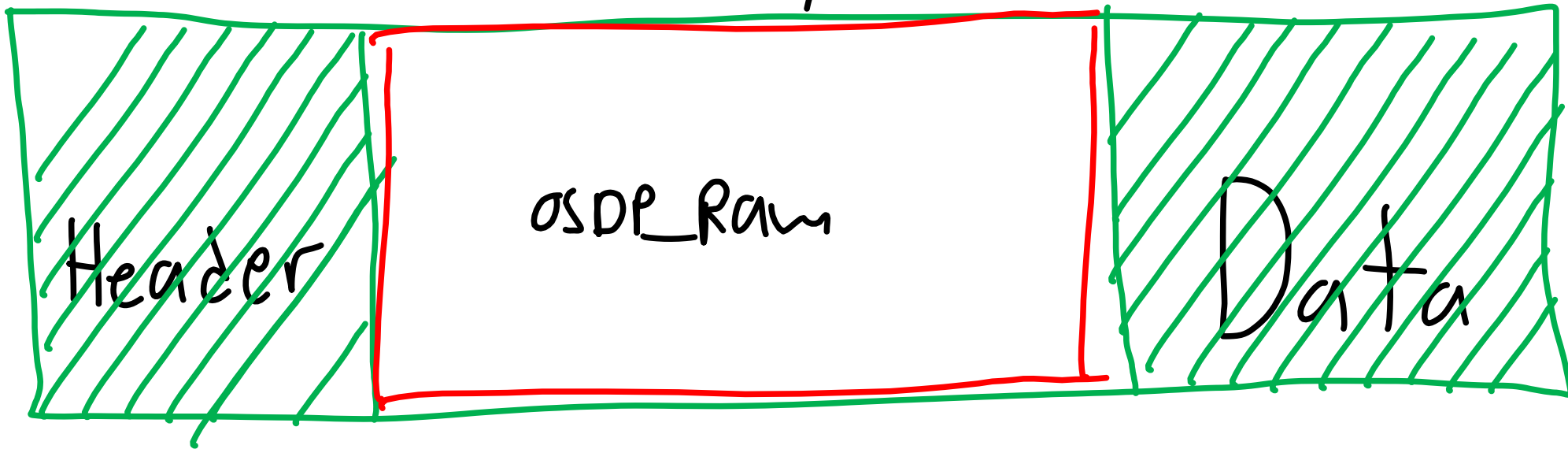


SCS_17&SCS_18: The whole packet is encrypted...right?

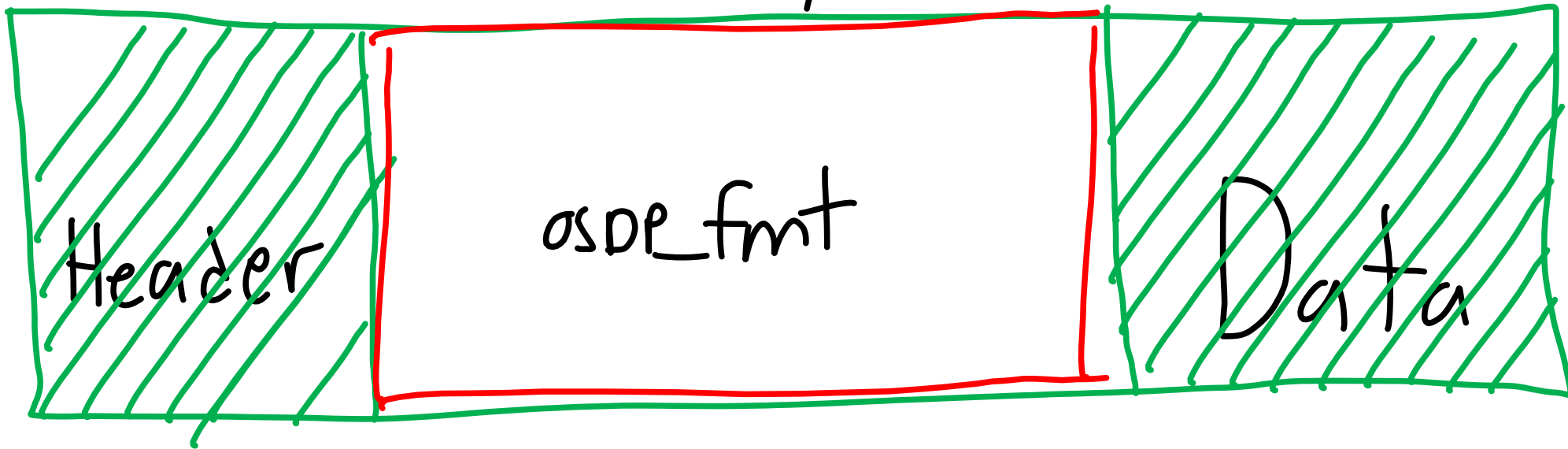


command bytes = P2/KP data transmitted

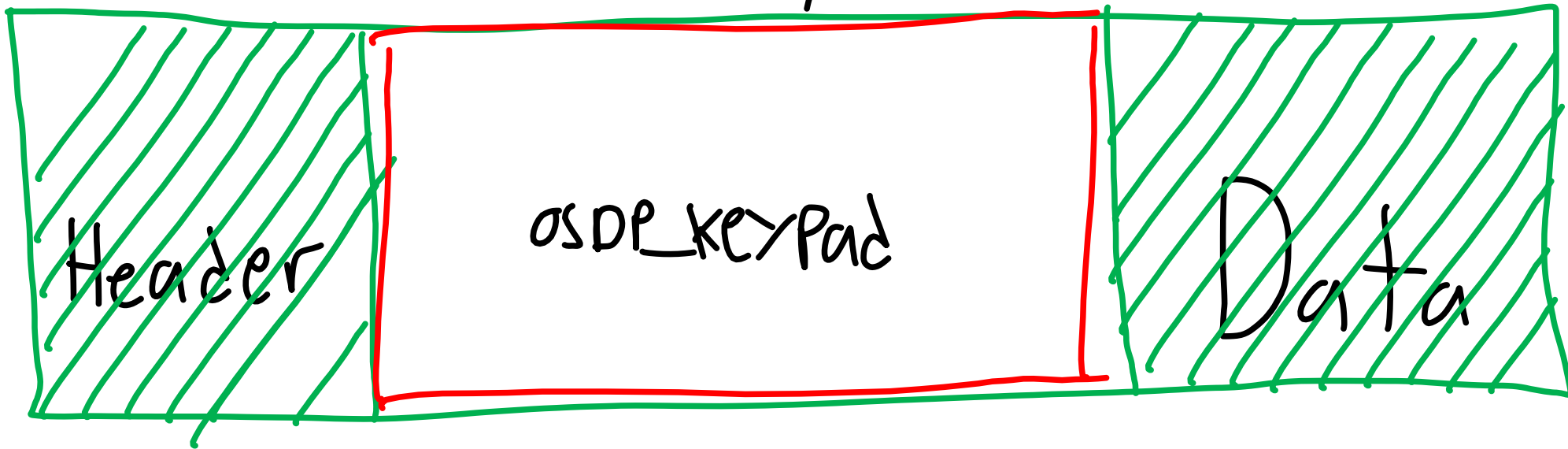
OSDP-SC
reality



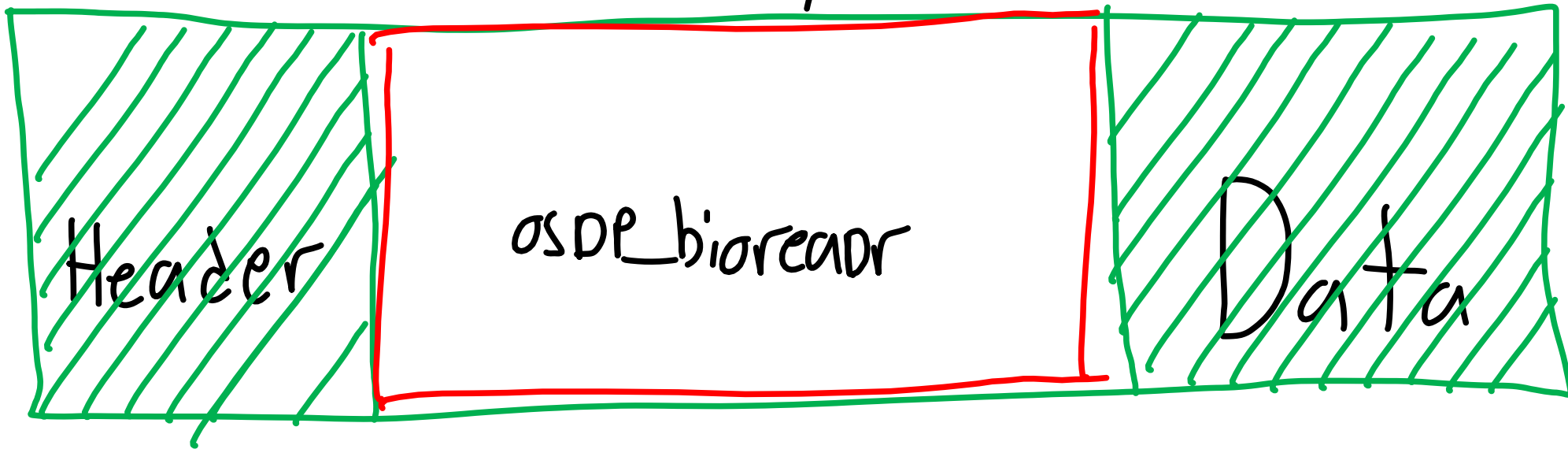
OSDP-SC reality



OSDP-SC
reality



OSDP-SC
reality



OSDP-SC
reality





Weak Keys Attack

Weak Keys



not Protocol Specific



$$SCBK = Enc(Uid || \sim(Uid, MK))$$

S-ENC Derivation



D.4.2 Session Key Derivation

A set of three keys are derived and used for each secure communication session. The derivation operation uses the SCBK and encrypts a data block generated for each key. The data block for each key is as follows:

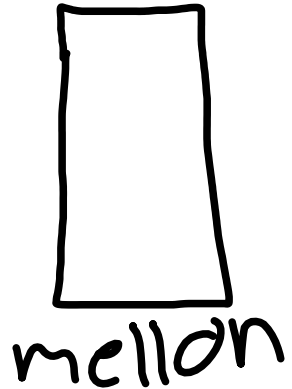
S-ENC: 0x01,0x82,rnd[0],rnd[1],rnd[2],rnd[3],rnd[4],rnd[5],0,0,...

Introducing: Weak keys

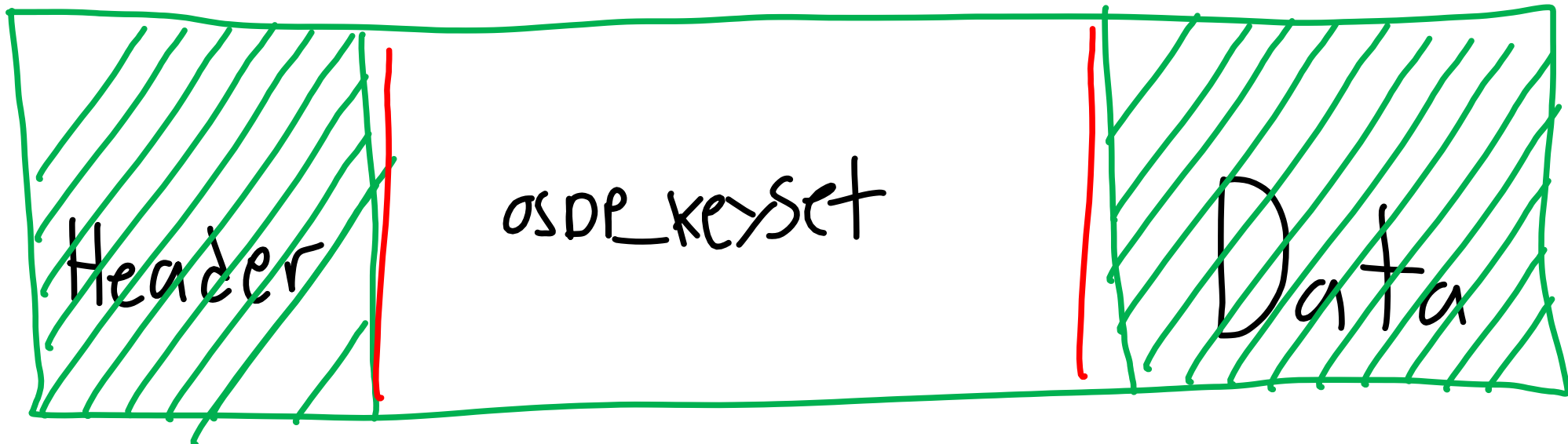


```
9
10     name = PD0
11     channel_type = uart
12     channel_device = /dev/ttyUSB0
13     channel_speed = 9600
14     scbk = 000102030405060708090a0b0c0d0e0f
15
16     address = 1
```

Using weak keys cant possibly backfire...



Key 1 = 0x00, 0x00, 0x00...
 Key 2 = 0x01, 0x02, 0x03...
 Key 3 = 0x30, 0x31, 0x32...
 Key 4 = ...



Encryption is not
magic fairy dust

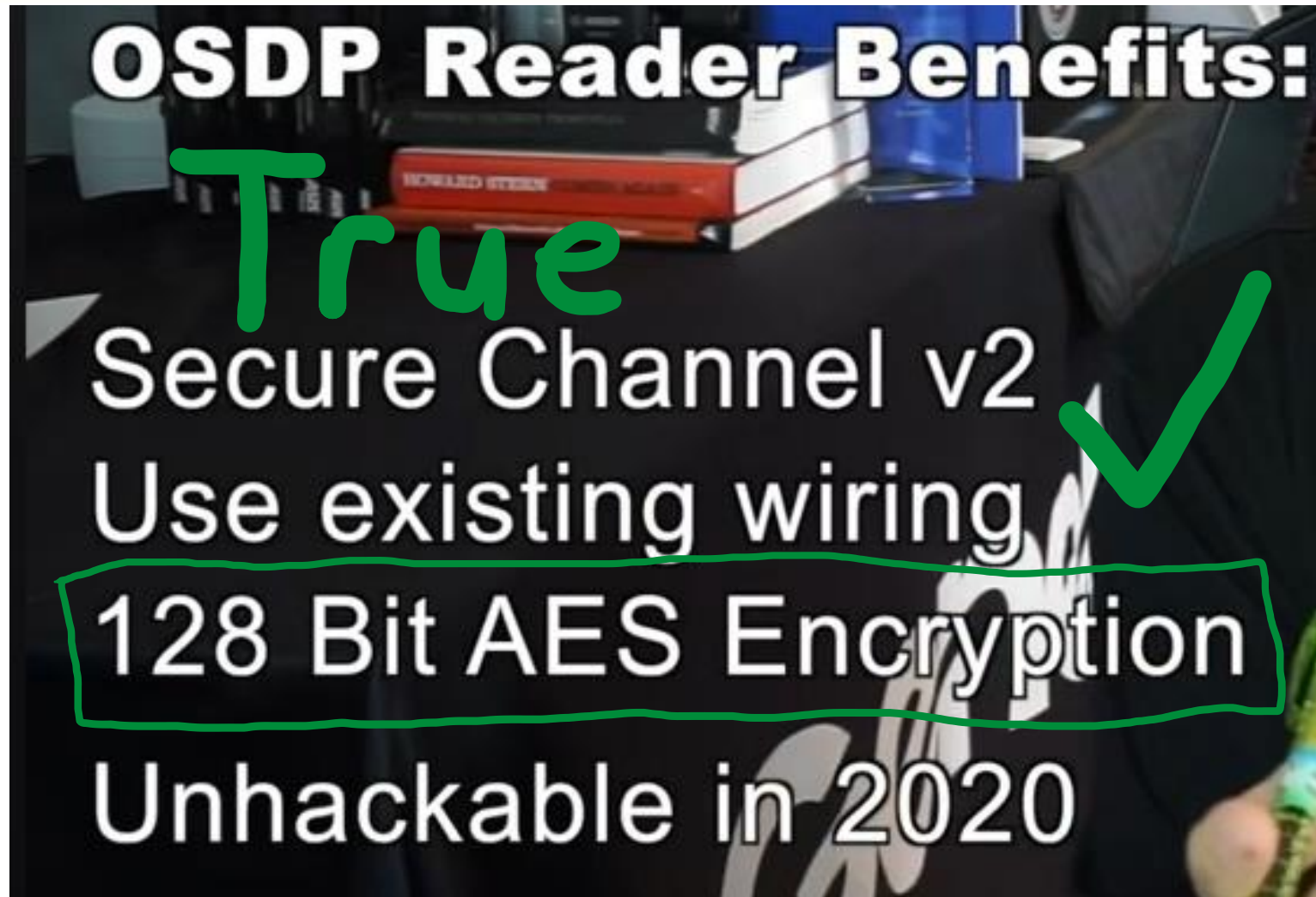
OSDP Reader Benefits:

Secure Channel v2

Use existing wiring

128 Bit AES Encryption

Unhackable in 2020

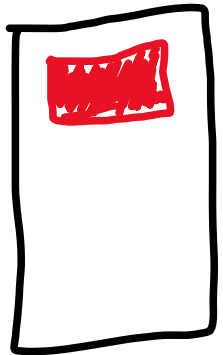


But ONLY AES

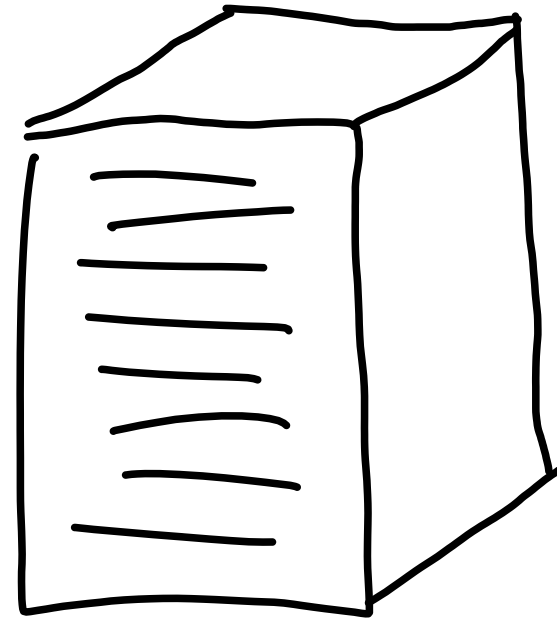
But ONLY AES

No asymmetric crypto

Key Exchange

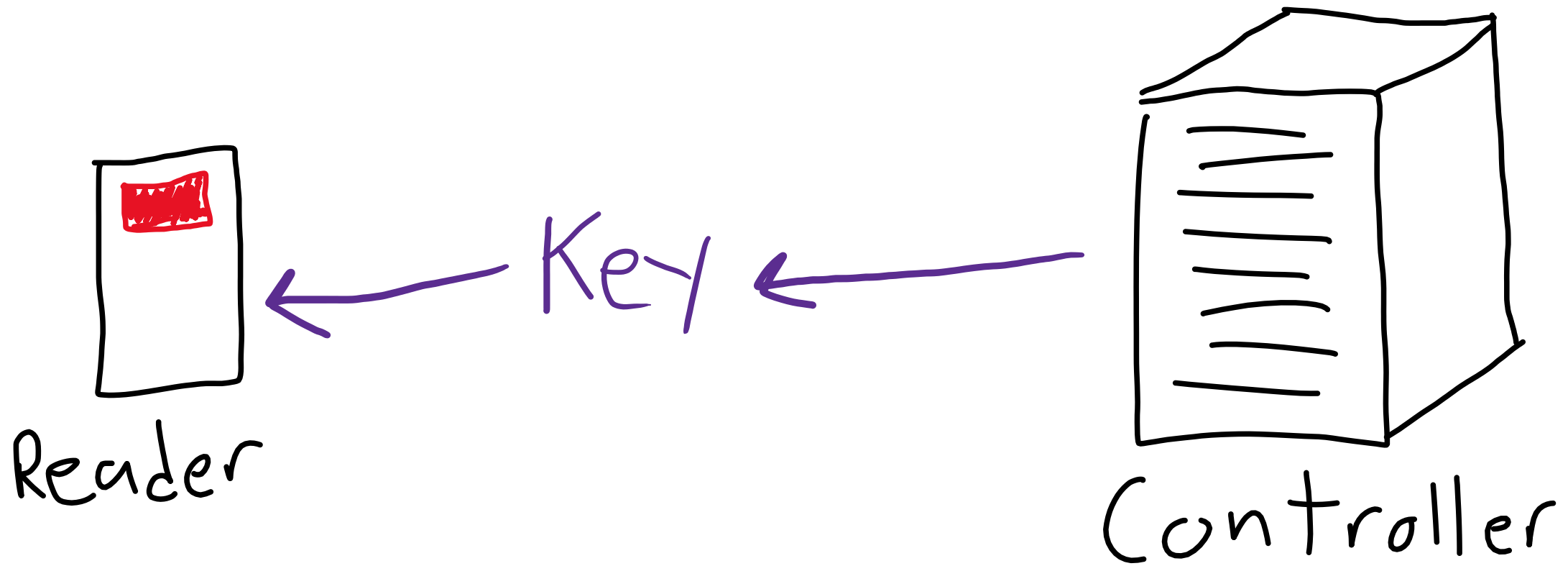


Reader

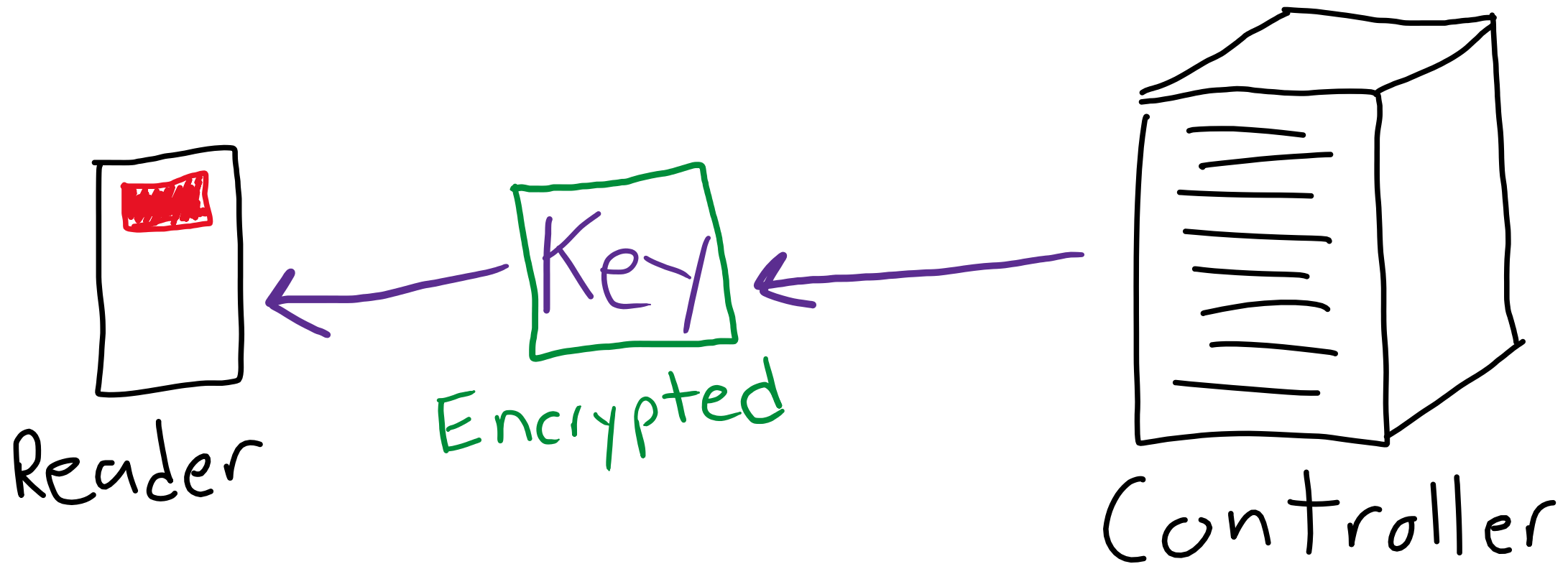


Controller

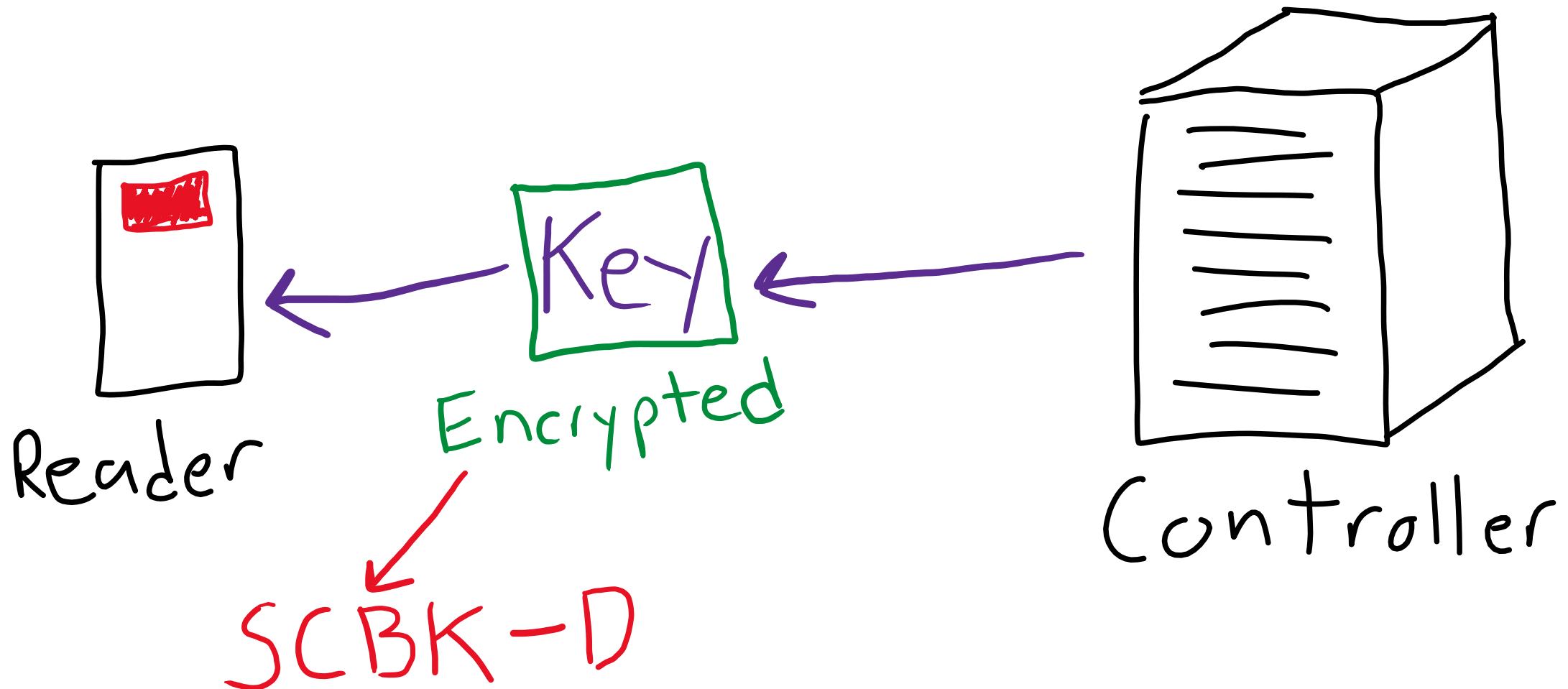
Key Exchange



Key Exchange



Key Exchange



SCBK - D

S C B K - D

Secure

S C B K - D
e c h
c a
u n
r e
e l

S C B K - D
e c h a s
c u n n e
r e l

S C B K - D
e c h a s e
c u n n e
r e i

S C B K - D
e c h a s e
c h a n n e l
u r e
r e
y
e f a u t

Key Exchange



S C B K - D
e h a e
c h a y
r n s
e n e
- -
t

0x30, 0x31,
0x32, 0x33,
...
0x3e, 0x3f



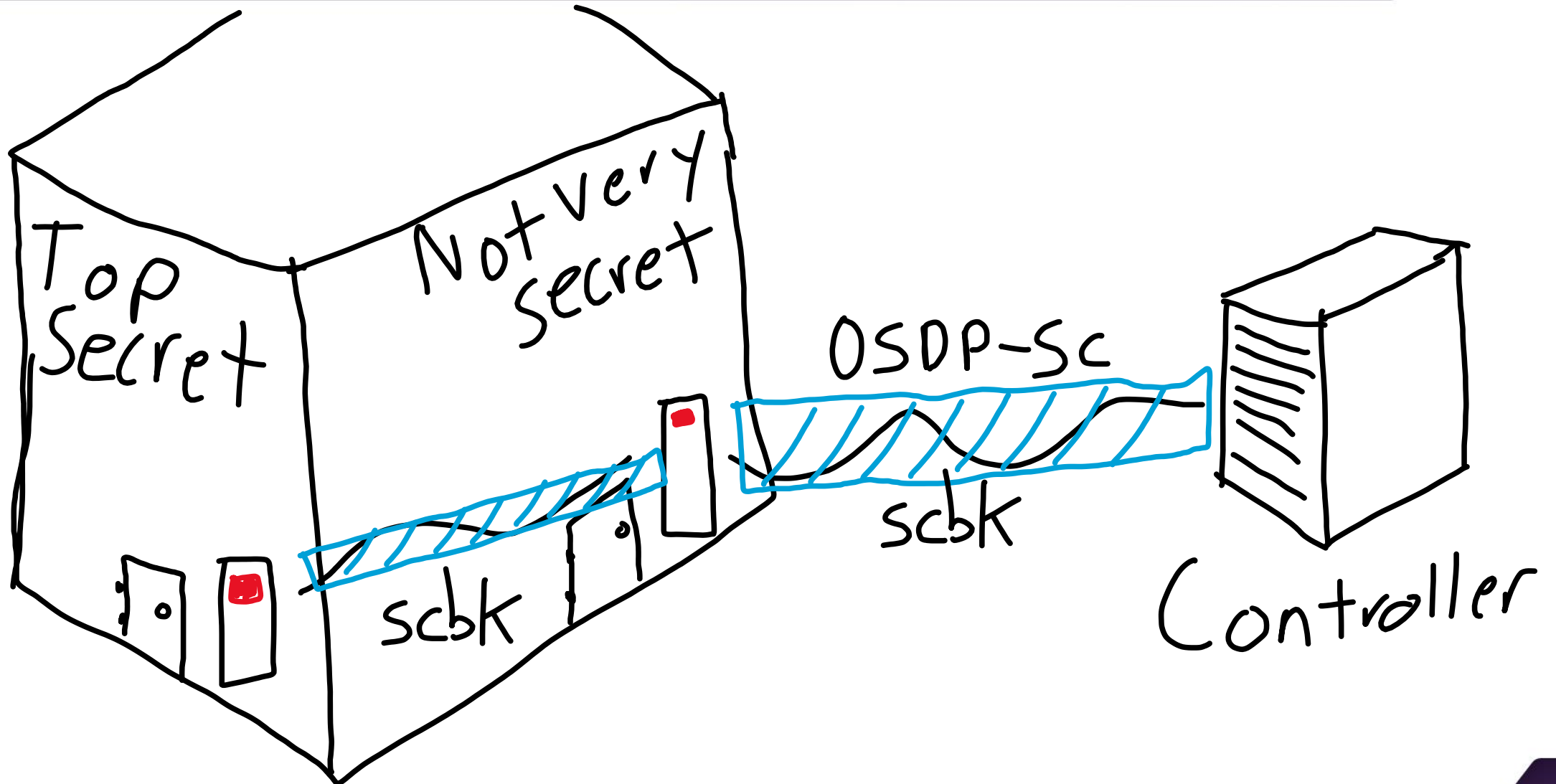
Keyset Capture

I've set everything up securely, I'm not affected by any of this

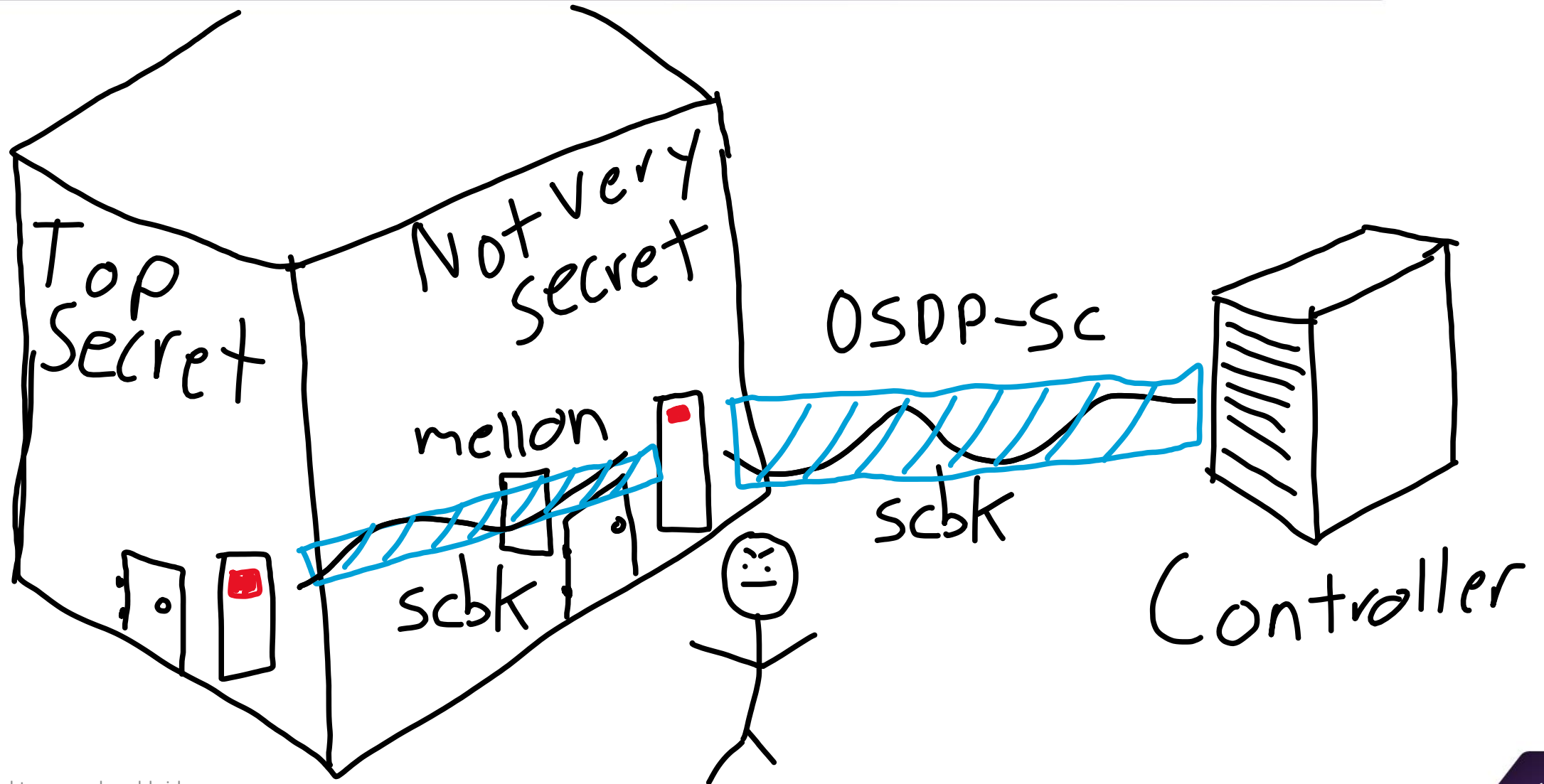


- ✦ **"But, a lot of this can be avoided by configuring the device properly"**
- ✦ **If best practices were normally followed and devices were set up securely, many of us would be out of a job.**
- ✦ **Remember the broadcast nature of the protocol?**

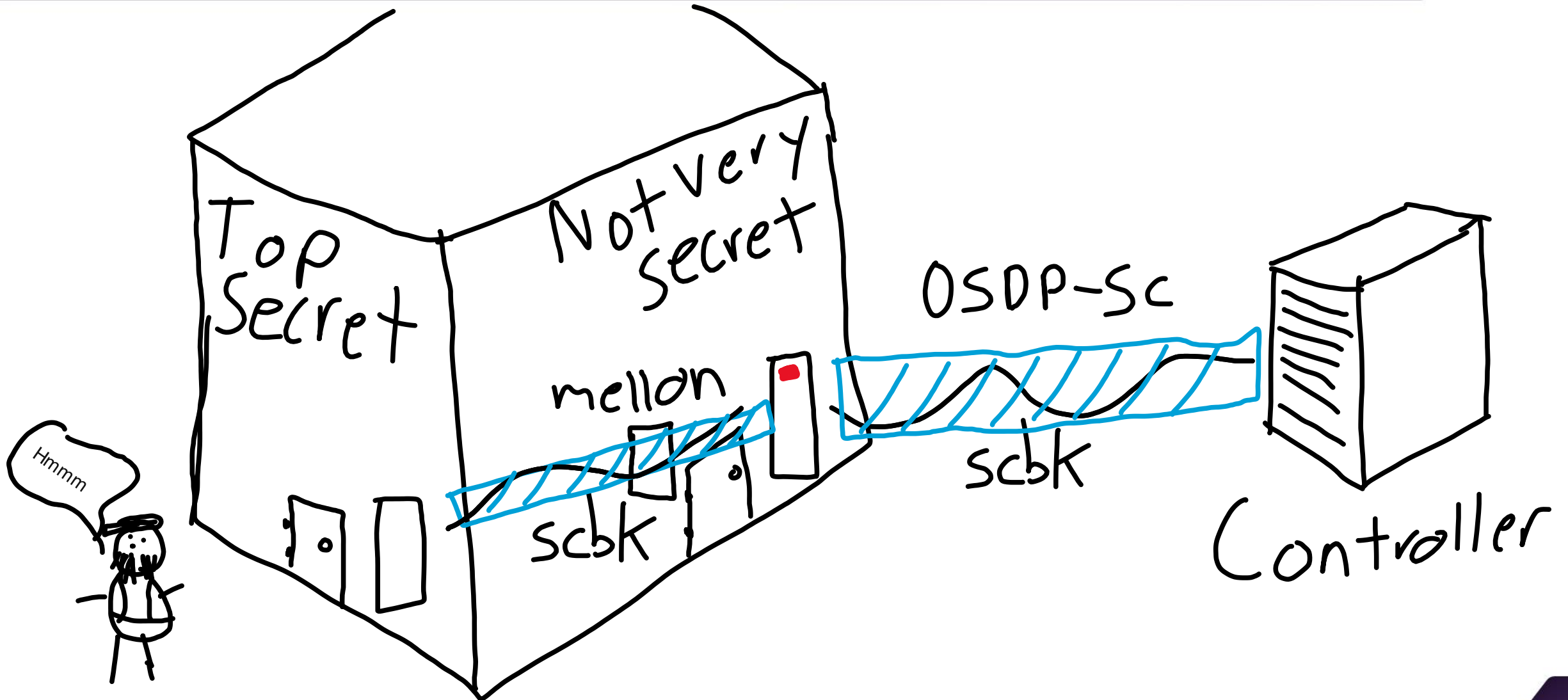
RS-485



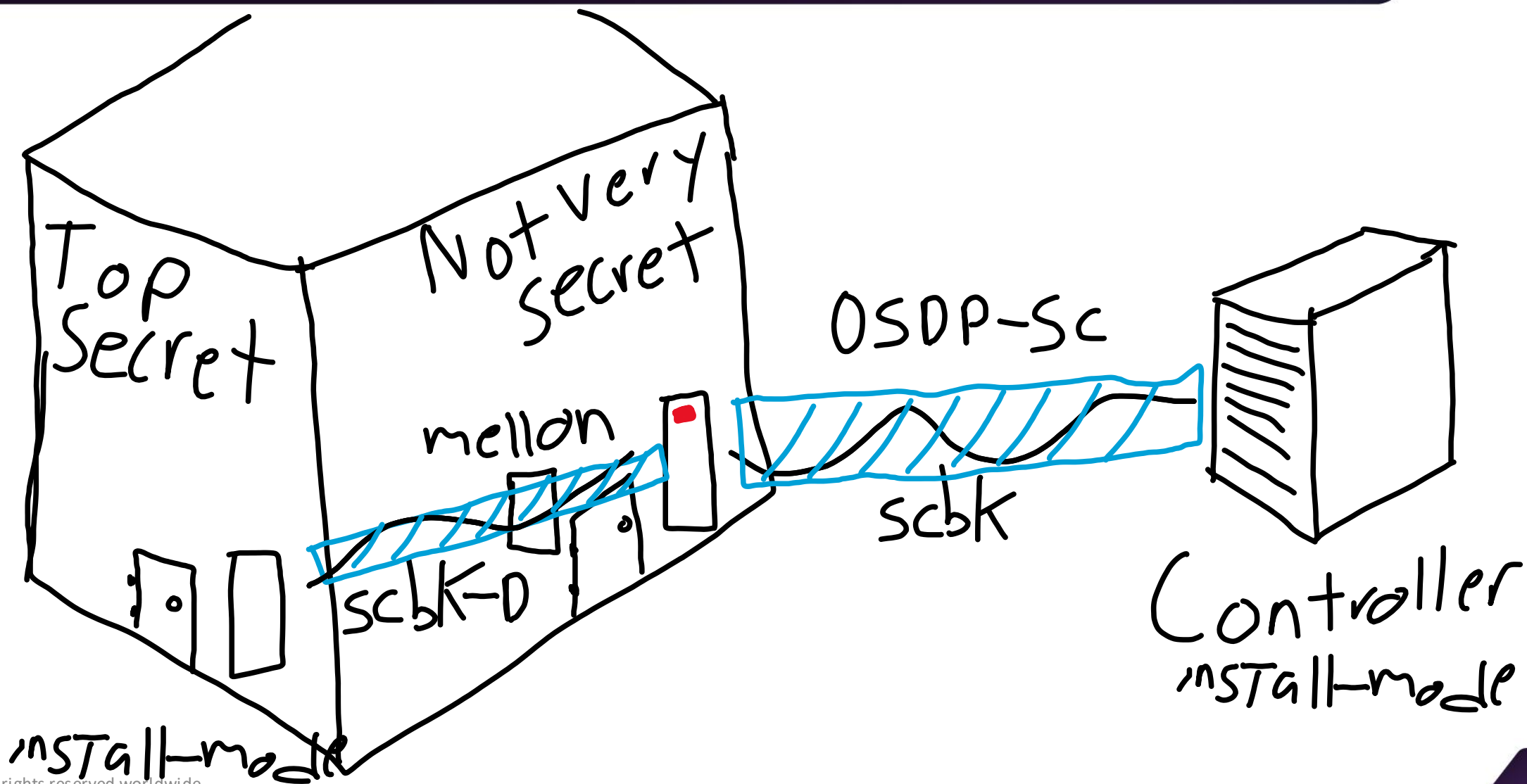
RS-485



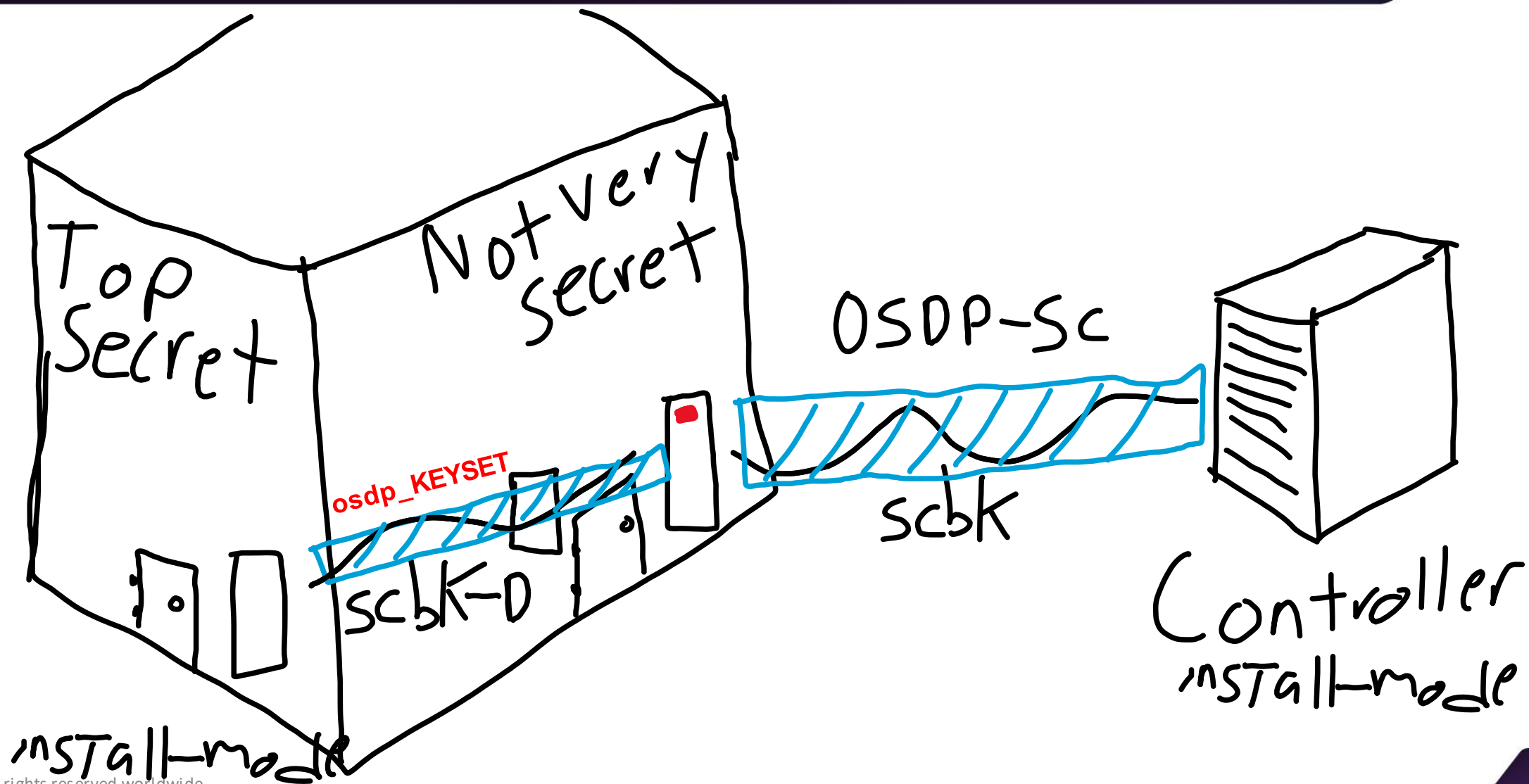
RS-485



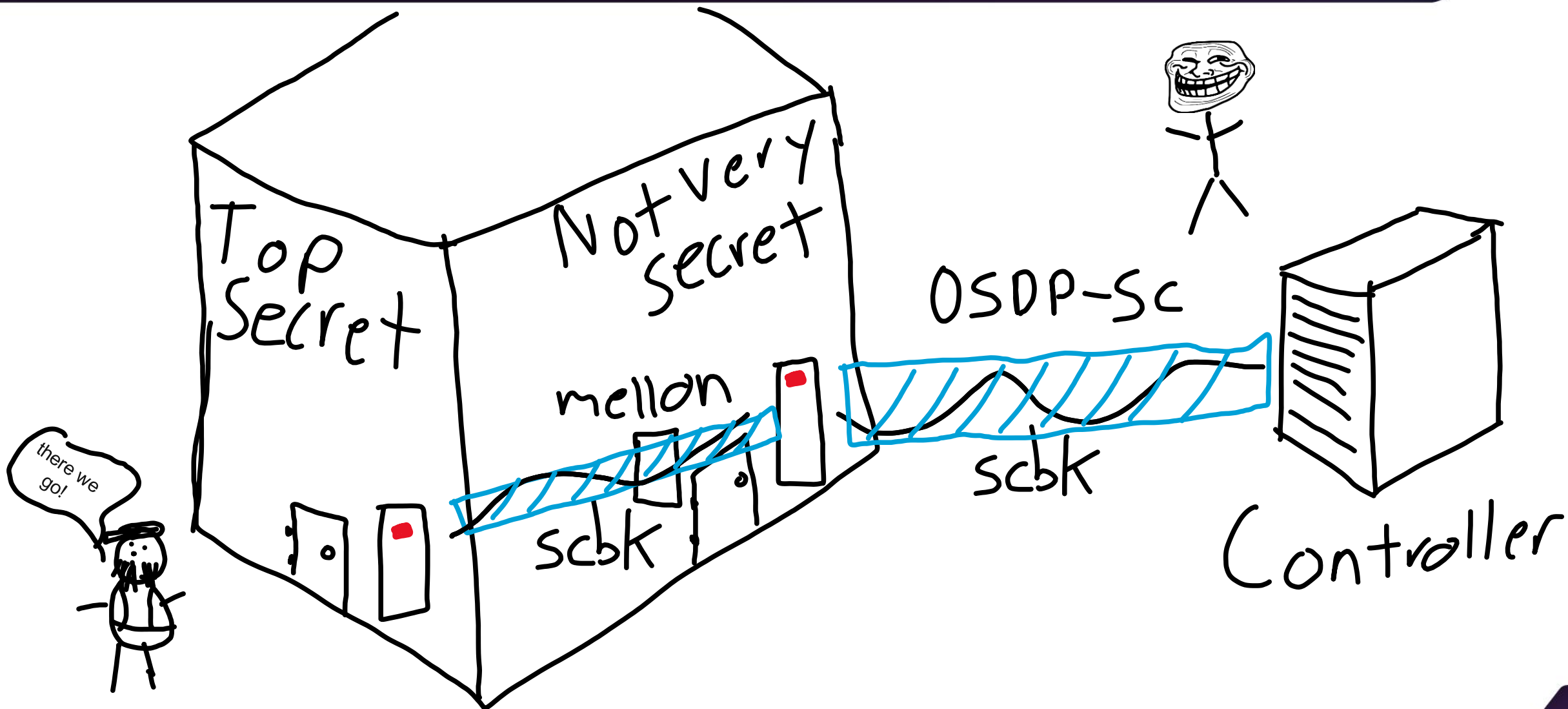
RS-485



RS-485



RS-485



Conclusion



... what do I do?!

Conclusion



Check your configs

Conclusion



Check your configs
✓ use encryption

Conclusion



- Check your configs
- ✓ use encryption
- ✓ Require encryption

Conclusion

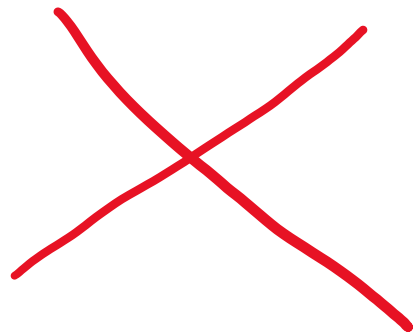


- Check your configs
- ✓ use encryption
- ✓ Require encryption
- ✓ Disable Install Mode

Conclusion



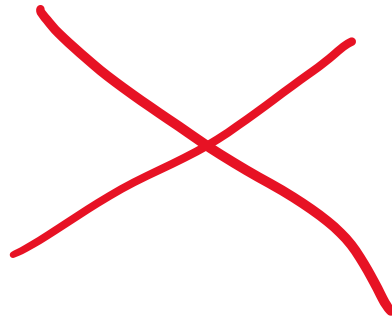
Never configure a reader
in production.



Conclusion



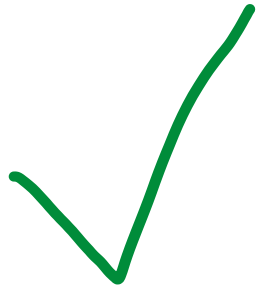
Don't ignore tamper alerts



Conclusion



Buy OSDP Verified Devices



Conclusion



Don't trust

"It's encrypted"



Thanks!